DAVINCI: Decentralized autonomous vote integrity network with cryptographic inference

Pau Escrich[1], Marta Bellés-Muñoz[2], Jordi Piñana[1], Lucas Menéndez[1], Alex Kampa[1], Roger Baig[3], and Jose Luis Muñoz-Tapia[3]

[1] Vocdoni Association
[2] Pompeu Fabra University
[3] Polytechnic University of Catalonia

{pau,painan,lucas,alex}@vocdoni.org, marta.belles@upf.edu
{roger.baig,jose.luis.munoz}@upc.edu

Last update: October 17, 2026

Abstract. DAVINCI is the evolution of the Vocdoni voting protocol, designed to empower civil society by providing essential tools for secure, verifiable, and anonymous digital voting. Using recent advancements in zero-knowledge (ZK) proof systems and blockchain technology, DAVINCI operates as a specialized ZK rollup that inherits network security from its settlement layer, either the Ethereum mainnet or an Ethereum rollup built on the OP Stack. Voters prove that their encrypted ballots are well-formed with a zero-knowledge succinct non-interactive argument of knowledge (ZK-SNARK), and sequencers prove each batch of ballots inside a zero-knowledge virtual machine (zkVM): a single succinct proof attests to the verification of every ballot proof, signature, and census proof in the batch, to the re-encryption of the ballots, and to the resulting update of the election state and of the encrypted tally. The system relies on cryptographic proofs to ensure integrity and security, eliminating the need for centralized authorities. By integrating ZK-SNARKs, a zkVM, and threshold homomorphic encryption, DAVINCI enables end-to-end verifiability, privacy, and trustlessness in election processes. The protocol distributes the election key among a set of key wardens through distributed key generation, coordinates the process through a single registry smart contract, and publishes in Ethereum data blobs enough data for anyone to rebuild the election state. Silent re-randomization of stored ballots makes revoting undetectable, which provides receipt-freeness without a trusted server. With a focus on accessibility, scalability, receipt-freeness, and automation, DAVINCI aims to facilitate high-frequency and low-cost voting, fostering mass adoption of voting and simplifying civil participation. Finally, the design of DAVINCI is grounded in practicality: all components are implemented using current technologies, deployed in production, and evaluated on production-grade hardware. In sum, the proposed architecture is not merely theoretical, but a working system. The source code has been made open source, allowing practitioners and researchers to further investigate its details and potential for future uses.

Table of Contents

1 Introduction **3**
- 1.1 Contributions . . . 3
- 1.2 Related work . . . 4
- 1.3 Paper organization . . . 5

2 Background **5**
- 2.1 Off-chain storage . . . 5
- 2.2 Ethereum . . . 6
- 2.3 Merkle trees . . . 7
- 2.4 Zero-knowledge proofs . . . 7

3 Protocol intuition **9**

4 Voting protocol **11**
- 4.1 Parties involved . . . 11
- 4.2 Smart contracts . . . 12
- 4.3 Census . . . 14
- 4.4 State tree . . . 16
- 4.5 Circuits and zkVM programs . . . 19
  - 4.5.1 Ballot circuit . . . 19
  - 4.5.2 Batch program . . . 21
  - 4.5.3 Results program . . . 27
- 4.6 Protocol flow . . . 29
  - 4.6.1 Election setup . . . 30
  - 4.6.2 Encryption key generation . . . 32
  - 4.6.3 Voting period . . . 33
  - 4.6.4 End of the voting period . . . 37
  - 4.6.5 Tally decryption . . . 38
  - 4.6.6 Election finalization . . . 39

5 Ballot protocol **40**

6 Analysis **43**
- 6.1 Security discussion . . . 43
- 6.2 Implementation . . . 45
- 6.3 Performance evaluation . . . 47

7 Conclusions **48**

Acknowledgments **49**

A Cryptographic primitives **51**
- A.1 Elliptic curves . . . 51
- A.2 Hash functions . . . 52
- A.3 Merkle trees . . . 53
- A.4 Commitment scheme . . . 54
- A.5 Digital signature scheme . . . 54
- A.6 Encryption scheme . . . 54
- A.7 Distributed key generation scheme . . . 56
- A.8 Zero-knowledge proof systems . . . 56

1 Introduction

Online voting remains one of the most studied yet elusive applications in applied cryptography. As digital services expand across both public and private sectors, secure and universally verifiable online voting systems have become an increasingly desirable goal. Remote electronic voting promises scalability, accessibility, and administrative efficiency; yet the design of systems that simultaneously ensure privacy, integrity, coercion resistance, and transparency under realistic adversarial models remains a formidable challenge. Numerous deployments have revealed deep-rooted usability flaws and security gaps, particularly when verifiability mechanisms are either poorly understood by users or reliant on centralized infrastructure.

Against this backdrop, the Vocdoni project was initiated in 2018 with the aim of rethinking online voting from first principles. The name Voĉdoni, meaning *to give voice* in Esperanto, reflects the project's foundational goal: to empower collectives, from small associations to millions of citizens, to engage in secure and verifiable decision-making, regardless of technological or institutional barriers. Central to this vision was the idea that voting is not limited to formal governmental elections but serves as a more general-purpose mechanism for collective signaling. Vocdoni introduced a fully anonymous end-to-end verifiable voting system designed to operate efficiently on a range of devices, including smartphones. To support these goals, the team deployed a custom infrastructure emphasizing resilience, neutrality, and transparency.

Technically, the architecture of Vocdoni was based on a bespoke byzantine fault tolerant (BFT) layer-1 blockchain, named Vochain. At the time, efficient zero-knowledge (ZK) proof systems were emerging, but not yet practical for most deployments. Vochain provided a performant and low-cost environment (achieving approximately 700 transactions per second) in which advanced cryptographic tools could be used without the constraints imposed by general-purpose blockchains based on the Ethereum virtual machine (EVM). The ability to issue voting transactions without requiring user fees enabled broader accessibility. Over several years of development and deployment, this architecture proved both viable and valuable in practice. However, broader adoption as a universal voting protocol highlighted the need for further architectural refinements and stronger formal guarantees.

In this work, we introduce DAVINCI, a new protocol that builds upon the lessons and conceptual groundwork laid by Vocdoni. DAVINCI adopts a modular design and integrates state-of-the-art cryptographic tools, including succinct ZK proofs, zero-knowledge virtual machines, improved bulletin board constructions, and robust coercion resistance mechanisms, reflecting the most recent advances in academic research. Unlike monolithic designs, DAVINCI is conceived as a composable primitive: a foundational layer intended to support secure and verifiable voting in diverse contexts, from blockchain governance to institutional elections. This shift in design philosophy aims to address long-standing challenges in the field, offering a cleaner abstraction with clearer security boundaries and formal underpinnings.

An earlier design of DAVINCI enforced the protocol with a pipeline of four arithmetic circuits over three elliptic curves, recursively composed into one state-transition proof. This document specifies the current design, in which the sequencer-side circuits are replaced by programs written in an ordinary programming language and proven by a zkVM. The statement proven for each batch of ballots is the same in spirit, but it is now expressed as one auditable program, checked by one universal verifier, and extended with guarantees that were impractical to express as circuits, such as the in-proof layout of the published data and the silent re-randomization of stored ballots.

1.1 Contributions

This work introduces DAVINCI, a modular and verifiable protocol for digital voting, designed to support privacy-preserving, auditable, and adaptable election processes across diverse governance contexts. The protocol *models voting as a constrained state machine*, in which each valid operation is expressed as a formally defined state transition, enforced through succinct ZK proofs. At the core of the system lie three proof statements. Voters prove the validity of their encrypted ballots with an arithmetic circuit and a ZK-SNARK, cheap enough to be proven on a smartphone or in a web browser. Sequencers prove each batch of ballots with a batch program executed in a zkVM: the program verifies every ballot proof, signature, and census proof of the batch, re-encrypts the ballots, applies the resulting updates to the election state, and lays out the published data. At the end of the election, a results program proves the decryption of the tally. Application-layer state is maintained off-chain and committed on-chain using Merkle trees as cryptographic accumulators.

A key contribution of this work is the introduction of a generic ballot protocol that abstracts a wide range of voting systems into a unified, parameterized representation. Instead of designing specialized circuits for each voting rule, ballots are expressed as fixed-length vectors subject to configurable constraints, enabling support for approval, ranking, quadratic, single-choice, and multiple-choice elections within the same circuit framework. This design significantly reduces circuit complexity while allowing different tallying and counting rules to be enforced efficiently inside ZK proofs.

A second contribution is *silent revoting*, a receipt-freeness mechanism that does not rely on a trusted server. Voters may overwrite their ballots any number of times, and every batch re-randomizes, in addition to the ballots it writes, a random sample of the ballots already stored in the state. Overwrites and routine refreshes are indistinguishable in the published data, so a voter cannot prove to a third party that a given ballot is the one that counts. The batch proof guarantees that the refreshes are re-encryptions and therefore never alter the tally.

To coordinate protocol execution over a decentralized network of nodes, DAVINCI includes a set of Ethereum smart contracts that mediate the setup of elections, verify submitted proofs of correctness, and manage finalization procedures. A single registry contract holds the state of every election and checks each state transition against the pinned identity of the batch program. The contracts run either on Ethereum or on an Ethereum rollup built on the OP Stack, where settlement is cheaper; in the latter case, the data needed to rebuild the election state is still published in Ethereum blobs, and the registry verifies on the rollup that the blobs are part of the Ethereum chain. The election key can be provided by a threshold committee through distributed key generation (DKG), by an invite-only council, or by a single sequencer when the organizer accepts that trust. A standardized data encoding format supports all stages of the process, including vote casting, ballot batching, and result publication. This approach ensures that protocol interactions remain deterministic and interoperable across clients and backends. Altogether, the design aims to balance auditability and privacy while reducing trust assumptions on infrastructure providers. By cleanly separating functionalities and providing modular interfaces, the protocol is intended to serve as a foundation for both practical deployments and future extensions, such as integration with alternative identity systems, other data availability layers, or other consensus backends.

1.2 Related work

Foundations of cryptographic voting. Cryptographic e-voting originated with Chaum's mix-net construction [18], which achieves anonymity through verifiable shuffling of ciphertexts. Helios [1] was the first deployed web-based system to combine threshold ElGamal encryption with a public bulletin board and homomorphic tallying, and it remains a standard reference point for end-to-end (E2E) verifiability in practice. Belenios [19] refines this design with formal verifiability proofs and a clean separation between the registration authority and the voting server; the BeleniosRF variant additionally achieves receipt-freeness, but only under the assumption that the voting server is honest regarding that property. ElectionGuard [8] belongs to the same family, combining homomorphic tallying with threshold guardians, publicly verifiable proofs, and a centralized election administrator that drives the tally. A common feature of all of these systems is that, beyond their cryptographic core, they require a non-trivial setup of either the bulletin board, the tally authority, or both, and that the operator set is fixed for each election rather than open to participation.

Receipt-freeness and revoting. A receipt-free voting system prevents a voter from producing convincing evidence of their vote, mitigating bribery and simple coercion. Benaloh and Tuinstra [9] gave the first formal notion, and Hirt [29] showed how to achieve it efficiently by combining homomorphic encryption with re-encryption performed by a designated helper. Doan et al. [20] subsequently generalize Hirt-style receipt-freeness to a setting with multiple ballot-processing servers, removing the single-point-of-failure assumption common to earlier non-interactive constructions. These constructions remain server-based: none of them places the protocol on a public bulletin board with cryptographically enforced data availability, and the helpers responsible for re-encryption are selected in advance rather than drawn from an open set.

Blockchain-based and ZK-based voting. McCorry et al. [34] proposed the first self-tallying voting contract on Ethereum, but placing every vote directly on-chain is expensive and creates a linkable trail between voters and their choices. Permissioned-ledger proposals [33] improve operational properties but still rest on

honest-majority assumptions over a closed set of validators. Freedom Tool [41] uses ZK proofs over biometric passports to enforce eligibility, but votes appear on-chain in cleartext, trading ballot secrecy for transparency. MACI [14] uses ZK proofs to discourage bribery, but its coordinator is a single trusted party that can read individual votes; although later versions improve usability, this assumption remains. Semaphore [28] provides anonymous group-membership proofs and is reused as a building block in several voting prototypes, although it is not itself a complete voting protocol. Cicada [25] and Shutter Network [37] use timelock and threshold encryption to keep ballots hidden until the tally, but neither re-randomizes stored ciphertexts after submission. As a result, voters can still prove how they voted, weakening protection against coercion and vote-selling.

Zero-knowledge virtual machines. Rollups move the execution of transactions off-chain and post a validity proof to a settlement chain. Early validity rollups encoded their transition function as hand-written circuits, an approach that is efficient but expensive to audit and to evolve. zkVMs such as RISC Zero [13], SP1 [44], and ZisK [39] instead prove the correct execution of a program compiled to the RISC-V instruction set, using scalable transparent argument of knowledge (STARK)-based proof systems with recursion, and accelerate common operations (hashes, elliptic curve arithmetic) with dedicated precompiles. DAVINCI uses a zkVM to prove its election-specific transition function, which keeps the protocol logic in ordinary, testable code while retaining a constant-size proof and a single on-chain verifier.

1.3 Paper organization

The rest of the paper is organized as follows. In Section 2 we introduce the necessary background. In Section 3 we provide a high-level overview of a voting process, omitting technical details for clarity. In Section 4 we describe the full voting process in detail: the parties, the smart contracts, the census, the state tree, the proof statements, and the protocol flow. In Section 5 we focus on the ballot protocol. In Section 6 we analyze the protocol, covering its security properties, implementation details, and performance evaluation. In Section 7 we conclude the paper with final remarks and future work. Finally, we include Appendix A with the concrete instantiations of all the cryptographic primitives used in the protocol.

2 Background

The DAVINCI protocol builds on several decentralized technologies and advanced cryptographic tools. In this section, we give a high-level overview of these components and their role in the system. We first introduce content-addressed off-chain storage, which serves to distribute large datasets such as censuses and election metadata. We then describe Ethereum, the settlement layer where commitments are published, rules are enforced through smart contracts, and state data is published using data blobs, together with the OP Stack rollups on which DAVINCI can also settle. We continue with the Merkle trees that commit to the census and to the election state. Finally, we present the ZK proof systems that underpin the protocol's integrity and security: ZK-SNARKs and arithmetic circuits, which voters use to prove that their ballots comply with the election rules, and zkVMs, which sequencers use to prove that they processed the ballots correctly.

2.1 Off-chain storage

Large datasets, such as the full census (eligible voters and their weights) and the election metadata (which may include questions, options, images, etc.), do not need to be stored directly on a blockchain. Instead, DAVINCI stores them off-chain and publishes on-chain a short commitment to their content: the root of the census Merkle tree (see Section 2.3) for the census, and the SHA-256 digest of the exact bytes of the metadata document. Anyone who fetches a dataset can recompute the commitment and confirm its integrity, so the transport does not need to be trusted and can be chosen freely, from a conventional web server to a peer-to-peer network.

The interplanetary file system (IPFS) is a natural choice for such a transport. IPFS is a peer-to-peer distributed storage network that retrieves data based on its content rather than on its location. Every file stored in IPFS is split into blocks, each block is hashed, and the resulting cryptographic hash is used as its

unique identifier. This property, known as content addressing, ensures that data is tamper-evident: if the file changes, so does its hash. IPFS also provides deduplication, versioning, and distributed availability (data can be fetched from any node that stores it), which allows it to function as a decentralized content delivery network. In the context of DAVINCI, the census and metadata uniform resource identifiers (URIs) registered on-chain may point to IPFS content (typically through a gateway) or to any other location; in all cases, the on-chain commitment, not the location, is authoritative.

2.2 Ethereum

Ethereum is a decentralized blockchain platform that supports programmable transactions through its built-in execution environment, the Ethereum virtual machine (EVM). All interactions with the Ethereum network take the form of transactions, which must be broadcast to the network, validated by consensus, and permanently recorded on-chain. Every transaction requires the payment of a fee (measured in units of gas), denominated in ETH, to compensate validators for computation and storage. This fee model ensures that resources are used efficiently and prevents denial-of-service attacks by making large or complex operations costly. Each block header commits to the block's transactions through the root of a Merkle Patricia trie (MPT) [55], and to its parent block through the parent's hash, so a chain of headers anchored at a trusted block hash proves that a given transaction belongs to the canonical chain. In the context of DAVINCI, Ethereum serves as the settlement layer where critical commitments are published, ensuring transparency and immutability.

Smart contracts. Smart contracts are programs deployed on the Ethereum blockchain that execute deterministically in response to transactions. Once deployed, they cannot be altered, and their execution is guaranteed by the consensus of the network. In DAVINCI, smart contracts orchestrate the election by managing state transitions and storing critical data such as the current state root and the encryption public key. They enforce the protocol rules in a trustless environment, ensuring that no single participant can manipulate the election. By anchoring these rules in Ethereum smart contracts, DAVINCI guarantees that the election logic is applied consistently and verifiably across all participants.

Data blobs. Data availability is a key requirement for decentralized protocols. EIP-4844 [15] introduces data blobs as a mechanism for publishing large volumes of data alongside transactions at a lower cost than calldata or contract storage. A blob is a vector of 4096 elements of the scalar field of the BLS12-381 curve (128 KiB), interpreted as the evaluations of a polynomial over a multiplicative subgroup of that field. A blob-carrying transaction does not expose the blob contents to the EVM; it exposes only a *versioned hash* of a Kate–Zaverucha–Goldberg (KZG) commitment to the blob polynomial (see Appendix A.4). A contract can nevertheless check a claim about the blob contents with the point-evaluation precompile, which verifies that the committed polynomial evaluates to a given value at a given point. Blobs are kept by the consensus layer for 4096 epochs, approximately 18 days, after which they are pruned; for longer election periods, nodes or third-party archivers must retain them. Since the activation of EIP-7594 [43], a transaction carries at most six blobs. In DAVINCI, sequencers use data blobs to publish the data of each state transition, so that any participant can reconstruct and verify the evolution of the election without overloading Ethereum's permanent storage.

Rollups and the OP Stack. A rollup is a blockchain that executes transactions off the Ethereum mainnet (the *layer 1*) and posts its own transaction data to it, inheriting its data availability and, through fraud or validity proofs, its security. The OP Stack [38] is a widely deployed framework for optimistic rollups (*layer 2* chains), among which is OP Mainnet. Execution on such chains is substantially cheaper than on Ethereum, which makes them attractive for settling frequent state transitions. Two properties of the OP Stack matter for DAVINCI. First, each layer-2 block is derived from a recent layer-1 block, its *L1 origin*, and a system contract (L1Block) exposes the number and the hash of that block to the layer-2 EVM. Second, layer-2 contracts have no access to the versioned hashes of layer-1 blob transactions. DAVINCI therefore supports an *L1-blob mode*, in which the election data is published in Ethereum blobs and the registry contract on the rollup verifies, from a chain of Ethereum block headers anchored at the L1 origin, that the corresponding blob transactions are part of the Ethereum chain (see Section 4.6.3).

2.3 Merkle trees

Merkle trees [35] are cryptographic hash trees that enable compact proofs of data inclusion. Formally, a Merkle tree is a binary tree structure where each leaf node represents the cryptographic hash of a set of data and every non-leaf node is derived from the hash of its children. The apex of the tree, the *Merkle root*, acts as a succinct cryptographic commitment to the entire dataset. To prove that a specific element x exists within the set, a prover provides a Merkle path consisting of the sibling nodes along the path from the leaf to the root. A verifier can reconstruct the root in logarithmic time $O(\log n)$ and check it against the committed state, without requiring access to the full dataset. In DAVINCI, we use two specialized variants of this structure to commit critical data: a sparse Merkle tree for the voting state and an incremental Merkle tree for the voter registry (census).

Sparse Merkle trees. A sparse Merkle tree (SMT) represents a key-value map where each leaf is the hash of a (key, value) pair and each parent node is the hash of its two children. In an SMT, every possible key maps to a unique leaf position determined by the bits of the key, independently of the insertion order, and empty subtrees are represented by a constant. The proof of a leaf's membership or non-membership therefore has a bounded length and can be verified by a fixed procedure, which makes the structure well suited to ZK proofs. In DAVINCI, the state tree is an SMT of depth 64 indexed by 64-bit keys. Configuration parameters occupy reserved keys, each voter's encrypted ballot is stored at a key derived from the voter's identity, and each one-time vote identifier is recorded at its own key. Because the SMT covers a vast key space, a user or sequencer can prove that a given key is present in the state (or conversely, that it remains empty) by providing a short Merkle proof against the tree's root, rather than revealing the entire state. This enables compact proofs of state correctness, for instance showing in ZK that a vote identifier has not appeared before or that a new ballot's ciphertext is correctly inserted into the state. The SMT used in DAVINCI is arbo [46], whose structure and proof format follow the circomlib sparse Merkle tree specification [2], instantiated with the SHA-256 hash function. SHA-256 is cheap both on the EVM, where it is a precompile, and in the zkVM, where it is an accelerated operation, so the same tree can be updated inside a proof and checked by a smart contract.

Incremental Merkle trees. The census (voter registry) is maintained with an incremental Merkle tree (IMT), a binary Merkle tree designed for efficient sequential updates. Unlike a sparse tree, the IMT grows in height only as needed with the number of leaves and eliminates the overhead of hashing dummy "zero" siblings. In this scheme, voters are assigned sequential leaf indices in a continuously evolving tree, rather than being placed at cryptographic key-derived positions. In DAVINCI, the census Merkle tree is a lean IMT [40] hashed with Poseidon: each eligible voter's address and voting weight are stored in the next available leaf, and only the Merkle root of this tree (the `censusRoot`) is published on-chain. This design makes membership updates and proof generation extremely low-cost for the census. Appending a new voter requires recomputing hashes only along the single path from the new leaf to the root, and the tree's depth adjusts optimally (e.g., a tree of M voters has height $\lceil \log_2 M \rceil$, instead of a fixed large height). Consequently, proving one's inclusion in the voter list is more efficient than it would be with an SMT: the Merkle proof is shorter and involves no unnecessary default nodes. A lean IMT root does not commit to the number of leaves, and a node with a single child is promoted to the next level instead of being hashed with an empty sibling, so a membership proof does not pin the position of its leaf. For this reason, the slot where a voter's ballot is stored is derived from the voter's address, never from the voter's position in the census (see Section 4.4).

2.4 Zero-knowledge proofs

Zero-knowledge (ZK) proofs are cryptographic protocols that allow one party (the prover) to convince another (the verifier) that a certain statement is true, without revealing any information beyond the validity of the statement itself. In the context of voting, this enables participants to prove that their encrypted ballot is well-formed and complies with the election rules, without disclosing their actual choice.

ZK-SNARKs. We focus on a specific type of proof system called zero-knowledge succinct non-interactive arguments of knowledge (ZK-SNARKs). Informally, while a ZK proof convinces the verifier that a valid witness exists, an argument of knowledge additionally ensures that the prover actually knows such a witness,

except with negligible probability. ZK-SNARKs extend this notion with two crucial properties: they are non-interactive, requiring only a single message from prover to verifier, and succinct, meaning that proof size and verification cost remain small regardless of the size of the underlying statement. For instance, Groth16 proofs [27], which rely on pairing-based cryptography over elliptic curves, are only about 200 bytes long and can be verified in a few milliseconds, and PlonK proofs [23] are less than a kilobyte long and are verified on the EVM for a few hundred thousand gas. These properties make ZK-SNARKs particularly well-suited for blockchain applications, where verification cost and data size are critical. In DAVINCI, voters generate Groth16 proofs to show that their encrypted ballots are correct, and the proofs of the sequencers reach the smart contracts as PlonK proofs (see the paragraph on zkVMs below). In essence, ZK-SNARKs enable both voters and sequencers to produce compact proofs that can be checked on-chain, ensuring that all protocol rules are followed without requiring trust in any party.

Trusted setup. A limitation of some ZK-SNARK protocols is their reliance on a common reference string (CRS) generated during a so-called trusted setup. The CRS is built from random values that must not be known to either the prover or the verifier. To achieve this, the CRS can be created by a trusted third party or, more robustly, via a secure multi-party computation (MPC) protocol [16]. In an MPC setup, it suffices that at least one participant discards their secret contribution to ensure the integrity of the whole ceremony. Groth16 requires a setup specific to each circuit, preceded by a universal *powers of tau* phase, whereas PlonK requires only a universal setup that serves every circuit up to a given size. STARKs require no trusted setup at all. In DAVINCI, only the ballot circuit and the final PlonK wrapper of the zkVM depend on a structured reference string (see Section 6.2 and Appendix A.8).

Arithmetic circuits. In general, ZK-SNARKs operate by proving the satisfiability of an arithmetic circuit. An arithmetic circuit (from now on also called circuit or ZK circuit) is a directed acyclic graph where nodes represent addition or multiplication gates over a finite field. The inputs and outputs of the circuit correspond to the public inputs and private witnesses of the computation, while the intermediate wires carry intermediate values. Any deterministic computation, from verifying an encryption to checking Merkle proofs, can be compiled into an arithmetic circuit [24]. In practice, proving with ZK-SNARKs means showing knowledge of a secret witness (e.g., a plaintext ballot or encryption randomness) that satisfies all the equations encoded in the circuit. For example, the ballot circuit in DAVINCI encodes the constraints that a ciphertext is well-formed, that the vote identifier is correctly derived, and that the ballot complies with the rules of the voting mode. The prover generates a proof of satisfiability, while the verifier only checks the proof against the public inputs, without learning the secret witness. Circuits are compact for algebraic statements over their native field, such as elliptic curve operations on an embedded curve, but they become large and difficult to audit for statements with heavy control flow, variable-length data, or arithmetic over foreign fields, which is precisely the nature of the work of a sequencer.

Zero-knowledge virtual machines. A zkVM proves the correct execution of a program rather than the satisfiability of a hand-written circuit. The program, called the *guest*, is written in a conventional language (Rust, in DAVINCI), compiled to a standard instruction set such as RISC-V [53], and executed on an input supplied by the *host*. The zkVM proves that the guest ran to completion on some input and committed a given vector of *public values*; the input itself remains private. Formally, for a program $\mathcal{P}$ we write $\mathsf{VM.Prove}(\mathcal{P}, \mathsf{witness}) \to (\texttt{publicValues}, \pi)$ and $\mathsf{VM.Verify}(\texttt{programVK}, \texttt{publicValues}, \pi) \to \{\texttt{true}, \texttt{false}\}$, where `programVK` is a verification key that identifies $\mathcal{P}$ (a commitment to its compiled code). A verifier that pins `programVK` therefore pins the exact program whose execution is being proven, and any change to the program changes its key.

DAVINCI uses the ZisK zkVM [39], which proves RISC-V executions with FRI-based STARKs [7,6] over the Goldilocks field and accelerates hashing (SHA-256, Keccak), modular arithmetic, and elliptic curve operations (SECP256K1, BN254, BabyJubjub) with precompiles. ZisK splits an execution into segments proven by specialized state machines, aggregates the resulting proofs recursively into a single *VADCOP-final* STARK, and, for on-chain verification, wraps that proof in a PlonK proof over BN254. The PlonK verifier takes a single public input, the SHA-256 digest of the program key, the public values, and the root of the VADCOP-final setup, `rootCVadcopFinal`, so one deployed verifier contract serves every program. STARKs are transparent and rely only on collision-resistant hashing; the PlonK wrapper is the only part of the sequencer's proof that depends on a structured reference string and on pairings over BN254.

3 Protocol intuition

The following section provides an overview of the DAVINCI protocol, describing its main phases and actors without delving into technical details. An election is governed by smart contracts deployed on a settlement chain, either Ethereum or an Ethereum rollup built on the OP Stack, which ensure transparency, enforce rules, and verify proofs. As shown in Fig. 1, the protocol consists of five main phases, from the setup of the election to the publication and verification of the final results. The numbered tags of the figure match the phases described below.

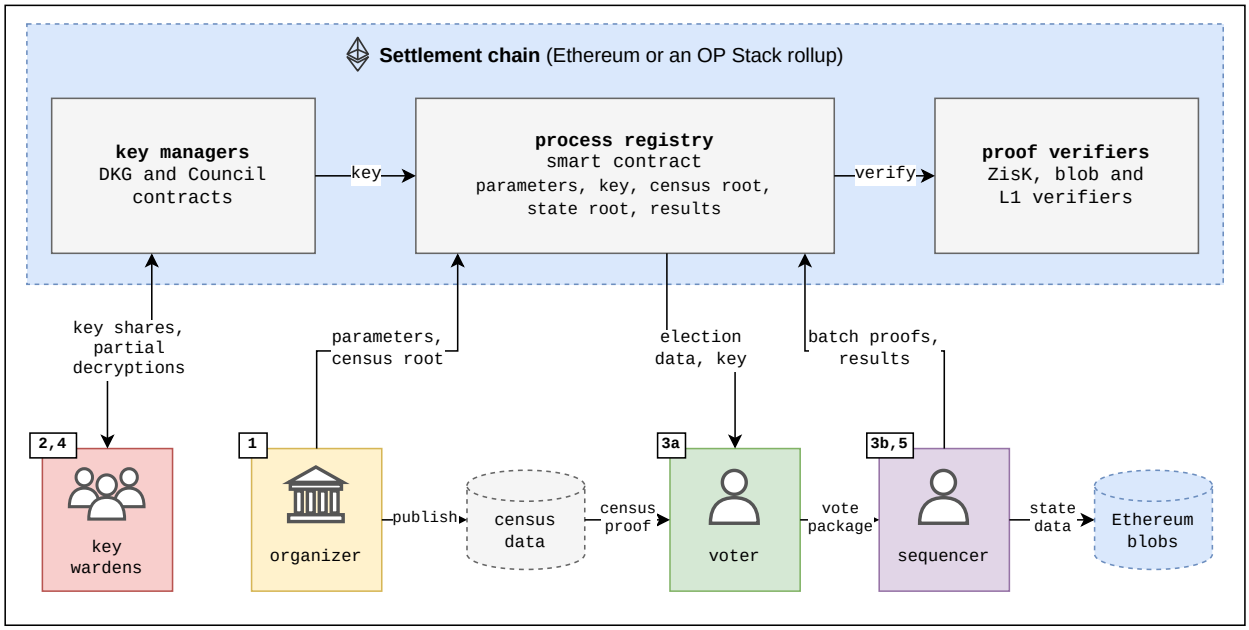


Fig. 1: Overview of the protocol flow. Tags mark the phases: (1) election setup, (2) encryption key generation, (3a) vote casting and (3b) vote batching during the voting period, (4) tally decryption, and (5) election finalization.

1. *Election setup.* In the first phase, the *organizer* gathers all census data and generates a cryptographic commitment to this data. Additionally, the organizer defines the voting parameters, including the number of ballot fields, the rules a valid ballot must satisfy (e.g., single choice, approval, weighted or quadratic voting), the voting period, and the maximum number of voters. The organizer also chooses who will hold the election key, as described in the next phase. Once these details are set, the organizer submits a transaction to the process registry smart contract. This transaction publicly records the voting parameters and the census commitment, and derives from them the initial state of the election, ensuring transparency and preventing any subsequent alterations to the voting setup.

2. *Encryption key generation.* A designated group of participants, referred to as *key wardens*, collaboratively generate a shared encryption public key, which voters will use to encrypt their votes. This key is established through a DKG protocol, ensuring that no single party can control or reconstruct the corresponding private key independently. In DAVINCI, the key wardens are either a committee of permissionless operators, which prepares encryption keys in advance and hands one to each new election, or a council of members invited by the organizer. For elections in which the organizer accepts a weaker trust model, a single sequencer can also provide the key. In every case, the encryption public key is recorded in the process registry when the election is created, and the voting period can start.

3. *Voting period.* During the voting period, two actors interact continuously: *voters*, who cast their encrypted ballots, and *sequencers*, independent entities responsible for collecting, verifying, and processing these ballots before committing them to the shared state. Both processes, (a) ballot submission by voters and (b) vote batching by sequencers, occur continuously and in parallel throughout this phase. The voting period remains open until the deadline defined by the organizer, allowing voters to submit or overwrite ballots at any time, while sequencers periodically process new submissions and update the election state accordingly.

   (a) *Vote casting.* Voters select their preferred choices according to the voting rules established by the organizer and captured in the process registry. Instead of submitting their votes directly on-chain, they send them off-chain to a sequencer of their choice for processing. To ensure privacy, votes are encrypted using the election's encryption public key. Additionally, voters compute a unique vote identifier that will allow them to verify that their vote has been included in the final result. Alongside the encrypted vote and the vote identifier, each voter must also provide the following data: a proof of valid voting, demonstrating that the vote complies with the election rules; a proof of eligibility, verifying that they are registered in the census; and a proof of identity ownership, in the form of a digital signature, confirming that they are the legitimate voter and are not impersonating someone else. To mitigate coercion and vote-buying, the protocol allows voters to overwrite their vote any number of times during the voting phase; only the last ballot counts.

   (b) *Vote batching.* During this phase, sequencers collect encrypted votes from multiple voters along with their corresponding proofs, and verify the validity of these submissions. That is, sequencers verify the signatures, to ensure the votes were cast by legitimate voters; they verify the proof of compliance, confirming that each vote adheres to the election voting rules; and they verify the census membership proofs against the commitment to the census data that was originally registered in the process registry. Valid votes are grouped into batches. For each batch, the sequencer re-encrypts the votes, so that voters can no longer decrypt or recognize their own ballots, and also re-encrypts a random sample of the ballots already stored in the state, so that an observer cannot tell a voter who overwrote a ballot from one whose ballot was merely refreshed. While this process does not alter the final tally, it prevents voters from proving their choice to third parties, which mitigates vote selling and coercion. The sequencer then updates the election state and the encrypted tally, and publishes the data of the batch in Ethereum data blobs. Instead of submitting individual verifications for each vote, the sequencer generates a single ZK proof that attests to the correctness of the whole batch: every verification, every re-encryption, the state update, and the published data. Finally, the sequencer submits the proof and the new state root to the process registry, which verifies the proof and the blobs and accepts the new state. Settlement is permissionless: any sequencer may submit the next state transition, and the first valid one to land advances the election.

4. *Tally decryption.* At the end of the voting period, sequencers stop accepting votes and settle the batches they still hold during a short grace window, after which the registry ceases to accept new state updates. Then, a threshold number of key wardens cooperate to decrypt the final tally. That is, instead of reconstructing the election's private key, each key warden publishes a partial decryption together with a proof that the contribution is correct. When the key is held by a single sequencer, that sequencer decrypts the tally itself and proves the decryption. This way, the individual votes remain encrypted throughout the voting period, and only the final aggregated result is decrypted at the end of the election.

5. *Election finalization.* The results and a proof of correctness are submitted to the process registry. The registry checks that the decrypted tally corresponds to the encrypted tally committed in the final state: either by verifying a ZK proof of the decryption, or by binding the final encrypted tally to the state root before handing it to the key wardens and reading the decryptions they proved. Once this verification succeeds, the results and the latest state root remain available on-chain, providing an immutable record of the election outcome that can be independently audited by anyone.

4 Voting protocol

This section outlines the DAVINCI voting protocol. First, we introduce the parties involved in the process in Section 4.1. In Section 4.2 we present the smart contracts that rule the DAVINCI protocol. Then, in Section 4.3, we describe the census data structures and in Section 4.4 the state Merkle tree. In Section 4.5, we present the proof statements that enforce the protocol: the ballot circuit proven by voters, and the batch and results programs proven by sequencers in a zkVM. Finally, in Section 4.6, we detail the full protocol flow step by step, from the voting setup to the results validation and process finalization.

4.1 Parties involved

An election involves four types of participants: organizer, key wardens, voters, and sequencers. In addition, any party can follow an election as an observer.

Organizer. The organizer is the entity responsible for defining and setting up the election. Its key responsibilities include defining the voting parameters, gathering the census data, and choosing who holds the election key. The organizer is identified by the Ethereum address that creates the election, and only that address can later change the parameters that the protocol allows to be changed (the status, the end time, the census of a dynamic census, the metadata, the maximum number of voters, and the grace window). The organizer ensures that the election is structured correctly but does not participate in vote collection or tallying.

Key wardens. Key wardens are a set of decentralized parties that collaboratively generate a public encryption key that voters use to encrypt their votes. The corresponding secret key is secret-shared among them so that any θ of the ν key wardens, for a threshold θ, can later collaborate to decrypt the final tally. Individual ballots remain private because the protocol only invokes threshold decryption on the final aggregate, not on each ballot separately. DAVINCI supports two kinds of key wardens: a *DKG committee*, drawn by lottery from a permissionless set of registered operators, which prepares a pool of keys in advance and assigns one to each election; and a *council*, a small group of members invited by an organizer, which runs a dedicated key ceremony. When the organizer accepts the corresponding trust, a single sequencer can play the role of the key wardens and hold the whole election key (see Section 4.6.2).

Voters. Voters are the participants that belong to the census and are allowed to cast their votes in the election. Each voter holds a SECP256K1 key pair, whose Ethereum address identifies the voter in the census, and is assigned a weight, which can bound their vote according to the ballot rules. Voters may cast as many ballots as they wish during the voting period; each new ballot overwrites the previous one.

Sequencers. Sequencers are a set of parties that during the voting period receive and verify encrypted ballots from voters, re-encrypt votes, and update the shared public state accordingly. A sequencer groups valid ballots into batches, proves each batch with the batch program in a zkVM, publishes the batch data in blobs, and submits the proof to the process registry. The role is permissionless: any party with an Ethereum account can run a sequencer for any election, and several sequencers can serve the same election concurrently, in which case they race to settle the next state transition. Proof generation is computationally intensive and is typically delegated to a GPU *prover* service. The prover learns the private inputs of the batch, including the re-encryption secrets, so a sequencer must run its own prover or trust its operator with ballot privacy (but not with integrity, which the proof guarantees).

Observers. Any party can run a sequencer node without a signing key. Such a node, called an observer, never settles transitions, but it replays every transition settled by others from the published blobs, checks that the resulting state roots match the ones recorded on-chain, and serves the state of the election, including the inclusion proofs that voters use to check that their ballots were recorded. Observers therefore provide independent verification of the election and of its tally.

4.2 Smart contracts

The DAVINCI protocol operates on an Ethereum-compatible network, which serves as the source of truth for the election. By leveraging an EVM blockchain, all election transitions become immutable and publicly verifiable. To coordinate and validate each phase of the process, DAVINCI deploys a set of smart contracts that enforce protocol rules, verify ZK proofs, and maintain on-chain commitments such as state roots, encryption keys, and final results. Together, these contracts provide a trustless execution environment that guarantees the correct and transparent progression of the election without relying on any centralized authority. The contracts are deployed either on Ethereum, where they read the blobs of their own chain, or on an OP Stack rollup in L1-blob mode, where they verify blobs published on Ethereum. Fig. 2 shows the contracts and their dependencies.

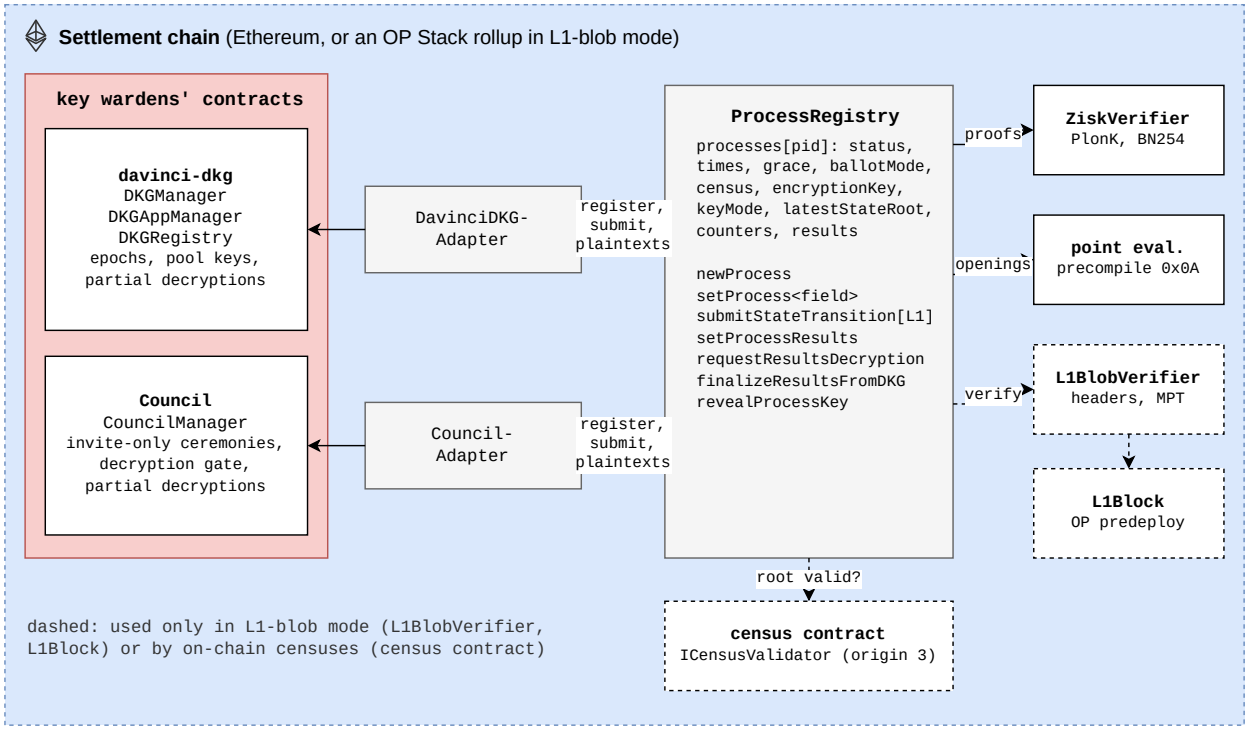


Fig. 2: The DAVINCI smart contracts and the calls between them. The process registry is the only contract that organizers, sequencers, and voters interact with; it delegates proof verification, blob checks, census root checks, and threshold decryption to the other contracts.

Process registry. The ProcessRegistry smart contract manages the lifecycle of every election (a *process*), from its creation to the registration of final results. It stores, for each process, the record summarized in Table 1, and it ensures that all transitions follow the protocol. A process is identified by a 31-byte `processID` that concatenates the 20-byte address of its creator, a 4-byte prefix taken from the last four bytes of Keccak(`chainID`||`registryAddress`), and a 7-byte per-creator nonce. The prefix binds identifiers to one registry on one chain, and the nonce makes the identifier of the next process of an organizer predictable, which lets the organizer obtain an encryption key for it before creating it. At creation, the registry validates the parameters, obtains the encryption key, and computes the genesis root of the state tree (see Section 4.4) on-chain, so the first state transition must start from a root that the registry derived itself.

The registry is deployed with immutable parameters that pin the cryptographic identity of the protocol: the address of the ZisK verifier; the program verification keys of the batch program and of the results program; the root `rootCVadcopFinal` of the ZisK setup under which both are wrapped; and `ballotVKHash`, the SHA-256 digest of the verification key of the ballot circuit. A state transition or a results proof is accepted

only if it was proven by the pinned program under the pinned setup, and a batch is accepted only if it verified ballot proofs under the pinned ballot key. The registry is also deployed with the parameters of the grace window (see Section 4.6.4) and, optionally, with the addresses of a DKG manager, a council manager, and an L1 blob verifier. It has no owner and cannot be upgraded: a new protocol release is deployed as a new registry.

Table 1: Per-process record stored by the process registry.

Field	Content
`status`	One of `READY`, `PAUSED`, `ENDED`, `CANCELED`, `RESULTS` (see Fig. 16).
`organizationId`	Address of the creator, the only account allowed to change the process.
`startTime`, `duration`	Voting period; the end time is `startTime` + `duration`.
`maxVoters`	Maximum number of distinct voters (ballot slots) the process accepts.
`ballotMode`	Ballot protocol parameters (see Section 5).
`census`	Census origin, root, URI, and, for an on-chain census, contract address.
`encryptionKey`	BabyJubjub public key under which ballots are encrypted.
`keyMode`, `dkgEpochId`, `dkgAid`	Who holds the election key, and the identifiers of the key in the DKG or council manager.
`latestStateRoot`	Root of the state tree after the last settled transition.
`votersCount`, `overwrittenVotesCount`	Number of distinct ballot slots written, and number of overwrites.
`batchNumber`, `lastVoteAt`	Number of settled transitions, and time of the last one.
`creationBlock`	Block of creation; lower bound for the roots an on-chain census may use.
`grace`	Idle window during which transitions keep settling after the end.
`metadataURI`, `metadataHash`	Location and SHA-256 digest of the metadata document.
`result`	Decrypted tally, one value per ballot field.

ZisK verifier. The ZiskVerifier contract verifies the PlonK proofs that wrap the STARK proofs of the zkVM. It exposes a single function,

$$\texttt{verifySnarkProof}(\texttt{programVK}, \texttt{rootCVadcopFinal}, \texttt{publicValues}, \pi),$$

which reverts unless π is a valid PlonK proof for the public input

$$\mathsf{SHA256}(\texttt{programVK} \,||\, \texttt{publicValues} \,||\, \texttt{rootCVadcopFinal}) \bmod s,$$

where s is the order of the BN254 scalar field. The public values are the 64 output words of the guest program (512 bytes) and the proof is 768 bytes long, independently of the batch size. Because the program key and the setup root are part of the hashed input, the same verifier contract serves the batch program and the results program, and a proof of any other program or setup fails. The contract is generated by the ZisK setup and contains no DAVINCI-specific logic; sequencers check the hash of its code against a pinned value before trusting a registry.

Point-evaluation precompile. The point-evaluation precompile of EIP-4844, at address `0x0A`, takes a versioned hash, an evaluation point z, a claimed value y, a KZG commitment, and an opening proof, and succeeds only if the commitment matches the versioned hash and the committed blob polynomial evaluates to y at z. The registry calls it once per blob of every state transition, with the point and value that the batch program derived and committed to (see Section 4.5.2). The precompile is available on Ethereum and on OP Stack chains alike.

L1 blob verifier. The L1BlobVerifier contract is used only by registries deployed in L1-blob mode on an OP Stack rollup. It proves on the rollup that a set of Ethereum blob transactions belongs to the canonical Ethereum chain, and returns the versioned hashes they carry. Its input is a list of Ethereum block headers,

newest first, and a list of transactions, each with a MPT proof against the transactions root of one of the headers. The verifier hashes every header, checks that each one is the parent of the previous one, and requires one of them, the *anchor*, to be either the current L1 origin exposed by the L1Block system contract or an origin recorded earlier. Every transaction must sit in the anchor or in an older header, at most `MAX_AGE` = 7200 Ethereum blocks (one day) behind the current origin, and must be a blob-carrying (type-3) transaction, whose list of versioned hashes is returned together with the block number of each. Because the L1 origin advances every few seconds, the verifier also offers a permissionless function, `record`, that stores the current origin's number and hash, so that a proof can later be anchored at that block. The contract has no owner and no other state.

Key management. Elections whose key is held by key wardens rely on external key-management contracts, reached through adapters that the registry deploys in its constructor and that are the only accounts allowed to request decryptions on its behalf. The DavinciDKGAdapter connects the registry to the davinci-dkg contracts [50]: the DKGRegistry, which records the operators eligible for committees; the DKGManager, which runs the epochs in which a committee jointly generates a pool of keys and later decrypts ciphertexts; and the DKGAppManager, which assigns pool keys to applications (here, elections). The CouncilAdapter connects the registry to the CouncilManager [49], which runs invite-only key ceremonies and their decryptions. Both managers verify a ZK proof for every contribution to a key and for every partial decryption, so the registry can read their decrypted values without trusting the parties who submitted them. A registry deployed without a manager simply disables the corresponding key modes.

Census contract. This contract is only used by the on-chain census model (see Section 4.3), since the off-chain and credential service provider (CSP) models store the census commitment directly as a parameter and require no such contract. An on-chain census implements the ICensusValidator interface: it maintains the census as a lean IMT that only grows, and exposes the current census root and, for any root, the last block at which that root was current (zero for a root it never held). During each state transition, the ProcessRegistry queries the census contract to confirm that the `censusRoot` used in the membership proofs was current at some block no older than the election's creation block, rejecting the transition otherwise. This lets the census grow during the voting period while ensuring sequencers can only prove membership against roots the contract recognizes.

4.3 Census

The census defines the set of eligible voters and their associated voting weights. At its core, a census is simply a dataset that, for each voter, contains a unique identifier (an Ethereum address), a voting weight, and a means to produce a membership proof that can be verified inside a ZK proof. DAVINCI supports two mechanisms for generating such proofs:

- *Merkle-tree-based membership proofs.* The census is organized as a lean IMT hashed with Poseidon (see Appendix A.3) and denoted by $\mathsf{MT}^{\mathtt{census}}$. Each voter is stored as one leaf that packs the voter's address and weight into a single field element,

$$\mathsf{leaf} = \mathtt{address} \cdot 2^{88} + \mathtt{weight}, \qquad 0 \le \mathtt{weight} < 2^{88}, \tag{1}$$

  and a membership proof (`censusProof`) is a Merkle path from that leaf to the committed census root. Sequencers keep a copy of the census, derive the membership proofs themselves, and serve them to voters who need their weight to build a ballot.
- *Credential-based membership proofs.* Eligibility is certified by a digital signature issued by a CSP. The CSP assigns a unique index (`idx`) to each voter and signs it alongside the process identifier (`processID`), the voter's identifier (`address`), and their weight (`weight`). The voter attaches the credential to the ballot.

In both mechanisms, the census only certifies eligibility and weight; where the voter's ballot is stored is determined by the voter's address (Merkle censuses) or by the index assigned by the CSP (credential-based censuses), as described in Section 4.4. To support the different census models, the ProcessRegistry smart contract stores the following parameters:

- `censusOrigin`: the census model, one of the four values of Table 2.
- `censusURI`: a URI pointing to an external reference (URL, CDN path, IPFS gateway, etc.) from which sequencers can download the census data or, for a CSP census, where voters obtain their credentials.
- `censusRoot`: the on-chain commitment used for membership verification. Its interpretation depends on the `censusOrigin`. For Merkle censuses, it is the root of the census tree; in the CSP model, it is the Ethereum address of the CSP's signing key, read as a 160-bit integer.
- `contractAddress`: for the on-chain model only, the address of the census contract.

Organizers are free to construct the census from various data sources, such as private membership registries, self-sovereign identity credentials, or Ethereum-based tokens (e.g., ERC-20 or NFTs) whose balances define eligibility. A token-based census is naturally built as an on-chain census, by a contract that adds members to its tree according to their holdings. This flexibility supports a broad range of use cases and governance scenarios. The supported census models are summarized in Table 2 and described in detail below. Whichever model is used, the batch program checks every voter's eligibility inside the proof, and the registry checks that the census root used by the proof is one the election accepts.

Off-chain static census (origin 1). The organizer constructs a fixed census prior to the election. The Merkle tree is built locally and the resulting root is stored in `censusRoot`, and `censusURI` specifies the endpoint from which sequencers can retrieve the full list of members, from which they rebuild the tree and check its root. Since the census is static, no updates occur during the voting period, and every state transition must prove membership against the fixed root.

Off-chain dynamic census (origin 2). The Merkle tree remains off-chain, but the organizer may publish a new version of it during the voting period, by replacing `censusRoot` and `censusURI` in the ProcessRegistry. The registry accepts replacements only from the organizer, only while the process is ready or paused, and only before its end time. From then on, a state transition must prove membership against the latest root; batches proven against an older root are rejected and must be rebuilt. Ballots that were already settled remain in the state and count, even if their voters are absent from the new census, so organizers are expected to use this model to append voters, not to remove them.

On-chain census (origin 3). In this configuration, the census is managed by a dedicated census contract (ICensusValidator) that supports adding new members during the voting period, for instance as tokens are acquired. The address of this contract is stored in the process; the registry stores the contract's root at creation for reference only. The census contract keeps a history of the roots it has held, never evicts them, and never changes the weight of an existing member. A state transition may therefore prove membership against any root that the contract held at some block between the election's creation block and the current block, and the ProcessRegistry queries the contract during each transition to verify this condition. Sequencers rebuild the tree by replaying the contract's member additions.

Credential service provider (origin 4). Voter eligibility is certified by a digital signature issued by a CSP. A voter obtains a signed credential from the CSP and attaches it to the ballot, and the batch program verifies the signature inside the proof. The credential is an ECDSA signature over SECP256K1 (see Appendix A.5), in the Ethereum personal-message format, over the 92-byte message

$$\texttt{processID}_{32} \,||\, \texttt{address}_{20} \,||\, \texttt{weight}_{32} \,||\, \texttt{idx}_{8}, \tag{2}$$

where subscripts denote big-endian byte lengths and `idx` is a unique index assigned by the CSP to each voter. In this model, `censusRoot` stores the Ethereum address of the CSP's signing key, and `censusURI` specifies the endpoint where voters obtain their credential. The batch program recovers the signer's key from each credential and requires all the credentials of a batch to recover the same key, whose address it publishes in place of a census root. This model is suitable when eligibility is managed externally through attested identities rather than by maintaining a Merkle-tree dataset; its main limitation is that a credential, once issued, cannot be revoked.

Table 2: Summary of census configurations supported by the DAVINCI protocol.

Origin	Model	`censusRoot`	`censusURI`	Root a state transition may use
1	Off-chain static	Fixed Merkle root	Endpoint to obtain the census	The root fixed at creation
2	Off-chain dynamic	Latest Merkle root, replaced by the organizer	Endpoint to obtain the latest census	The latest root only
3	On-chain	Census contract (ICensusValidator)	Endpoint to obtain the census (sequencers rebuild it from the contract's events)	Any root the contract held since the creation block
4	Credential service provider (CSP)	Address of the CSP's signing key	Endpoint to obtain an eligibility credential	The CSP address (signatures recovered in the proof)

Uniqueness of ballot slots. Since a voter's ballot slot is derived from the voter's address by a hash truncated to 63 bits (Section 4.4), two distinct members of a Merkle census could, in principle, be assigned the same slot, and the ballot of one would then overwrite the ballot of the other. Finding an address whose slot collides with one of M existing members requires about $2^{63}/M$ key generations, which is feasible for a determined attacker when M is large. Slot uniqueness is therefore part of the census model: on-chain census contracts reject a registration whose slot is already taken, sequencers refuse any off-chain census that contains two members with the same slot, and the batch program rejects a batch that writes the same slot twice. For a CSP census, the slot is derived from the index, and the CSP must assign a distinct index to each voter.

4.4 State tree

The state Merkle tree, denoted by $\mathsf{MT}^{\mathtt{state}}$, represents the evolving global state of an election in DAVINCI. It is implemented as a SMT of fixed depth $D = 64$, as described in Appendix A.3. The state tree acts as a cryptographic accumulator that compactly commits to all relevant election data, including configuration parameters, vote identifiers, encrypted ballots, and the encrypted tally. By organizing the election state in a single SMT, DAVINCI supports efficient membership and update proofs that can be verified inside ZK proofs, enabling succinct and verifiable state transitions. Its structure is depicted in Fig. 3.

The root of the state tree, denoted `stateRoot`, is committed on-chain and serves as the authoritative reference to the current election state. Each state transition performed by a sequencer updates the tree and produces a new root. The correctness of this update, namely, that it follows the protocol rules and correctly incorporates new votes, is attested by a proof of the batch program, which the ProcessRegistry smart contract verifies before the new root is accepted.

Hashing and encoding. The state tree uses the SHA-256 hash function (SHA256, see Appendix A.2), which both the zkVM and the EVM evaluate efficiently. A leaf with key κ and value v is hashed as $\mathsf{SHA256}(\kappa_{\mathrm{LE8}} \,||\, v_{\mathrm{LE32}} \,||\, \mathtt{0x01})$, where the subscripts denote the little-endian encoding of the key in 8 bytes and of the value in 32 bytes, and an internal node with children L and R is hashed as $\mathsf{SHA256}(L \,||\, R)$. The path of a key is given by its bits, least significant first. Values that are themselves digests, such as the hash of an encrypted ballot, are SHA-256 outputs read as big-endian integers. The state root is the raw 32-byte digest of the root node, and it is stored as such in the registry. Following the arbo convention, a digest denotes the integer of which it is the little-endian encoding; when the root is used as an integer, for instance in the evaluation points of the blobs, its value is read in little-endian order.

Namespaced key space. The state tree operates over keys in the range $[0, 2^D - 1]$. To prevent collisions between different categories of data stored in the state, the 64-bit key space is partitioned into three disjoint numeric namespaces using a fixed threshold N. We define $N = 2^{D-1}$, which we use to split the key space into a lower region for configuration parameters and encrypted ballot storage, and an upper region strictly reserved for vote identifiers. Within the lower region, the first sixteen keys are reserved for configuration. This strict separation ensures that configuration entries, vote identifiers, and ballots never collide.

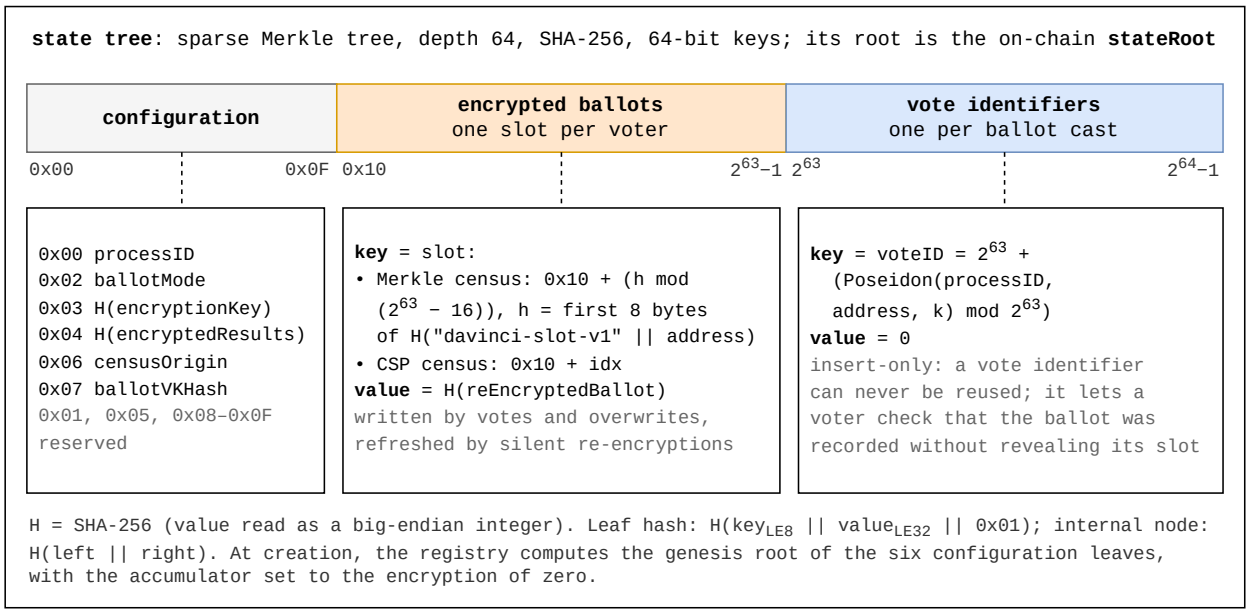


Fig. 3: Structure of the state Merkle tree. The 64-bit key space is split into three namespaces: configuration parameters, encrypted ballot slots, and vote identifiers.

Configuration namespace. The lowest indices of the tree are reserved for a small set of process parameters. Most are configuration values fixed at election setup and never modified thereafter (`processID`, `ballotMode`, `encryptionKey`, `censusOrigin`, `ballotVKHash`); the remaining entry is a mutable accumulator (`encryptedResults`) updated on every state transition. In all cases these keys occupy deterministic, well-known positions and use a negligible portion of the key space. The entries are:

- `0x0`: the election identifier (`processID`), read as a 248-bit integer. [4]
- `0x2`: the ballot mode configuration, which is the set of rules for validating votes (`ballotMode`), packed into a single 247-bit integer (see Section 5).
- `0x3`: the encryption public key used to encrypt the ballots (`encryptionKey`). For a key with twisted Edwards coordinates (x, y), the leaf stores $\mathsf{SHA256}(x_{\mathrm{BE32}} \,||\, y_{\mathrm{BE32}})$.
- `0x4`: the running encrypted results (`encryptedResults`), stored as the SHA-256 digest of the 64 coordinates of its 16 ciphertexts, each encoded in 32 big-endian bytes.
- `0x6`: the census origin (`censusOrigin`).
- `0x7`: the digest of the ballot circuit's verification key (`ballotVKHash`), which fixes the only ballot proofs the batch program accepts for this election.

Let `configMax` denote the largest index reserved for configuration parameters, `configMax` $= 15$. When an election is created, the ProcessRegistry computes the *genesis root*: the root of the tree that contains exactly the six entries above, with `encryptedResults` set to the identity accumulator, in which every ciphertext is the pair of identity points $((0, 1), (0, 1))$ and therefore an encryption of zero. Every subsequent transition must start from the root the registry holds, so the configuration of an election cannot differ from the one recorded on-chain.

Encrypted ballot namespace. The region immediately following the configuration parameters, the interval $[\texttt{configMax} + 1, N - 1]$, is reserved for storing encrypted ballots. Ballot storage locations, called *slots*, are

[4] The indices `0x1` and `0x5` are currently unused: `0x1` is reserved for backward compatibility (historically used for alternative census structures), and `0x5` is reserved for future use (an earlier design stored a separate subtractive results accumulator there; the current design keeps a single net results accumulator at `0x4`). The indices `0x8` to `0xF` are reserved for future parameters.

deterministic and derived from the voter's identity. For a Merkle census, the slot is derived from the voter's address:

$$\begin{aligned} h &= \mathsf{SHA256}(\texttt{"davinci-slot-v1"} \,||\, \texttt{address}_{20}), \\ \texttt{slot}(\texttt{address}) &= (\texttt{configMax}+1) + \big(\mathsf{BE64}(h_{[0..8]}) \bmod (N - \texttt{configMax} - 1)\big), \end{aligned} \tag{3}$$

where the first eight bytes of the digest h are read as a big-endian integer. For a CSP census, the slot is $(\texttt{configMax}+1)+\texttt{idx}$, where $\texttt{idx}$ is the index signed by the CSP. In both cases, the result lies in the ballot namespace. The value stored at a slot is the SHA-256 digest of the voter's latest encrypted ballot, after re-encryption by the sequencer; the ciphertexts themselves are published in the data blobs (see Section 4.5.2).

This construction assigns each eligible voter a unique, reserved leaf in the state tree. Because the address is authenticated by the voter's signature and bound to the census proof (or the index is authenticated by the CSP's signature), a voter can only write to their own slot. Deriving the slot from the address rather than from the voter's position in the census also keeps slots stable when the census grows, and avoids relying on the position of a leaf in a lean IMT, which a membership proof does not pin (see Section 2.3). This enables efficient *last-vote-wins* semantics: if a voter overwrites their ballot, the new ciphertext replaces the previous one at the same slot, while the tally is updated by subtracting the old contribution and adding the new one. The uniqueness of slots among the members of a census is discussed in Section 4.3.

Besides the slots written by votes, every state transition also rewrites a random sample of already occupied slots with fresh re-encryptions of the ballots they hold (see Section 4.5.2). As a consequence, a change in the value of a slot does not reveal whether its voter cast a new ballot. Only the first write to a slot, which turns an empty leaf into an occupied one, is distinguishable, and it reveals that the voter participated, not how.

Vote identifier namespace. The upper portion of the tree, $[N, 2^D-1]$, is reserved for *vote identifiers* ($\texttt{voteID}$), which allow voters to verify that their vote has been included in the election state, without revealing or linking to the corresponding encrypted ballot. Vote identifiers are derived by hashing the process identifier, the voter's address, and fresh randomness, and then mapping the result into the allowed range by setting the most significant bit to 1:

$$\texttt{voteID} = N + \big(\mathsf{Poseidon}(\texttt{processID}, \texttt{address}, k) \bmod N\big), \tag{4}$$

where k is the secret randomness of the voter's ballot encryption. The value stored under a vote identifier is zero: the leaf only records that the identifier has been used. The ballot circuit enforces this derivation, and the batch program enforces that $N \le \texttt{voteID} < 2^D$ and that each vote identifier is inserted into an empty leaf, guaranteeing that vote identifiers cannot overlap with configuration entries or ballot storage and cannot be reused.

Unlike classical nullifiers, vote identifiers are not used to prevent duplicate voting directly: duplicate voting is prevented by the slot, which a voter can only overwrite. Instead, a vote identifier is a receipt of inclusion that does not reveal the slot. Since the vote identifier depends on a fresh random value k, if a collision occurs (i.e., the computed leaf is already occupied), the sequencer rejects the ballot, and the batch program would reject any transition that inserted it. In that case, the voter must resample k, re-encrypt the ballot, and recompute a new vote identifier until an unused leaf is obtained. As a result, collisions do not compromise correctness or liveness, but merely require retrying the identifier derivation.

Note that the maximum number of vote identifiers that can be stored is bounded by the size of this namespace (2^{D-1}), which upper-bounds the total number of votes and overwrites that can be processed throughout the election. This capacity is orders of magnitude larger than any realistic election workload: historical elections peak at $\approx 10^9$ votes [42], ensuring that identifier exhaustion is practically impossible and that collisions are exceptionally rare [5].

Together, the state tree provides a compact and tamper-proof commitment to the current status of the election. At the end of the election, the final state root together with the on-chain verification of all transitions serves as a complete cryptographic record of the election, encapsulating both the tally and the integrity of the entire voting procedure.

[5] Based on the birthday paradox, the probability of at least one collision occurring is given by $P = 1 - e^{-n^2/2d}$, where n is the number of vote identifiers and $d = 2^{D-1}$. For a large-scale national election of 10^8 votes emitted, this probability is vanishingly small ($\approx 0.054\%$), and for an extreme workload of 10^9 votes, the probability of a single collision is only $\approx 5.3\%$, meaning that the need to resample k remains highly improbable.

4.5 Circuits and zkVM programs

At the core of DAVINCI lie three proof statements that enforce the correctness of every step of the election. Each statement corresponds to a distinct task and, taken together, they guarantee that all protocol rules are satisfied without revealing any private information. Specifically, the *ballot circuit* (Section 4.5.1), proven by the voter, ensures that an encrypted vote is valid and well-formed; the *batch program* (Section 4.5.2), proven by a sequencer in a zkVM, verifies a batch of votes, applies it to the state tree, and lays out the published data, producing the single proof that is verified on-chain for each state transition; and the *results program* (Section 4.5.3) proves that the election results were correctly decrypted from the final state. The ballot circuit is an arithmetic circuit proven with Groth16, small enough to be proven on a voter's device. The two programs are Rust programs compiled to RISC-V and proven with the ZisK zkVM, whose proofs are verified on-chain by the ZiskVerifier contract (see Section 2.4). The flow of these proofs and their interactions is shown in Fig. 4.

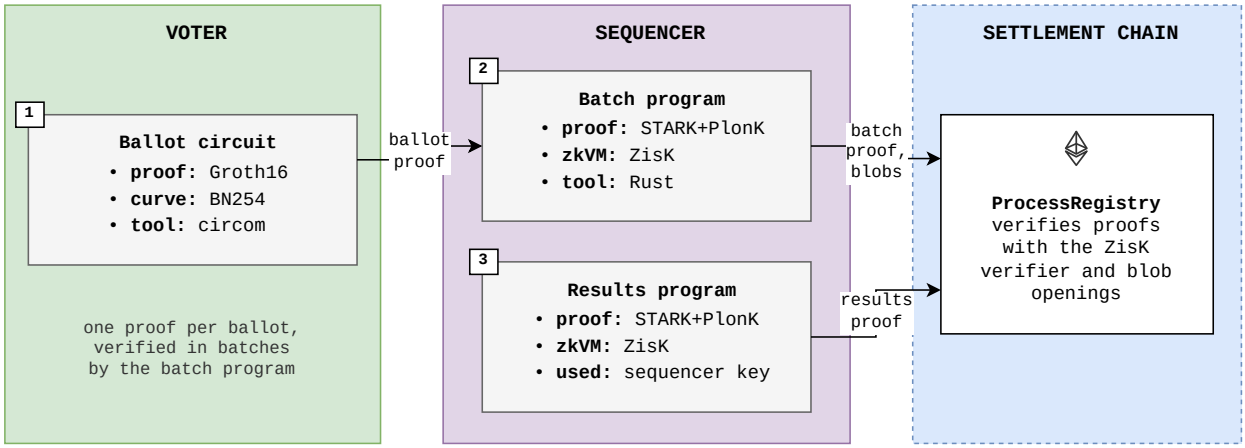


Fig. 4: Flow of the voter's circuit and the sequencer's zkVM programs.

The batch program replaces the pipeline of a verifier circuit, an aggregation circuit, and a state transition circuit used by an earlier design of DAVINCI. The statement is the same in spirit, but writing it as one program removes the need for recursion over several curves and makes it possible to prove properties that are impractical in circuits, such as the reconstruction of the published blob data from the verified state. A zkVM program differs from a circuit in one respect that the protocol must account for: the program always runs to completion and commits its public values, including a flag that reports whether every check passed. A proof of the batch program is therefore only meaningful together with the flag, and the registry rejects any proof whose flag is not set (see Section 4.5.2).

Remark 1. For clarity, the statements are described in this section with their full set of inputs and outputs explicitly listed. In the actual implementation, the ballot circuit applies an optimization: rather than exposing all of its public inputs PI individually, it exposes the voter's address, the vote identifier, and a single commitment $\texttt{inputsHash} = \mathsf{Poseidon}(\mathsf{PI})$ to all the inputs that the batch program must bind (see the multi-input Poseidon construction in Appendix A.2). The batch program recomputes the commitment from the values it uses and checks that it matches the one proven by the voter. This optimization does not alter the semantics of the circuit but significantly reduces the cost of verifying ballot proofs. Similarly, the public values of the zkVM programs reach the ZisK verifier as a single SHA-256 digest. Other implementation aspects such as the exact number of constraints and the proving frameworks used are deferred to Section 6.2.

4.5.1 Ballot circuit

The ballot circuit ($\mathsf{BallotCircuit}$), illustrated in Fig. 5, is proven locally by the voter at the time of casting a ballot. Its purpose is twofold: first, to prove that the encrypted ballot is valid and complies with the protocol

rules defined by the organizer; and second, to prove that the vote identifier (`voteID`) has been correctly derived. The correctness of this circuit is attested by a ZK proof generated using the Groth16 protocol described in Appendix A.8, instantiated over the BN254 curve. We call the resulting proof *ballot proof* and denote it as `ballotProof`. The circuit is compiled for the maximum number of ballot fields, 16; ballots with fewer fields leave the remaining ones unconstrained here, and the batch program requires their ciphertexts to be the identity. We detail below the inputs and constraints of the ballot circuit.

Public inputs

- public `address`: voter's address.
- public `voteID`: vote's unique identifier.
- public `inputsHash`: commitment to the inputs that the batch program binds, namely `processID`, `ballotMode`, `encryptionKey`, `address`, `voteID`, `encryptedBallot`, and `weight`.

Private inputs

- private `processID`: election identifier.
- private `ballotMode`: ballot protocol configuration, packed as in the state tree.
- private `encryptionKey`: encryption public key.
- private `weight`: voter's weight.
- private `encryptedBallot`: encrypted ballot, i.e., one ElGamal ciphertext per ballot field.
- private `ballot`: voter's voting choice, i.e., one value per ballot field.
- private k: random scalar.

Constraints

- *Inputs commitment.* The commitment is computed over the inputs in a fixed order:

$$\texttt{inputsHash} = \mathsf{Poseidon}(\texttt{processID}, \texttt{ballotMode}, \texttt{encryptionKey}, \texttt{address}, \texttt{voteID}, \texttt{encryptedBallot}, \texttt{weight}).$$

- *Vote encryption.* Correct encryption of the ballot. Every field $j \in \{1, \ldots, \texttt{numFields}\}$ is encrypted with its own randomness, derived from k by iterated hashing:

$$k_0 = k, \qquad k_j = \mathsf{Poseidon}(k_{j-1}), \qquad \texttt{encryptedBallot}_j = \mathsf{Enc}_{\texttt{encryptionKey}}(\texttt{ballot}_j; k_j).$$

- *Encryption key.* The key `encryptionKey` lies in the prime-order subgroup of BabyJubjub and is not the identity.
- *Protocol rules compliance.* The voter's ballot meets the protocol rules (see Section 5):

$$\text{protocolRulesCompliance}(\texttt{ballotMode}, \texttt{weight}, \texttt{ballot}) = \texttt{true}.$$

- *Vote identifier.* The vote's identifier is correctly computed ($N = 2^{63}$, see Section 4.4):

$$\texttt{voteID} = N + \big(\mathsf{Poseidon}(\texttt{processID}, \texttt{address}, k) \bmod N\big).$$

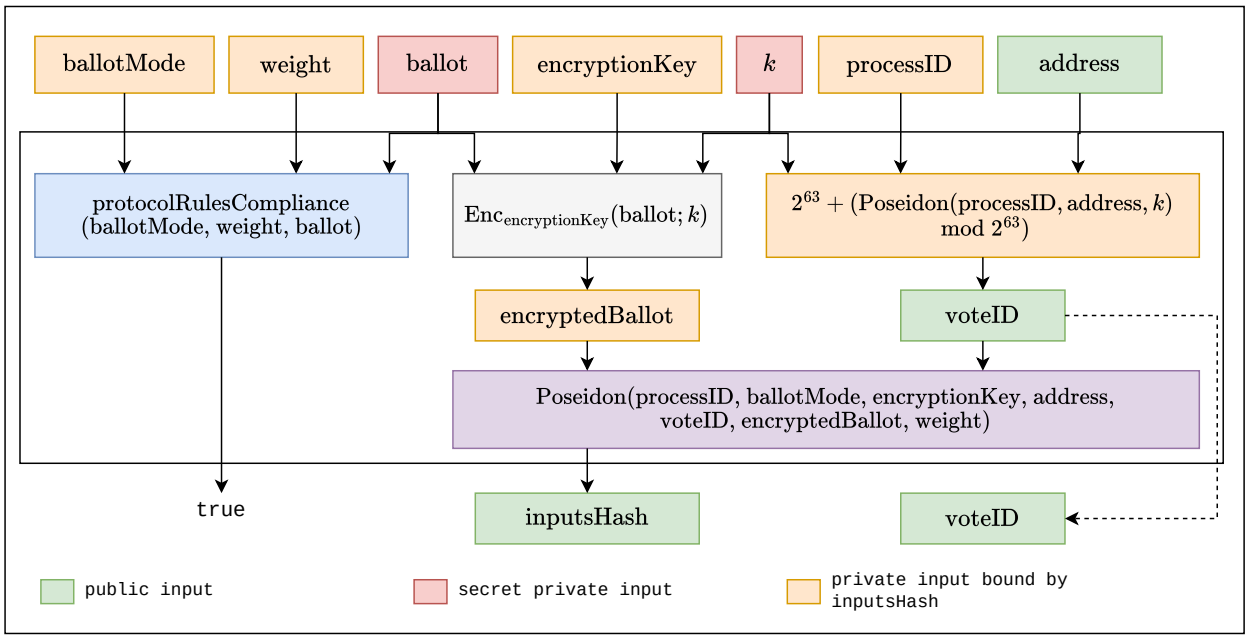


Fig. 5: Ballot circuit. Public values are framed in green, secret private values in red, and private values bound by `inputsHash` in orange.

The ballot proof does not authenticate the voter: anyone who knows a voter's address could prove a ballot for it. Authentication is provided by a separate signature over the vote identifier, $\texttt{signature} = \mathsf{S.Sign}_{\mathsf{sk}}(\texttt{voteID})$, produced with the voter's SECP256K1 key and verified by the batch program. Because the vote identifier is bound to the encrypted ballot through k and through `inputsHash`, the signature authorizes exactly one encrypted ballot.

4.5.2 Batch program

The batch program (BatchProgram), illustrated in Fig. 6, is executed by the sequencer to update the global state of the election with a batch of votes. Its purpose is to verify that every vote in the batch is valid, authenticated, and cast by an eligible voter; that the batch has been correctly incorporated into the state Merkle tree, with overwrites and vote identifiers handled consistently; that every stored ballot is a correct re-encryption of the ballot the voter proved; that a sample of other stored ballots has been silently re-encrypted; that the encrypted results accumulator is updated accordingly; and that the data published in Ethereum blobs is exactly the data needed to rebuild the new state. The correctness of an execution is attested by a ZisK proof wrapped in PlonK over BN254, which we call the *batch proof* and denote as `batchProof`. It is submitted on-chain and verified by the ProcessRegistry smart contract, ensuring that every accepted state transition complies with the protocol rules.

To formalize the setting, let the sequencer collect a batch of n votes, $1 \le n \le 1024$, of which w overwrite a ballot already stored in the state and $n - w$ are the first ballot of their voter. In addition, the sequencer selects a set $\mathcal{R}$ of $|\mathcal{R}| \le 2048$ occupied slots, not written by the batch, whose ballots it re-encrypts. Each ballot consists of 16 ElGamal ciphertexts over BabyJubjub, of which the first $n_f = \texttt{numFields}$ are active and the rest are the identity. We write B_i for the encrypted ballot proven by voter i, B'_i for its re-encryption, and $B_i^{\texttt{old}}$ for the stored ballot that it overwrites, if any; R_j and R'_j for the stored ballot of the j-th refreshed slot and its re-encryption; and A for the results accumulator `encryptedResults`. A ballot is a vector of 16 ciphertexts, and $B_{i,f}$ denotes its f-th ciphertext. In the following, we describe the public outputs, the private inputs, and the constraints of the batch program. The private inputs are organized as the program reads them, in five blocks: ballot proofs, state transition, eligibility, re-encryption, and blobs.

Public outputs

- public `ok`, `failMask`: whether every constraint below holds, and a bit mask of the groups of constraints that failed.
- public $\texttt{stateRoot}^{\texttt{old}}$: current Merkle root of the state tree.
- public $\texttt{stateRoot}^{\texttt{new}}$: new tree root after all state transitions are applied.
- public `numVotes`: number of votes in the batch, n.
- public `numOverwrites`: number of votes in the batch that overwrite a stored ballot, w.
- public `censusRoot`: root of the census tree used by the membership proofs or, for a CSP census, the address of the CSP.
- public `occupiedBefore`: number of occupied ballot slots before the batch.
- public `blobsDigest`, `nBlobs`: digest of the blob commitments and evaluations, and number of blobs.

Private inputs

- private vk: verification key of the ballot circuit.
- private $\{\texttt{ballotProof}_i, \texttt{address}_i, \texttt{voteID}_i, \texttt{inputsHash}_i, \texttt{signature}_i\}_{i=1}^{n}$: ballot proofs with their public inputs, and the voters' signatures.

- private `processID`: election identifier.
- private $\{\texttt{inclusionProof}^{\kappa}\}_{\kappa\in\{\texttt{0x0},\texttt{0x2},\texttt{0x3},\texttt{0x6},\texttt{0x7}\}}$: inclusion proofs of the configuration leaves `processID`, `ballotMode`, `encryptionKey`, `censusOrigin`, and `ballotVKHash` under $\texttt{stateRoot}^{\texttt{old}}$.
- private $\{\tau_i^{\texttt{voteID}}\}_{i=1}^{n}$, $\{\tau_i^{\texttt{encryptedBallot}}\}_{i=1}^{n}$, $\{\tau_j^{\mathcal{R}}\}_{j=1}^{|\mathcal{R}|}$, $\tau^{\texttt{encryptedResults}}$: chains of tree transitions that insert the vote identifiers, write the ballots, refresh the sample, and update the results accumulator (see below).
- private $\{\texttt{reEncryptedBallot}_i\}_{i=1}^{n}$: the re-encrypted ballots written to the voters' slots.
- private $\{\texttt{encryptedBallot}_i^{\texttt{old}}\}$: the w stored ballots that the batch overwrites.
- private $\{\texttt{encryptedBallot}_j^{\mathcal{R}}\}_{j=1}^{|\mathcal{R}|}$: the stored ballots of the refreshed slots.
- private $\texttt{encryptedResults}^{\texttt{old}}$: current accumulator of votes.

- private $\{\texttt{censusProof}_i\}_{i=1}^{n}$: census membership proofs (Merkle paths, or CSP signatures with their indexes and weights).

- private `encryptionKey`: encryption public key.
- private `seed`: secret seed of the batch, from which all re-encryption randomness is derived.
- private $\{\texttt{encryptedBallot}_i\}_{i=1}^{n}$: the voters' encrypted ballots, as proven.

- private $\{\mathsf{blobCommitment}_b\}_{b=1}^{\texttt{nBlobs}}$: KZG commitments to the blobs.

Constraints

- *Ballot proof verification.* Every ballot proof is valid under the ballot verification key of the election. The program first checks that the key is the one committed in the state, $\mathsf{SHA256}(\mathsf{vk}) = \texttt{ballotVKHash}$, where the key is serialized as its points $\alpha, \beta, \gamma, \delta$ and the coefficients of L, each coordinate as four 64-bit little-endian limbs, and the digest is read as a big-endian integer. It also checks that all proof points are on their curves and not the identity, and that the $\mathbb{G}_2^{\mathsf{BN}}$ points of the key are in the prime-order subgroup. It then verifies the n proofs at once with a randomized batch check [3]: it derives coefficients $\rho_1 = 1$ and $\rho_i = \mathsf{lo}_{128}\big(\mathsf{SHA256}(\mathsf{SHA256}(T) \,||\, \mathsf{LE64}(i-1))\big)$ for $i \geq 2$ from the transcript $T = \texttt{"groth16-batch-v2"} \,||\, A_1 \,||\, B_1 \,||\, C_1 \,||\, x_1 \,||\, \cdots \,||\, A_n \,||\, B_n \,||\, C_n \,||\, x_n$ of all proofs (A_i, B_i, C_i) and public inputs x_i, encoded as raw limbs (a zero coefficient is replaced by 1), and checks the single pairing equation obtained as the ρ-weighted product of the n Groth16 verification equations, computing every scalar multiplication itself. A batch containing an invalid proof passes this check with probability at most 2^{-128}. For every $i \in \{1, \ldots, n\}$, this establishes

$$\mathsf{P.Verify}\big(\texttt{ballotProof}_i, (\texttt{address}_i, \texttt{voteID}_i, \texttt{inputsHash}_i)\big) = \texttt{true}.$$

- *Authentication.* Every ballot is signed by the owner of its address. For every i, the vote identifier is a 64-bit integer, a public key is recovered from the signature over the 32-byte message $\texttt{0x00}^{24} \,||\, \texttt{voteID}_{i,\mathrm{BE8}}$ in the Ethereum personal-message format, and its address is the voter's:

$$\mathsf{pk}_i = \mathsf{S.ExtractPublicKey}(\texttt{voteID}_i, \texttt{signature}_i), \qquad \texttt{address}_i = \mathsf{low\text{-}20}\big(\mathsf{Keccak}(\mathsf{pk}_i)\big).$$

- *Census membership.* Each processed vote must correspond to an eligible voter, according to the census configuration specified by the parameter `censusOrigin`.
  - If the census is a Merkle tree ($\texttt{censusOrigin} \in \{1,2,3\}$), then for every i the program verifies a Merkle membership proof of the leaf $\texttt{address}_i \cdot 2^{88} + \texttt{weight}_i$ (Eq. (1)),

$$\mathsf{MT}^{\texttt{census}}.\mathsf{Verify}\big(\texttt{address}_i \cdot 2^{88} + \texttt{weight}_i, \texttt{idx}_i, \texttt{censusProof}_i, \texttt{censusRoot}\big) = \texttt{true},$$

    where $\texttt{idx}_i$ is the position carried by the proof; all proofs share the same root, which is published as `censusRoot`, no leaf appears twice, and every path has a canonical shape (at most 61 siblings, and no index bit beyond the path length).
  - If $\texttt{censusOrigin} = 4$ (CSP), then for every i the program recovers a public key from the credential over the message of Eq. (2) and requires all credentials of the batch to recover the same key, whose address is published as `censusRoot`:

$$\mathsf{pk}^{\mathsf{CSP}} = \mathsf{S.ExtractPublicKey}\big((\texttt{processID}, \texttt{address}_i, \texttt{weight}_i, \texttt{idx}_i), \texttt{censusProof}_i\big),$$
$$\texttt{censusRoot} = \mathsf{low\text{-}20}\big(\mathsf{Keccak}(\mathsf{pk}^{\mathsf{CSP}})\big).$$

    Addresses must be canonical 160-bit integers, and no address and no index may appear twice in the batch.
- *Binding.* The values used by the different checks are the same. For every i, the address of the census leaf (or credential) equals $\texttt{address}_i$; the commitment recomputed from the values used below equals the one proven by the voter,

$$\texttt{inputsHash}_i = \mathsf{Poseidon}(\texttt{processID}, \texttt{ballotMode}, \texttt{encryptionKey}, \texttt{address}_i, \texttt{voteID}_i, B_i, \texttt{weight}_i);$$

  and the ballot written to the voter's slot is the re-encryption computed below. The election key is the one committed in the state: $\mathsf{SHA256}(\texttt{encryptionKey}_{x,\mathrm{BE32}} \,||\, \texttt{encryptionKey}_{y,\mathrm{BE32}})$ equals the value of leaf `0x3`. Finally, the blob block refers to the same `processID` and $\texttt{stateRoot}^{\texttt{old}}$ as the state block.
- *Configuration.* The configuration of the election is read from the current state. The program verifies the inclusion proofs of the five configuration leaves under $\texttt{stateRoot}^{\texttt{old}}$, in the order `0x0`, `0x2`, `0x3`, `0x6`, `0x7`, and requires the value of leaf `0x0` to be the `processID` of the state block:

$$\mathsf{MT}^{\texttt{state}}.\mathsf{Verify}(v_\kappa, \kappa, \texttt{inclusionProof}^\kappa, \texttt{stateRoot}^{\texttt{old}}) = \texttt{true}, \qquad \kappa \in \{\texttt{0x0}, \texttt{0x2}, \texttt{0x3}, \texttt{0x6}, \texttt{0x7}\}.$$

  The number of active fields n_f is read from the packed `ballotMode` and must lie in $[1, 16]$.
- *Key namespaces.* Every vote identifier lies in the vote identifier namespace, $N \le \texttt{voteID}_i < 2^{64}$, and every ballot is written to its voter's slot, which lies in the ballot namespace: $\texttt{slot}_i = \texttt{slot}(\texttt{address}_i)$ (Eq. (3)) for a Merkle census, or $\texttt{slot}_i = 16 + \texttt{idx}_i$ for a CSP census. The slots of a batch are pairwise distinct.
- *Transition verification.* The state evolves from $\texttt{stateRoot}^{\texttt{old}}$ to $\texttt{stateRoot}^{\texttt{new}}$ through a contiguous chain of tree transitions, in this order (see Fig. 7), where each transition is a valid insertion or update proven against the root left by the previous one, and the last one ends at $\texttt{stateRoot}^{\texttt{new}}$:
  1. For every vote $i = 1, \dots, n$, insert the vote identifier with value zero: $\mathsf{MT}^{\texttt{state}}.\mathsf{Insert}(\texttt{voteID}_i, 0)$. An insertion of a key already present fails, so a vote identifier can never be reused.
  2. For every vote $i = 1, \dots, n$, write the ballot digest to the voter's slot, with an insertion if the slot is empty and an update if it is occupied:

$$\mathsf{MT}^{\texttt{state}}.\mathsf{Insert}\big(\texttt{slot}_i, \mathsf{SHA256}(B'_i)\big) \quad \text{or} \quad \mathsf{MT}^{\texttt{state}}.\mathsf{Update}\big(\texttt{slot}_i, \mathsf{SHA256}(B'_i)\big).$$

     The number of updates is exactly w, and the value replaced by each update is $\mathsf{SHA256}(B_i^{\texttt{old}})$.
  3. For every refreshed slot $j = 1, \dots, |\mathcal{R}|$, in strictly increasing order of slots,

$$\mathsf{MT}^{\texttt{state}}.\mathsf{Update}\big(\texttt{slot}_j^{\mathcal{R}}, \mathsf{SHA256}(R'_j)\big),$$

     where the replaced value is $\mathsf{SHA256}(R_j)$. The refreshed slots are occupied, distinct, and disjoint from the slots written in step 2.

4. Update the accumulator: $\mathsf{MT}^{\mathtt{state}}.\mathsf{Update}(\mathtt{0x4}, \mathsf{SHA256}(A^{\mathtt{new}}))$, where the replaced value is $\mathsf{SHA256}(A^{\mathtt{old}})$. This update is present if and only if the batch writes or refreshes at least one ballot.

- *Re-encryption.* Every ballot written to the tree is a re-encryption of its source, with randomness derived from the secret seed. The program derives a chain of scalars from the seed and the current root,

$$\eta_0 = \mathsf{SHA256}(\mathtt{"davinci\text{-}reenc\text{-}v1"} \,||\, \mathtt{seed} \,||\, \mathtt{stateRoot}^{\mathtt{old}}) \bmod s, \qquad \eta_{c+1} = \mathsf{SHA256}(\eta_c) \bmod s,$$

  where s is the order of the BN254 scalar field (the base field of BabyJubjub), and consumes one scalar per active ciphertext, first through the n voters' ballots and then through the $|\mathcal{R}|$ refreshed ballots, field by field (see Fig. 8). where the seed, the root, and every η_c enter the hash as 32-byte big-endian integers. A ciphertext (C_1, C_2) that consumes η_c becomes $(C_1 + \eta_c \cdot G^{\mathrm{BJ}},\ C_2 + \eta_c \cdot \mathtt{encryptionKey})$, that is,

$$B'_{i,f} = \mathsf{ReEnc}_{\mathtt{encryptionKey}}(B_{i,f}; \eta_c), \qquad R'_{j,f} = \mathsf{ReEnc}_{\mathtt{encryptionKey}}(R_{j,f}; \eta_{c'}),$$

  for the scalars η_c and $\eta_{c'}$ that these ciphertexts consume. Inactive fields must be the identity, before and after, in every ballot. The key `encryptionKey` must be on the curve, not the identity, and in the prime-order subgroup.
- *Results accumulation.* The new accumulator adds the new ballots, subtracts the overwritten ones, and adds the difference introduced by each refresh, which is an encryption of zero:

$$A^{\mathtt{new}} = A^{\mathtt{old}} + \sum_{i=1}^{n} B'_i - \sum_{\text{overwrites}} B_i^{\mathtt{old}} + \sum_{j=1}^{|\mathcal{R}|} \big(R'_j - R_j\big),$$

  where sums of ballots are field-wise homomorphic additions of ciphertexts (EncAdd) and subtraction adds the inverse point.
- *Silent refresh.* The number of refreshed slots is large enough to hide the overwrites: with $o = \mathtt{occupiedBefore}$, it must hold that $o \geq w$ and

$$|\mathcal{R}| \ \geq\ \min\big(\mathsf{target}(n, w),\ o - w\big), \qquad \mathsf{target}(n, w) = \min\big(2048,\ \max(16,\ 2w,\ n)\big).$$

- *Data blob.* The blob contents are rebuilt from the verified data and bound to the commitments.
  - From the vote identifiers and from every slot written or refreshed by the batch, with its new ballot, construct the blob cells as described in Section 4.5.2, and split them into $\mathtt{nBlobs} = \lceil T/4096 \rceil$ blobs, where T is the number of cells. The number of commitments must be `nBlobs`, with $1 \leq \mathtt{nBlobs} \leq 32$.
  - For every blob b, derive the polynomial $p_b(x)$ whose evaluations over the blob domain are the cells of the blob.
  - Compute the evaluation point from the commitment and the election context,

$$z_b = \mathsf{SHA256}\big(\mathtt{processID}_{\mathrm{BE32}} \,||\, \mathtt{stateRoot}^{\mathtt{old}}_{\mathrm{BE32}} \,||\, \mathsf{blobCommitment}_b\big) \bmod q_{\mathsf{BLS}},$$

    where q_{BLS} is the order of the BLS12-381 scalar field.
  - Compute the evaluation $y_b = p_b(z_b)$.
  - Output $\mathtt{blobsDigest} = \mathsf{SHA256}(\mathsf{blobCommitment}_1 \,||\, y_1 \,||\, \cdots \,||\, \mathsf{blobCommitment}_{\mathtt{nBlobs}} \,||\, y_{\mathtt{nBlobs}})$.

  The program does not verify the KZG openings, which would require pairings over BLS12-381; the registry verifies them with the point-evaluation precompile, against the versioned hashes of the blobs actually published.

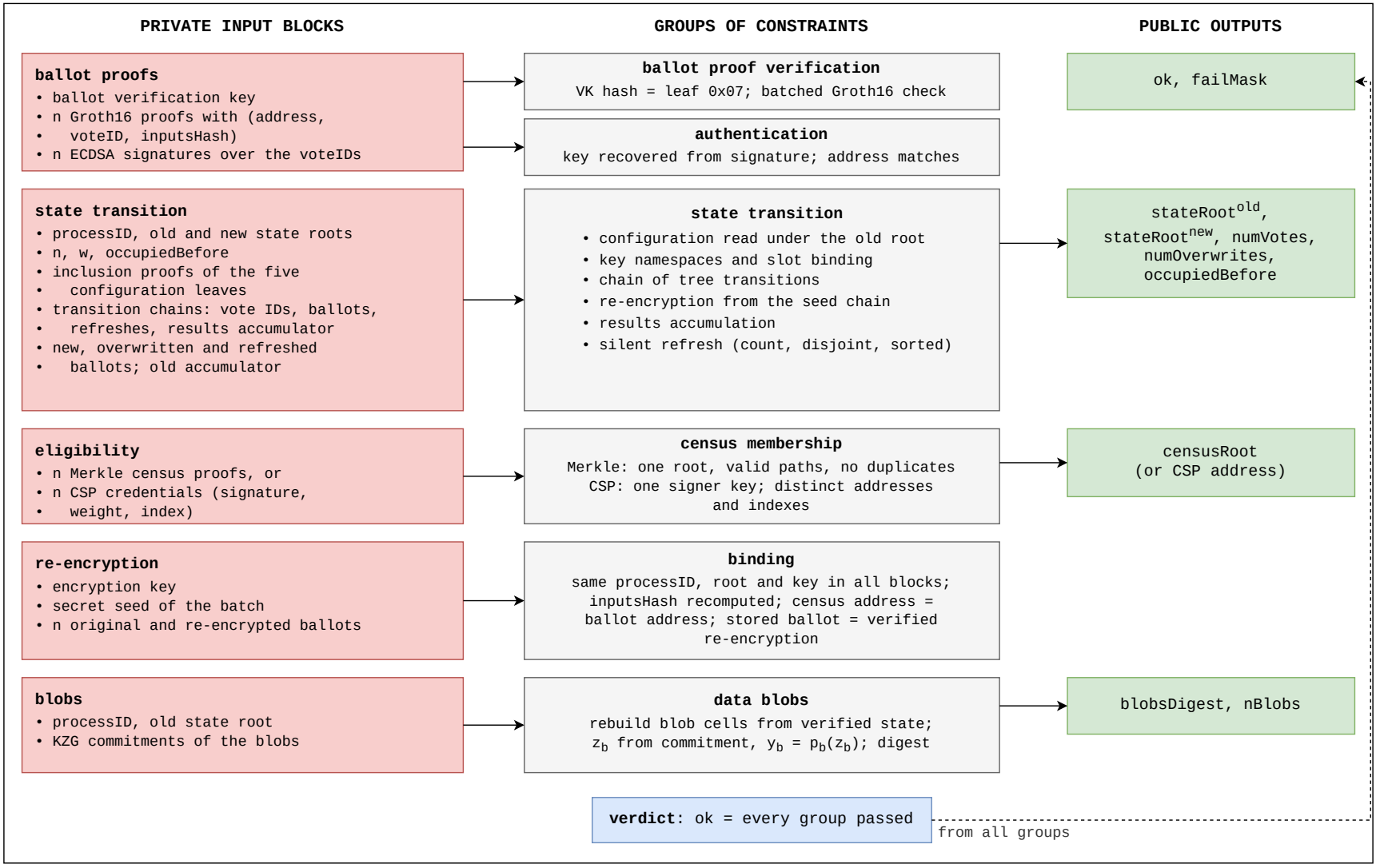


Fig. 6: Batch program. The private input blocks (left) feed the groups of constraints (center), which produce the public outputs (right). Private values are framed in red, public values in green.

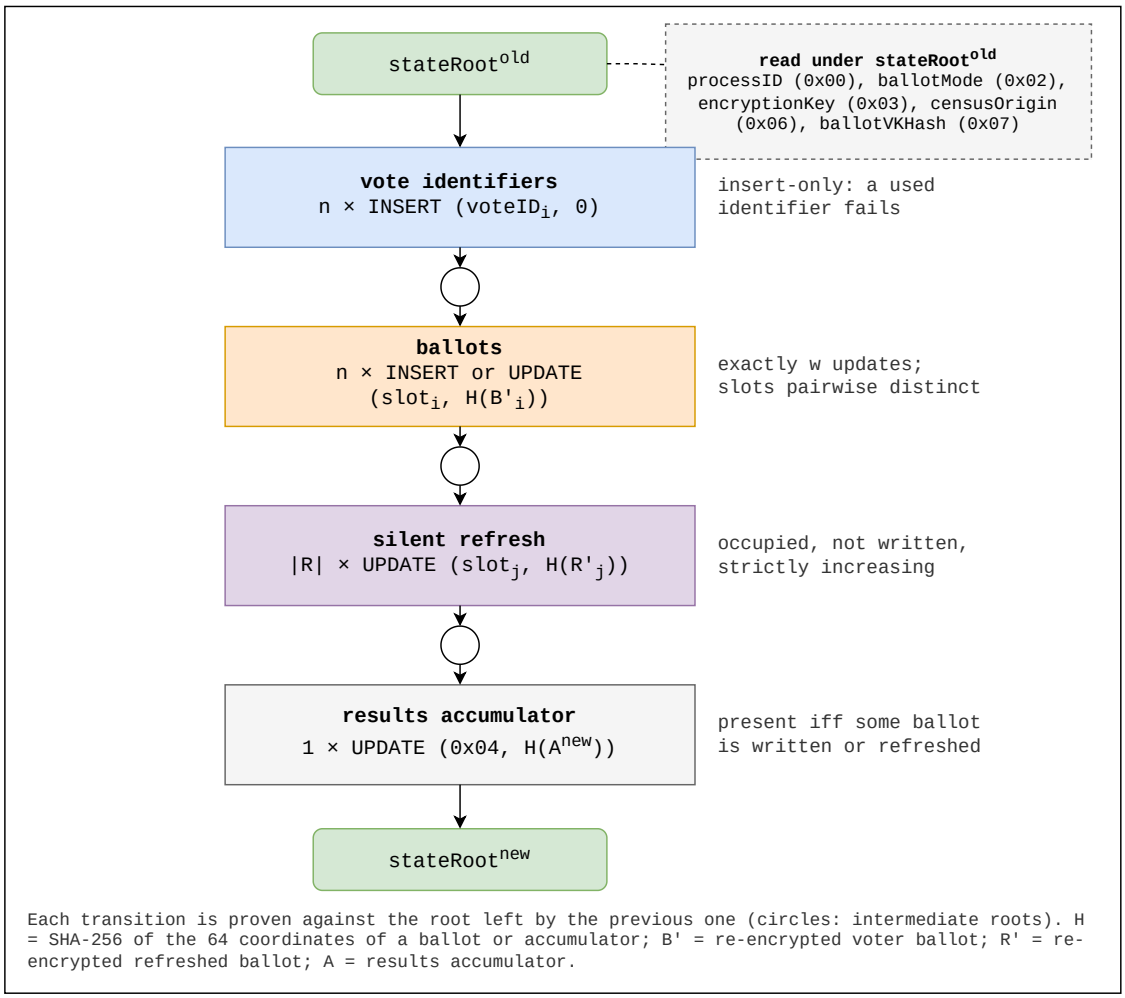


Fig. 7: Chain of state tree transitions proven by the batch program. Configuration leaves are read under the old root; then vote identifiers, ballots, refreshes, and the results accumulator are written in a fixed order, each transition starting from the root left by the previous one.

Verdict and public values. Each group of constraints sets a bit of `failMask` when it fails, and all groups are evaluated even after a failure, so the mask helps diagnose a rejected batch. The program sets `ok` $= 1$ if and only if every constraint holds. The public values form 46 32-bit words, laid out as in Table 3; a ZisK proof carries 64 such words, of which words 46 to 63 are zero, and on-chain the public values encode each word as 8 little-endian bytes, 512 bytes in total. The proof of an execution with `ok` $= 0$ is a valid proof of a rejected batch, so every consumer of a batch proof must check, besides the proof itself, that `ok` $= 1$ and `failMask` $= 0$.

Checks left to the verifier. The batch program proves a transition from whatever root and census it is given. The binding of the proof to the right election, to the current state, and to the published data is performed by the registry when the proof is submitted (see Section 4.6.3), which checks that: the proof is valid under the pinned program key and setup; `ok` $= 1$ and `failMask` $= 0$; `stateRoot`$^{\texttt{old}}$ is the latest root of the process; `censusRoot` is a root the election accepts (Table 2); `occupiedBefore` equals the number of distinct slots written so far; the batch carries at least one vote; and every $(\mathsf{blobCommitment}_b, y_b)$ pair opens correctly against the versioned hash of the corresponding blob.

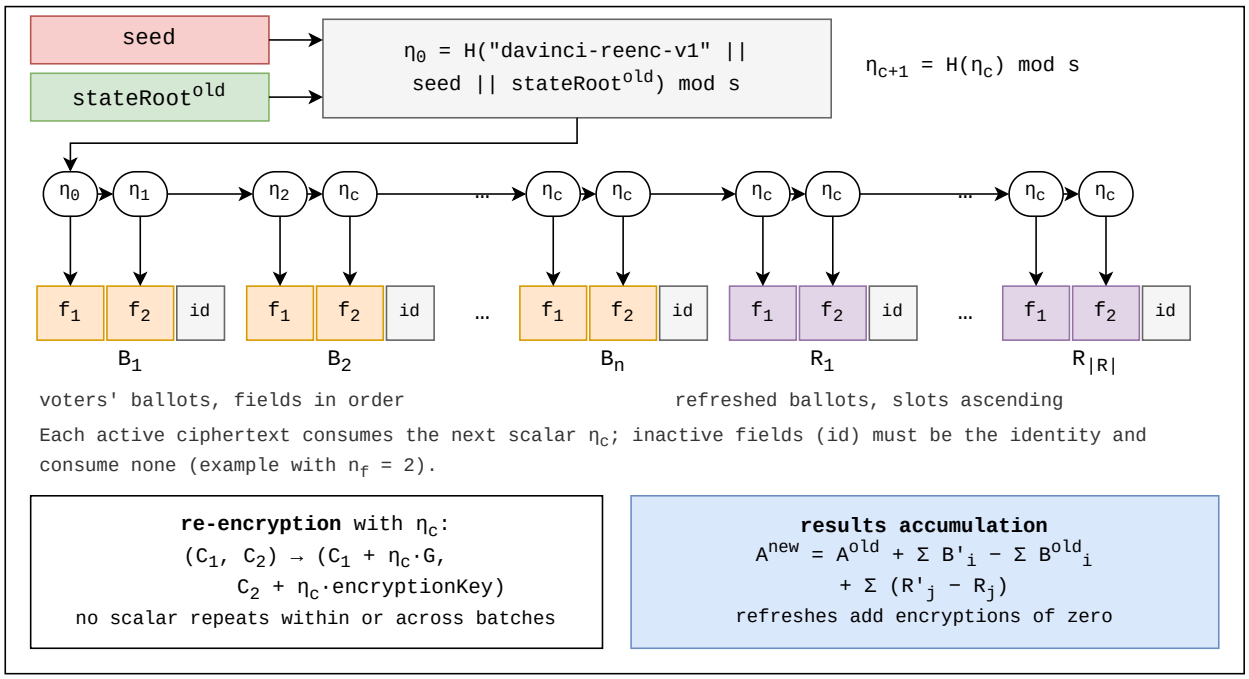


Fig. 8: Re-encryption in the batch program. A single secret seed and the old state root start a chain of scalars, each consumed by exactly one active ciphertext, first of the voters' ballots and then of the refreshed ballots. The accumulator absorbs the new ballots, the inverse of the overwritten ones, and the encryptions of zero introduced by the refreshes.

Blob layout. The blobs of a state transition carry exactly the data an observer needs to rebuild the new state from the old one, and nothing else. Every cell is a 32-byte big-endian element of the BLS12-381 scalar field. The program lays out the cells as follows (see Fig. 9):

1. A header cell, $n_{\mathsf{up}} + 2^{64} \cdot n_{\mathsf{id}}$, where $n_{\mathsf{id}} = n$ is the number of vote identifiers and $n_{\mathsf{up}} = n + |\mathcal{R}|$ the number of updated slots.
2. The list of the n_{id} vote identifiers, in increasing order and without their most significant bit, followed by the list of the n_{up} updated slots, in increasing order, packed four 63-bit integers per cell: $\mathsf{pack4}(a, b, c, d) = a \cdot 2^{189} + b \cdot 2^{126} + c \cdot 2^{63} + d$.
3. For each updated slot, in the same order, the $2n_f$ points of its new ballot, one cell per point: a point (x, y) is encoded as $y + 2^{254} \cdot (x \bmod 2)$, which identifies the point given the curve equation.
4. Zero cells up to the end of the last blob.

The updated slots include the new votes, the overwrites, and the refreshes, merged into one sorted list. An observer therefore cannot tell an overwrite from a refresh, which is the purpose of silent revoting; only the first write of a slot is distinguishable, as discussed in Section 4.4. The results accumulator is not published: an observer that holds the previous state recomputes it as $\mathtt{encryptedResults}^{\mathtt{old}} + \sum_u (B_u^{\mathtt{new}} - B_u^{\mathtt{old}})$ over the updated slots, where $B_u^{\mathtt{old}}$ is the identity for a previously empty slot, which equals the accumulator of the constraint above. The total number of cells is

$$T = 1 + \lceil (n_{\mathsf{id}} + n_{\mathsf{up}})/4 \rceil + 2\, n_f\, n_{\mathsf{up}},$$

so, for instance, a batch of 512 new votes with 512 refreshes at $n_f = 16$ needs $1 + 384 + 32768 = 33153$ cells, that is, 9 blobs, and a batch of 512 overwrites, which requires 1024 refreshes, needs 13 blobs.

4.5.3 Results program

The results program (ResultsProgram), illustrated in Fig. 10, is executed at the end of the election by the holder of the election key, when the key is held by a single sequencer, to prove that the published tally was

Table 3: Public values of the batch program, as 32-bit words. Roots and digests take eight words, in little-endian order; the registry reads `censusRoot` as the big-endian integer obtained by reversing these bytes.

Words	Name	Content
0, 1	`ok`, `failMask`	Verdict and failed groups of constraints.
2–9, 10–17	$\texttt{stateRoot}^{\texttt{old}}$, $\texttt{stateRoot}^{\texttt{new}}$	State roots before and after the batch.
18, 19	`numVotes`, `numOverwrites`	Votes and overwrites in the batch.
20–27	`censusRoot`	Census root, or CSP address.
28–35, 36	`blobsDigest`, `nBlobs`	Digest of the $(\mathsf{blobCommitment}_b, y_b)$ pairs, and blob count.
42	`occupiedBefore`	Occupied ballot slots before the batch.
37–41, 43–45		Reserved (zero) and diagnostic values.

correctly obtained from the encrypted results committed in the final state. Crucially, the program does not take the decryption key as input, nor does it decrypt inside the proof: instead, for each ballot field it verifies a proof of correct decryption against the encryption public key that is committed in the state tree. The correctness of an execution is attested by a ZisK proof wrapped in PlonK over BN254, verified on-chain by the ProcessRegistry through the ZiskVerifier contract. We call the resulting proof the *results proof* and denote it as `resultsProof`. Elections whose key is held by key wardens do not use this program: the registry binds the accumulator to the final state root itself, and the key wardens' contracts verify a proof of every partial decryption (see Section 4.6.6). We detail below the inputs and constraints of the results program.

Public outputs

- public `ok`, `failMask`: whether every constraint below holds, and the groups that failed.
- public `stateRoot`: final root of the state tree.
- public $\{\text{results}_j\}_{j=1}^{16}$: decrypted tally, one 64-bit entry per ballot field.

The public values are 43 32-bit words: `ok` and `failMask` in words 0 and 1, `stateRoot` in words 2 to 9, the low and high halves of results_j in words $10 + 2(j-1)$ and $11 + 2(j-1)$, and, in word 42, the index of the first decryption proof that failed. The registry reads the first `numFields` results.

Private inputs

- private `encryptedResults`: encrypted results accumulator stored in the state (16 ElGamal ciphertexts).
- private `encryptionKey`: encryption public key.
- private $\texttt{inclusionProof}^{\texttt{encryptedResults}}$: proof that `encryptedResults` is stored in leaf `0x4` of the state tree.
- private $\texttt{inclusionProof}^{\texttt{encryptionKey}}$: proof that `encryptionKey` is stored in leaf `0x3` of the state tree.
- private $\{\pi_j^{\text{dec}}\}_{j=1}^{16}$: per-field Chaum–Pedersen proofs of correct decryption.

Constraints

- *Canonical encodings.* Every coordinate is below the field order and every proof scalar below the subgroup order, so that each point and scalar has a single encoding.
- *Encryption key.* The key `encryptionKey` is on the curve, not the identity, and in the prime-order subgroup.
- *State commitment.* The encrypted results and the encryption public key match the values committed in the final state:

$$\mathsf{MT}^{\texttt{state}}.\mathsf{Verify}(\mathsf{SHA256}(\texttt{encryptedResults}), \texttt{0x4}, \texttt{inclusionProof}^{\texttt{encryptedResults}}, \texttt{stateRoot}),$$

$$\mathsf{MT}^{\texttt{state}}.\mathsf{Verify}(\mathsf{SHA256}(\texttt{encryptionKey}), \texttt{0x3}, \texttt{inclusionProof}^{\texttt{encryptionKey}}, \texttt{stateRoot}).$$

- *Correct decryption.* For every ballot field $j \in \{1, \ldots, 16\}$, the public result results_j is the decryption of the encrypted accumulator $\texttt{encryptedResults}_j$ under `encryptionKey`, as attested by π_j^{dec} (see Appendix A.6):

$$\mathsf{CP}.\mathsf{Verify}_{\texttt{encryptionKey}}\big(\texttt{encryptedResults}_j,\ \text{results}_j,\ \pi_j^{\text{dec}}\big) = \texttt{true}.$$

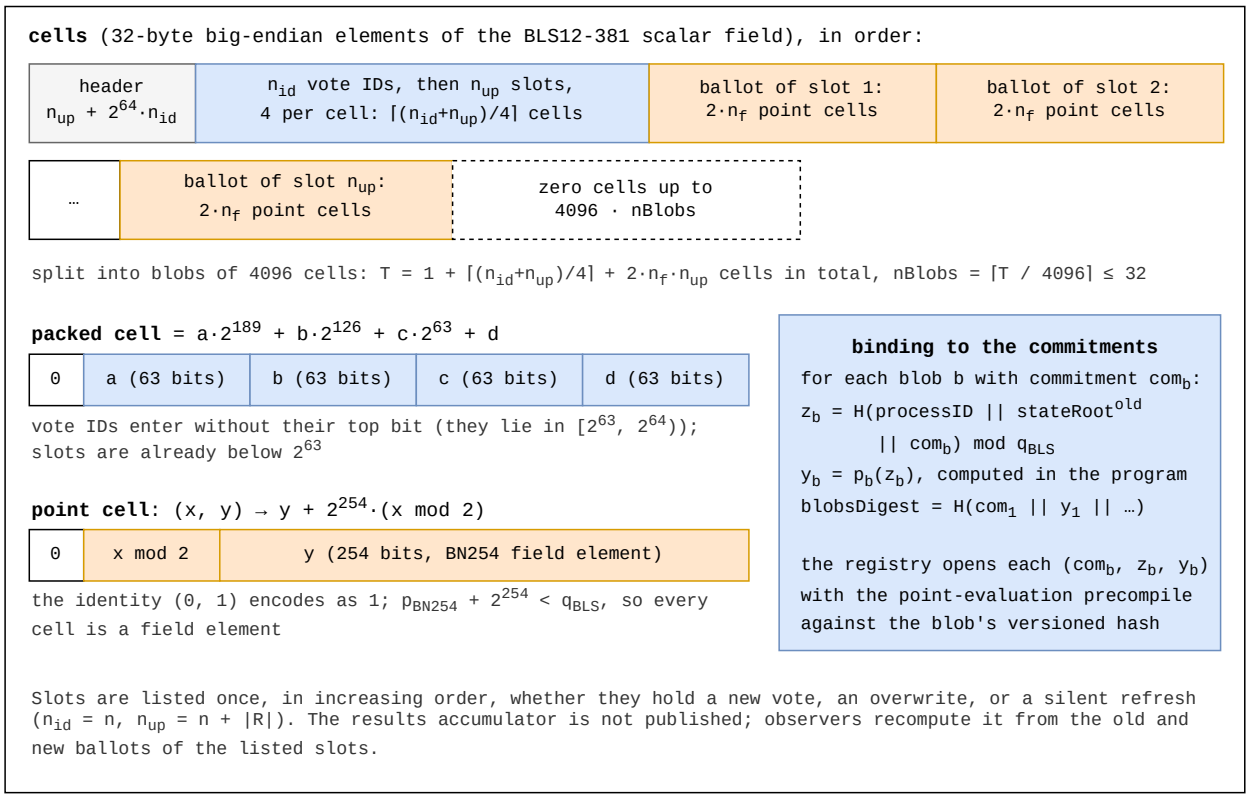


Fig. 9: Layout of the cells of the blobs of a state transition.

- *Results range.* Each result is a 64-bit integer, $0 \leq \text{results}_j < 2^{64}$, which is below the order t of the BabyJubjub prime-order subgroup, so the plaintext of each ciphertext is unique.

The per-field decryption proofs π_j^{dec} are produced by the key holder, who decrypts the aggregated results, recovers each value by a bounded discrete-logarithm search, and attests, through a discrete-log-equality proof, that the decryption was performed correctly under the election key. By checking these proofs against the public key committed at leaf `0x3`, the program guarantees that the released tally corresponds exactly to the encrypted results accumulated in the final state, without ever exposing the decryption key inside the proof. Inactive fields hold the identity ciphertext, for which only the value zero can be proven. If any constraint fails, the program outputs zero in place of the root and the results, so a consumer that omitted the check of `ok` still could not match a root or read a tally.

Remark 2. The batch proofs are verified on-chain one by one, which publishes the data of every batch as it is settled and lets several sequencers compete for each transition. The zkVM also supports a *chained* mode, in which batch proofs are produced as STARKs without the PlonK wrapper and an aggregator program (AggregatorProgram) verifies them recursively, enforces the continuity of state roots across batches, and folds a whole election into a single proof that is verified on-chain once, together with the results. Chained mode trades the per-batch settlement and data publication for a lower on-chain cost; it is not used by the deployment described in this paper, and the sequencer does not support it for dynamic censuses or threshold-held keys.

4.6 Protocol flow

In this section we describe the overall flow of the DAVINCI voting protocol. An election is structured into five main phases: election setup, encryption key generation, voting period, tally decryption, and election finalization. Each phase specifies the interactions between the organizer, key wardens, voters, sequencers, and the smart contracts that coordinate the election. An overview of the complete process is shown in Figs. 11 and 17.

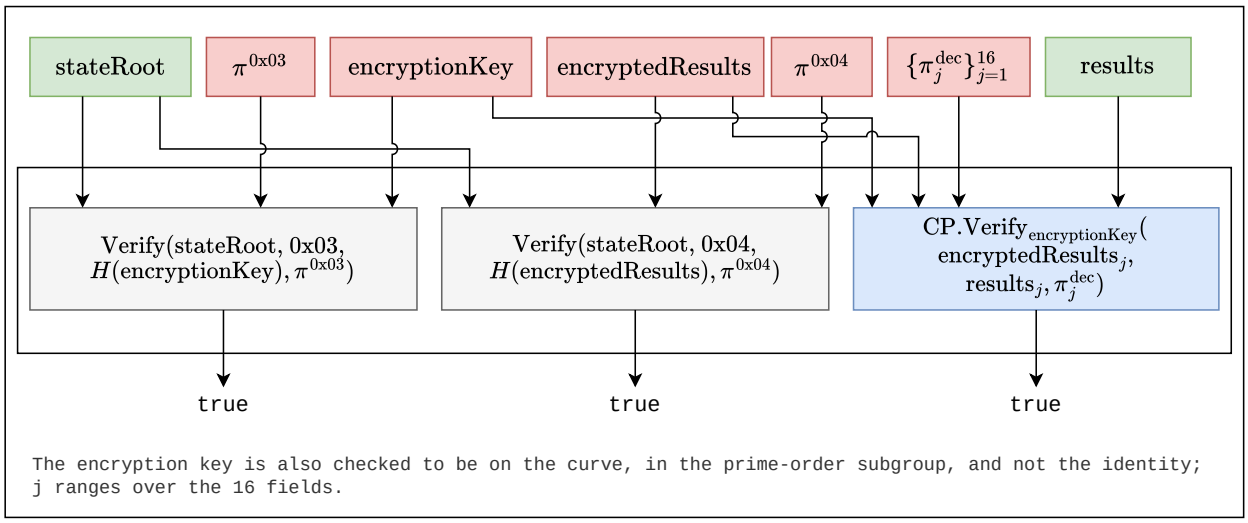


Fig. 10: Results program. Public values are framed in green and private values in red.

4.6.1 Election setup

For each new election, the organizer performs the following steps:

1. Prepare the census data `censusData` according to the chosen census model `censusOrigin` (see Section 4.3). For an off-chain census, build the lean IMT $\mathsf{MT}^{\texttt{census}}$ from it, publish the list of members, and record its location as `censusURI`; for an on-chain census, deploy or reuse a census contract; for a CSP census, record the endpoint of the CSP as `censusURI`.
2. Compute the census commitment: `censusRoot` $= \mathsf{MT}^{\texttt{census}}.\mathsf{Root}()$ for an off-chain census, or the address of the CSP's signing key for a CSP census.
3. Define the election details: the start time (`startTime`), the duration of the voting period (`duration`), the maximum number of voters (`maxVoters`), and the ballot rules `ballotMode` (see Section 5).
4. Publish the election metadata (questions, options, description, etc.) and record its location as `metadataURI` and the SHA-256 digest of its exact bytes as `metadataHash`.
5. Determine the process identifier `processID`, which the ProcessRegistry derives from the organizer's address, a prefix that encodes the chain and the registry, and the organizer's process nonce (see Section 4.2). Being deterministic, it can be read in advance from the registry, although it is ultimately recomputed and assigned by the contract upon registration.
6. Choose the key mode `keyMode` and obtain what it requires (see Section 4.6.2): the encryption public key `encryptionKey` for `processID` from a sequencer, in the sequencer mode; nothing, in the automatic DKG mode; an organizer key pair and a proof of possession of its secret, in the locked DKG mode; or the identifier of a council ceremony, in the council mode.
7. Register the process on-chain by submitting a transaction to the ProcessRegistry smart contract with the election parameters:

$$\begin{aligned}\mathsf{tx} = \big(&\texttt{status},\ \texttt{startTime},\ \texttt{duration},\ \texttt{maxVoters},\ \texttt{ballotMode},\\ &(\texttt{censusOrigin},\ \texttt{censusRoot},\ \texttt{censusURI},\ \texttt{contractAddress}),\ \texttt{metadataURI},\ \texttt{metadataHash},\\ &\texttt{encryptionKey},\ (\texttt{keyMode},\ \texttt{dkgParams})\big),\end{aligned}$$

where `status` is `READY` or `PAUSED` and `encryptionKey` is left empty in the DKG and council modes. The registry validates the parameters: the ballot mode must satisfy the bounds of Section 5; the worst-case tally must be decryptable, $\texttt{maxValue} \cdot \texttt{maxVoters} \leq 10^{12}$; the census must be consistent with its origin; the voting period must not start in the past and must end in the future; and the metadata must be non-empty. In the DKG and council modes, the registry obtains `encryptionKey` from the corresponding key-management contract (see Section 4.6.2). In every mode, `encryptionKey` must be a

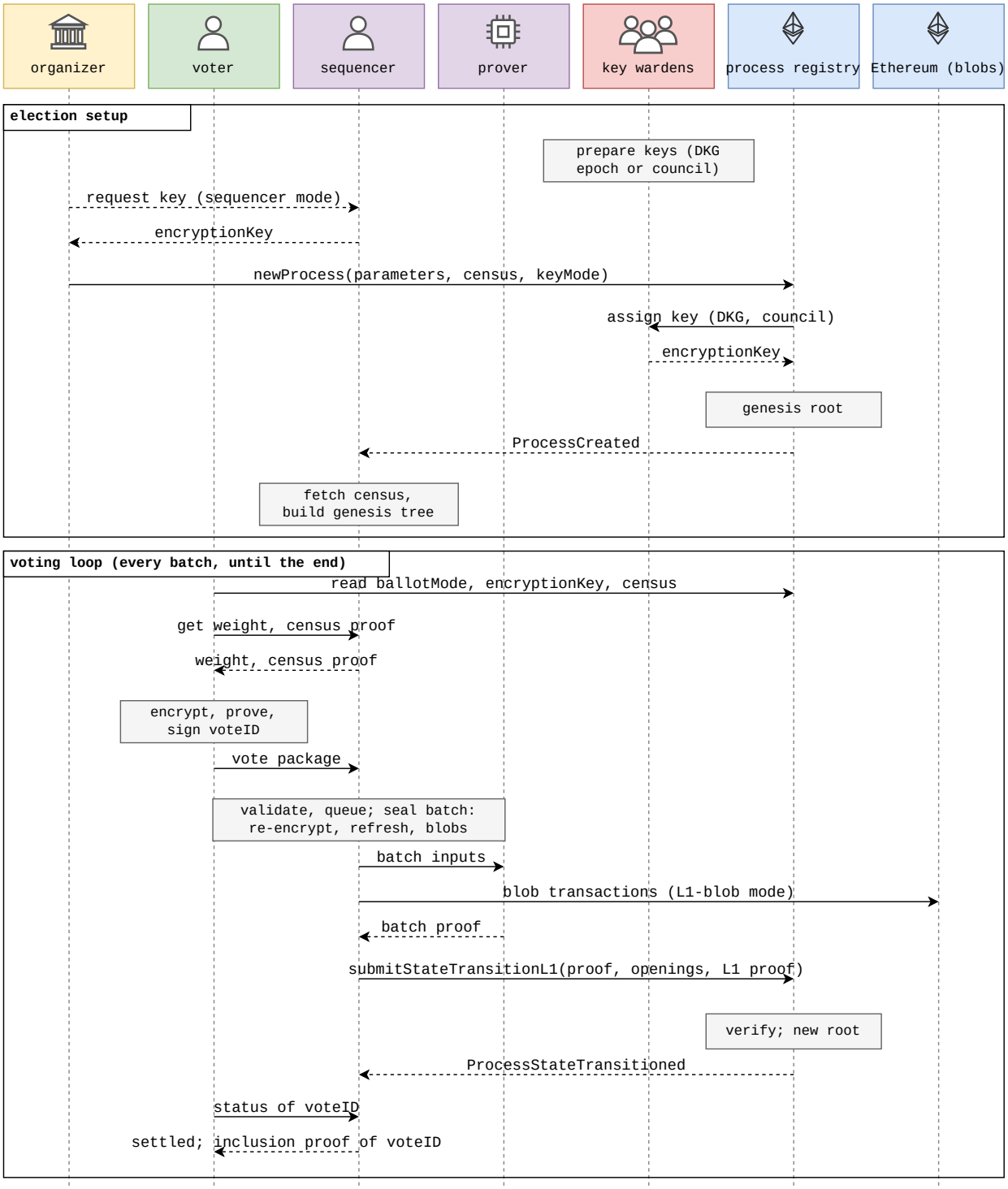


Fig. 11: Message flow of an election: setup and voting. The voting loop runs for every batch; settlement is shown in L1-blob mode, in which the blobs are posted to Ethereum while the batch is proven. Fig. 17 continues the flow after the end of the voting period.

canonical BabyJubjub point other than the identity. Upon accepting this transaction, the ProcessRegistry contract computes the genesis state root, whose tree contains only the configuration leaves (see Section 4.4): `processID` at `0x0`, `ballotMode` at `0x2`, `encryptionKey` at `0x3`, the results accumulator `encryptedResults` at `0x4` initialized to the encryption of zero, `censusOrigin` at `0x6`, and the pinned `ballotVKHash` at `0x7`:

$$\texttt{stateRoot} = \mathsf{MT}^{\texttt{state}}.\mathsf{Root}().$$

Once this transaction is accepted, the election is publicly registered on-chain, the registry emits a `ProcessCreated` event, and, at its scheduled start time, the voting period can begin. Sequencers learn of the new process from the event: each sequencer that wishes to serve it validates the configuration, downloads or indexes the census, rebuilds the census tree and checks its root, and creates the genesis state tree locally, checking that its root equals the one computed by the registry.

4.6.2 Encryption key generation

In this phase, the public encryption key (`encryptionKey`) that voters will use to encrypt their ballots is established. DAVINCI supports four key modes, chosen by the organizer at creation and summarized in Fig. 12. In all of them, the key is fixed when the process is created, it is committed in the genesis state, and the batch and results proofs are the same; the modes differ in who can decrypt and, consequently, in who publishes the results.

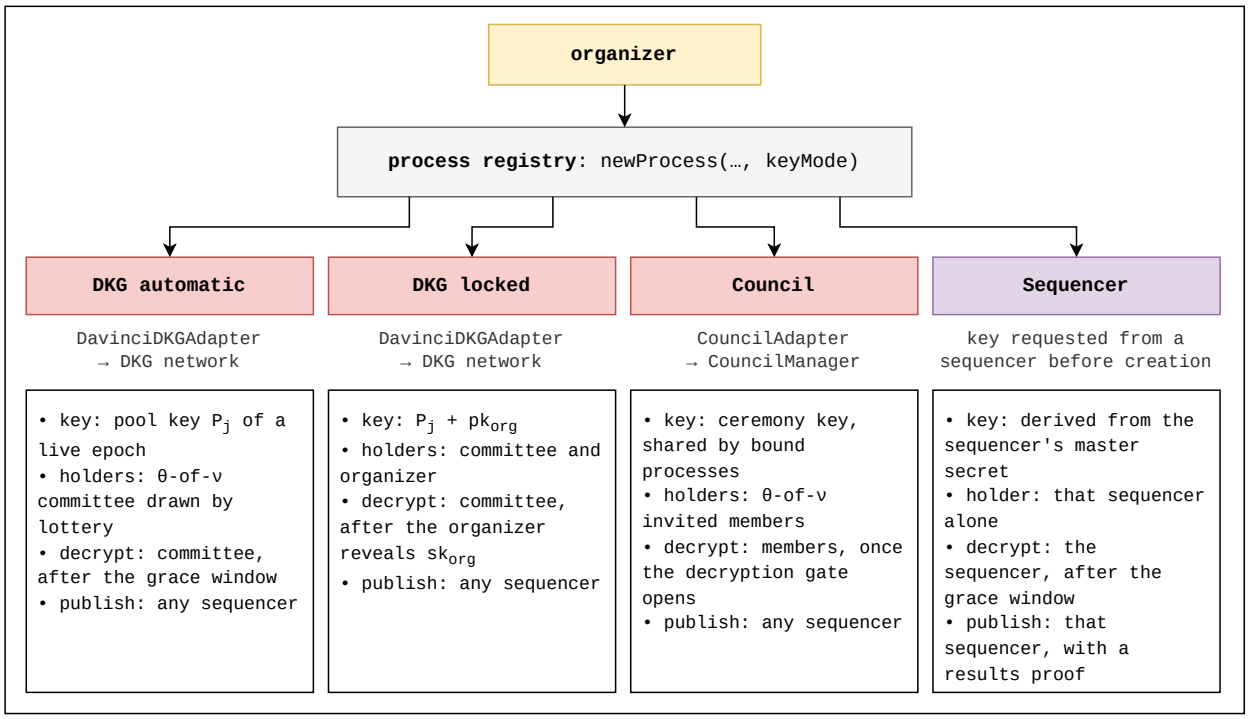


Fig. 12: Key modes. The organizer always interacts with the process registry only; the key-management contracts sit behind it. The modes differ in who can decrypt the tally and how the results are published.

DKG automatic mode. The key wardens are the operators of a permissionless DKG network [50], which runs the non-interactive DKG protocol of Appendix A.7 in *epochs*. At the start of each epoch, a committee of ν operators is drawn by a public lottery among the registered operators, and each committee member submits one contribution that deals, with Feldman verifiable secret sharing [21], shares of sixteen independent pool keys at once, together with a Groth16 proof that the contribution is well-formed and that every share

is encrypted to its recipient. The DKGManager verifies these proofs, so that correct participation can be checked without revealing any secret. Once enough valid contributions are collected, a single proof computes the sixteen pool keys $P_0, \ldots, P_{15}$ and a commitment to every member's share of each, and the epoch becomes live. Any θ of the ν committee members, where θ is the threshold of the epoch, can then decrypt under any pool key, while no smaller coalition learns anything about the secrets. When a process is created in this mode, the DavinciDKGAdapter registers it as an application of the newest live epoch, under an application identifier that embeds the adapter's address and a salt derived from the chain, the registry, and `processID`, so that no other party can claim it. The application is assigned a free pool key P_j, which becomes the election key, $\texttt{encryptionKey} = P_j$.

DKG locked mode. The organizer generates a key pair $(\mathsf{sk}_{\mathsf{org}}, \mathsf{pk}_{\mathsf{org}})$ and provides $\mathsf{pk}_{\mathsf{org}}$, together with a Schnorr proof of possession of $\mathsf{sk}_{\mathsf{org}}$, at creation. The election key is $\texttt{encryptionKey} = P_j + \mathsf{pk}_{\mathsf{org}}$, and the DKG contracts refuse every decryption of the application until the organizer reveals $\mathsf{sk}_{\mathsf{org}}$ on-chain, which they check against $\mathsf{pk}_{\mathsf{org}}$. The organizer thus decides when the tally can be opened, but not what it contains, and not even θ colluding committee members can open it alone. Losing $\mathsf{sk}_{\mathsf{org}}$ makes the tally permanently undecryptable.

Council mode. The key wardens are the members of a council: up to sixteen participants invited by an organizer, who run a dedicated key ceremony on the CouncilManager [49]. Each member deals a Feldman sharing of a random secret with a Groth16 proof of correct dealing, the election key is the sum of the dealers' public constants, and any θ of its ν members can decrypt, for a threshold θ fixed when the ceremony is created. A council ceremony also fixes when decryption may start, either on a scheduled date or when the organizer of the ceremony opens it, optionally with a fallback date. Several processes may be bound to the same ceremony, provided that the ceremony organizer has allowed the registry's CouncilAdapter and has authorized the account creating each process; all of them share the ceremony's key.

Sequencer mode. When the organizer accepts a weaker trust model, a single sequencer acts as the only key warden. Before creating the process, the organizer requests from a sequencer the key for the (predictable) identifier `processID`. The sequencer derives the secret key from a master secret,

$$\mathsf{sk} = \mathsf{HMAC\text{-}SHA256}\big(\mathsf{master},\ \texttt{"davinci-election-key-v1"} \,||\, \texttt{chainID} \,||\, \texttt{registry} \,||\, \texttt{processID}\big),$$

reduced into $[1, t)$, and returns $\texttt{encryptionKey} = \mathsf{sk} \cdot G^{\text{BJ}}$. Since the key is derived, the sequencer stores nothing per request, and a key handed out for one process is useless under any other. That sequencer can decrypt every ballot published in the blobs, and it is the only party able to publish the results. This mode suits testing and elections in which the organizer trusts the operator of the sequencer.

4.6.3 Voting period

In this phase, voters cast their ballots and send them to the sequencers, who collect and process them.

(a) Vote casting. Each voter holds an Ethereum key pair $(\mathsf{sk}, \mathsf{pk})$ with associated address `address` (see Appendix A.5). To cast a ballot, a voter does the following:

1. Fetch the parameters `encryptionKey`, `ballotMode`, `processID`, and the census configuration from the ProcessRegistry contract, and the metadata from `metadataURI`, checking its digest.
2. Retrieve the weight `weight` and, for a CSP census, the credential $\texttt{censusProof} = (\texttt{signature}^{\mathsf{CSP}}, \texttt{idx})$. For a Merkle census, the weight and the membership proof can be obtained from any sequencer serving the election, and checked against `censusRoot`; the sequencer does not need the proof from the voter, since it derives it from its own copy of the census.
3. Select their ballot choice `ballot` following the rules of `ballotMode`.
4. Sample fresh randomness $k \in \mathbb{F}_s$ and encrypt the ballot using `encryptionKey`, field by field, as specified by the ballot circuit:

$$\texttt{encryptedBallot} = \mathsf{Enc}_{\texttt{encryptionKey}}(\texttt{ballot}; k). \tag{5}$$

5. Use their address $\texttt{address}$ and the randomness k to compute a unique vote identifier, mapped into the reserved vote-identifier namespace (see Section 4.4):

$$\texttt{voteID} = N + \big(\mathsf{Poseidon}(\texttt{processID}, \texttt{address}, k) \bmod N\big). \tag{6}$$

6. Use the $\mathsf{BallotCircuit}$ from Section 4.5.1 to generate a ZK-SNARK proof that Eqs. (5) and (6) are correctly computed and that $\texttt{ballot}$ satisfies the ballot rules of $\texttt{ballotMode}$:

$$\begin{aligned}\texttt{ballotProof} = \mathsf{P.Prove}\big(&\mathsf{BallotCircuit},\\ &\mathsf{witness} = (\texttt{ballot}, k, \texttt{processID}, \texttt{ballotMode}, \texttt{encryptionKey},\\ &\qquad \texttt{weight}, \texttt{encryptedBallot}),\\ &\mathsf{PI} = (\texttt{address}, \texttt{voteID}, \texttt{inputsHash})\big).\end{aligned}$$

7. Sign the vote identifier with their Ethereum secret key sk to authenticate the vote:

$$\texttt{signature} = \mathsf{S.Sign}_{\mathsf{sk}}(\texttt{voteID}).$$

8. Select a sequencer and submit the package

$$\begin{aligned}\texttt{vote} = [&\texttt{processID}, \texttt{address}, \texttt{voteID}, \texttt{encryptedBallot}, \texttt{ballotProof},\\ &\texttt{inputsHash}, \texttt{signature}, \texttt{weight}, \texttt{censusProof}]\end{aligned}$$

   to the chosen sequencer, where $\texttt{censusProof}$ is only present for a CSP census.
9. Follow the status of the vote through its identifier. Once the vote is settled, obtain from any sequencer an inclusion proof of the leaf $\texttt{voteID}$ under the state root, and check it against the root recorded in the $\mathsf{ProcessRegistry}$.

A voter may repeat these steps any number of times while the voting period is open; each new ballot, once settled, replaces the previous one in the voter's slot. All ballots of one voter should be sent to the same sequencer, which settles them in the order they were cast; across sequencers, the ballot settled last prevails.

(b) Vote admission. Upon receiving a vote $\texttt{vote}_i$, a sequencer runs every per-vote check of the batch program before accepting it, so that a single invalid vote cannot invalidate a batch:

1. Check the ballot proof against the pinned ballot verification key:

$$\mathsf{P.Verify}\big(\texttt{ballotProof}_i, (\texttt{address}_i, \texttt{voteID}_i, \texttt{inputsHash}_i)\big) = \texttt{true}.$$

2. Recover the signer, $\mathsf{pk}_i = \mathsf{S.ExtractPublicKey}(\texttt{voteID}_i, \texttt{signature}_i)$, and check that its address is $\texttt{address}_i$.
3. Recompute $\texttt{inputsHash}_i$ from the process configuration and the submitted values, and check that the ciphertexts are canonical points with the identity in every inactive field.
4. Check eligibility: for a Merkle census, that $\texttt{address}_i$ is a member with weight $\texttt{weight}_i$ in the sequencer's copy of the census; for a CSP census, that the credential recovers the CSP address $\texttt{censusRoot}$.
5. Check the state: the vote identifier is neither in the state tree nor in the queue, the election is accepting votes (it is ready or paused, and its voting period is open), and the vote would not make the number of distinct voters exceed $\texttt{maxVoters}$.

Accepted votes are queued, one queue per slot, and their status becomes *pending*; a vote then moves through the states of Fig. 13.

(c) Vote batching. A sequencer seals a batch from the queue when it holds enough votes, when its oldest vote has waited long enough, or when the end of the voting period approaches. Waiting for several distinct slots before sealing mixes the votes of different voters in one transition; each wait carries a random jitter, so the settlement time of a batch reveals little about when its votes were cast. The batch takes the oldest vote of each slot, in arrival order, and is sized so that its proof and settlement fit before the deadline of the election, that it respects $\texttt{maxVoters}$, and that its blobs fit the limit of the settlement mode. To build the batch, the sequencer:

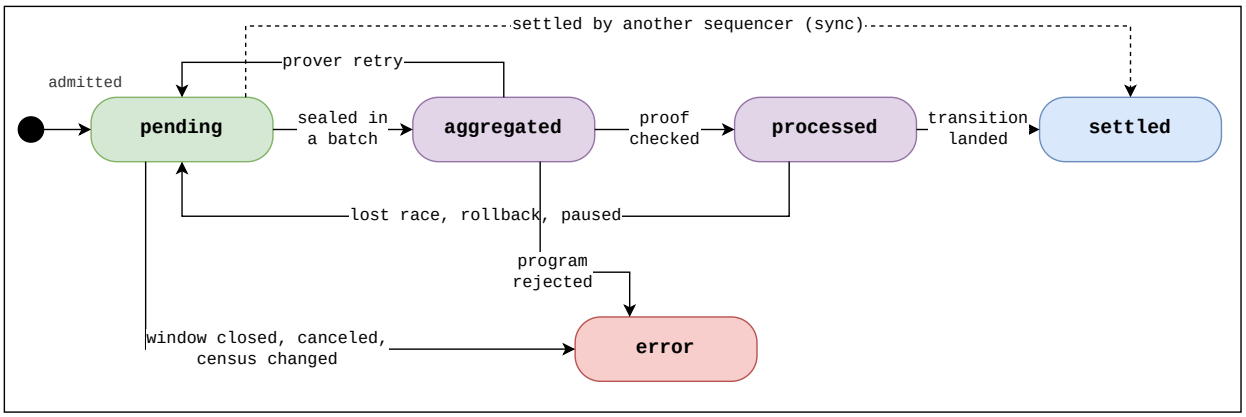


Fig. 13: Lifecycle of a vote in a sequencer. A batch that loses a settlement race, or that the registry refuses while the election is paused, returns its votes to the queue; votes still queued when the grace window closes, or that the program rejects, end in error.

1. Draws a fresh secret `seed` and a uniformly random refresh set $\mathcal{R}$ of occupied slots not written by the batch, of the size required by the batch program, from the operating system's random number generator. Both exist only in memory, for this batch, and are never logged or stored.
2. Re-encrypts every ballot along the scalar chain of the seed and the refreshed ballots after them (see Section 4.5.2).
3. Applies the batch to a copy of its state tree, producing the chains of tree transitions, the new results accumulator, and the new root.
4. Lays out the blob cells, computes the KZG commitment $\mathsf{blobCommitment}_b$ of each blob, the evaluation point z_b, the value $y_b = p_b(z_b)$, and the opening proof $\mathsf{openingProof}_b$ (see Section 4.5.2).
5. Computes the public values it expects the batch program to output.

A batch may reach the chain even if its sequencer later drops it, for instance after losing a race or when its blobs were posted but its settlement failed. If one of its votes is later sealed again in a different batch, with a different refresh draw, an observer comparing the two transitions could single out the slots written in both. To prevent this, every sequencer keeps a persistent coverage history of the batches that carried each vote, its own and those of other sequencers that it fetched, and a batch that carries such a vote again also refreshes every slot of those earlier batches that it does not write. The history contains only slot keys and vote identifiers that the blobs already publish, so keeping it reveals nothing new.

(d) Proving and settlement. The sequencer sends the private inputs of the batch to its prover and obtains the batch proof,

$$(\texttt{publicValues}, \texttt{batchProof}) = \mathsf{VM.Prove}\big(\mathsf{BatchProgram}, \\ \mathsf{witness} = (\text{ballot proofs, state transition, eligibility, re-encryption, blobs})\big).$$

Before using it, the sequencer checks that the proof is valid under the pinned program key and setup, that $\texttt{ok} = 1$, and that every public value equals the value it computed itself. It then settles the batch in one of the two data availability modes of the registry.

- *Blob mode* (registry on a chain with native blob transactions, such as Ethereum). The sequencer sends a single blob-carrying transaction that carries the $\texttt{nBlobs} \le 6$ blobs of the batch and calls

  $$\texttt{submitStateTransition}\big(\texttt{processID}, \texttt{publicValues}, \texttt{batchProof}, \\ \{\mathsf{blobCommitment}_b\}, \{y_b\}, \{\mathsf{openingProof}_b\}\big),$$

  where $\mathsf{openingProof}_b$ is the KZG opening proof of blob b at z_b. The registry reads the versioned hashes of the blobs of its own transaction.

- *L1-blob mode* (registry on an OP Stack rollup). The sequencer posts the blobs to Ethereum while the batch is being proven, in blob-carrying transactions of at most six blobs each, sent one after another. Once the proof is ready, every blob transaction is included, and the L1 origin of the rollup has reached the newest of them, the sequencer builds an L1 proof Π^{L1}: the Ethereum headers from the anchor down to the oldest blob transaction, and a MPT inclusion proof of each blob transaction in its block. If the origin moves past the anchor before the settlement lands, the sequencer can call `record` on the L1BlobVerifier to keep that block usable as an anchor. It then calls

$$\texttt{submitStateTransitionL1}\big(\texttt{processID}, \texttt{publicValues}, \texttt{batchProof}, \{\mathsf{blobCommitment}_b\}, \{y_b\}, \{\mathsf{openingProof}_b\}, \Pi^{\mathsf{L1}}\big),$$

  and the registry obtains the versioned hashes, in order, from $\mathsf{L1BlobVerifier}.\texttt{verify}(\Pi^{\mathsf{L1}})$. Fig. 14 shows the timeline of this mode.

Settlement is permissionless: the proof, the blob openings, and the continuity of state roots authenticate the transition, whoever sends it. Upon receiving the transaction, the registry performs the following checks, and reverts if any fails:

1. The entry point matches the registry's data availability mode.
2. The process is ready or ended, or paused past its end time, and the current time is between `startTime` and the end of the grace window (Section 4.6.4).
3. The public values are 512 bytes long and report $\texttt{ok} = 1$ and $\texttt{failMask} = 0$.
4. $\texttt{stateRoot}^{\texttt{old}}$ equals the latest state root of the process.
5. `censusRoot` is accepted for the census origin of the process (Table 2); for an on-chain census, the registry asks the census contract for the last block at which the root was current, and requires it to be between the creation block of the process and the current block.
6. `occupiedBefore` equals the process's `votersCount`, the number of distinct slots written so far.
7. The batch adds at least one vote, and $\texttt{votersCount} + (\texttt{numVotes} - \texttt{numOverwrites}) \leq \texttt{maxVoters}$.
8. There are exactly `nBlobs` commitments, evaluations, and opening proofs, $\texttt{nBlobs} \geq 1$, each commitment and opening proof is 48 bytes long, and there are as many non-zero versioned hashes: in blob mode, the transaction carries no blob beyond the first `nBlobs`; in L1-blob mode, the L1 proof yields exactly `nBlobs` versioned hashes.
9. The digest of the $(\mathsf{blobCommitment}_b, y_b)$ pairs equals `blobsDigest`.
10. The ZisK verifier accepts the proof:

$$\mathsf{ZiskVerifier}.\texttt{verifySnarkProof}(\texttt{batchProgramVK}, \texttt{rootCVadcopFinal}, \texttt{publicValues}, \texttt{batchProof}).$$

11. For every blob b, the point-evaluation precompile accepts the opening of $\mathsf{blobCommitment}_b$ at $z_b = \mathsf{SHA256}(\texttt{processID} || \texttt{stateRoot}^{\texttt{old}} || \mathsf{blobCommitment}_b) \bmod q_{\mathsf{BLS}}$ with value y_b, against the b-th versioned hash.

If all checks pass, the registry sets the latest state root to $\texttt{stateRoot}^{\texttt{new}}$, increases `votersCount` by $\texttt{numVotes} - \texttt{numOverwrites}$ and `overwrittenVotesCount` by `numOverwrites`, increments `batchNumber`, records the current time as `lastVoteAt`, and emits `ProcessStateTransitioned`; in L1-blob mode, it also emits `ProcessStateTransitionedL1` with the Ethereum block and the versioned hash of every blob, so that anyone can locate the data on Ethereum.

(e) Concurrent sequencers and synchronization. Several sequencers may serve the same election, and each builds its batches on the latest state root it knows. When two of them prepare a transition from the same root, the first one to land advances the election, and the settlement of the other reverts because its $\texttt{stateRoot}^{\texttt{old}}$ is no longer the latest root. The losing sequencer returns its votes to the queue, synchronizes, and builds its next batch on the new root; a lost race costs only the gas of the reverted transaction and, in L1-blob mode, the posted blobs. To synchronize, a sequencer fetches the blobs of every transition settled by others, checks them against the versioned hashes recorded by the settlement, decodes the vote identifiers and the updated slots with their ballots, recomputes the results accumulator from the ballots it already holds, applies the updates to its tree, and requires the resulting root to equal the one recorded on-chain. Queued votes whose identifier appears in the tree are thereby settled. An observer runs the same procedure for every

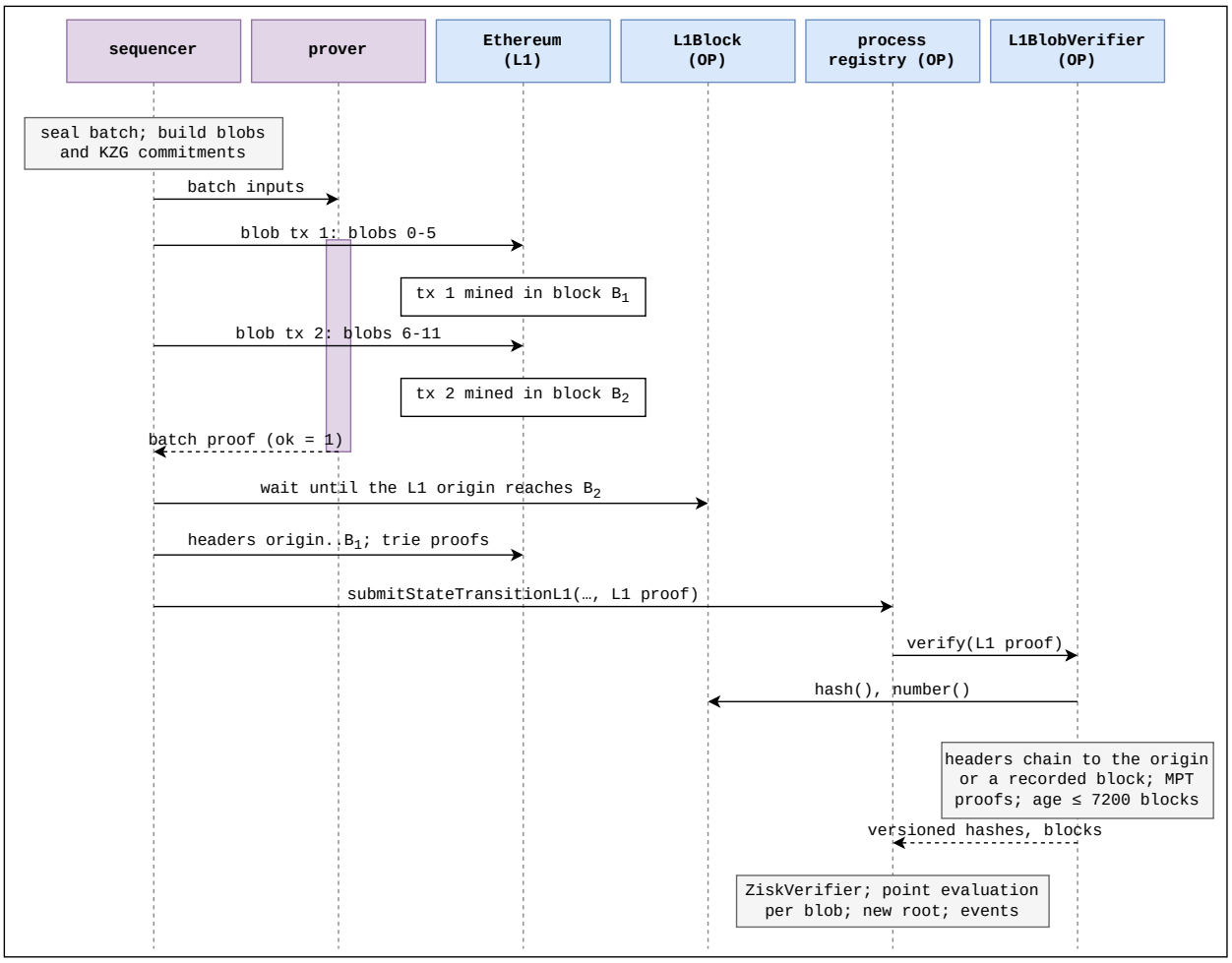


Fig. 14: Settlement in L1-blob mode. The blob transactions are posted to Ethereum while the batch is proven; the settlement on the rollup proves, from a chain of Ethereum headers anchored at the rollup's L1 origin, that they are part of the Ethereum chain.

transition, which checks the election, and its encrypted tally, independently of the sequencers. Sequencers apply transitions only once they are buried under a configurable number of blocks, and they detect deeper reorganizations of the settlement chain, in which case they roll back the affected transitions and replay the chain from the published blobs.

4.6.4 End of the voting period

The voting period ends at `startTime` + `duration`, or earlier if the organizer ends or cancels the process. From that moment, sequencers refuse new votes. Votes received before the end may still be queued or being proven, so the registry keeps accepting state transitions during a *grace window* that closes at

$$\texttt{graceEnd} = \min\big(\texttt{end} + \texttt{graceMaxTotal},\ \max(\texttt{end}, \texttt{lastVoteAt}) + \texttt{grace}\big), \tag{7}$$

where `end` is the end time and `lastVoteAt` the time of the latest settled transition. Every transition that lands during the window extends it, so a backlog drains batch by batch, while `graceMaxTotal` bounds the total extension. The organizer may set `grace` before the end within the bounds `graceFloor` and `graceCeil` of the registry, and may shorten the voting period, but only to an end time at least `noticeMin` away, so that every sequencer learns the new end before it applies. From shortly before the end, sequencers stop waiting to fill batches and seal everything pending as soon as the proving budget allows. Fig. 15 illustrates the window, and Fig. 16 the statuses a process goes through.

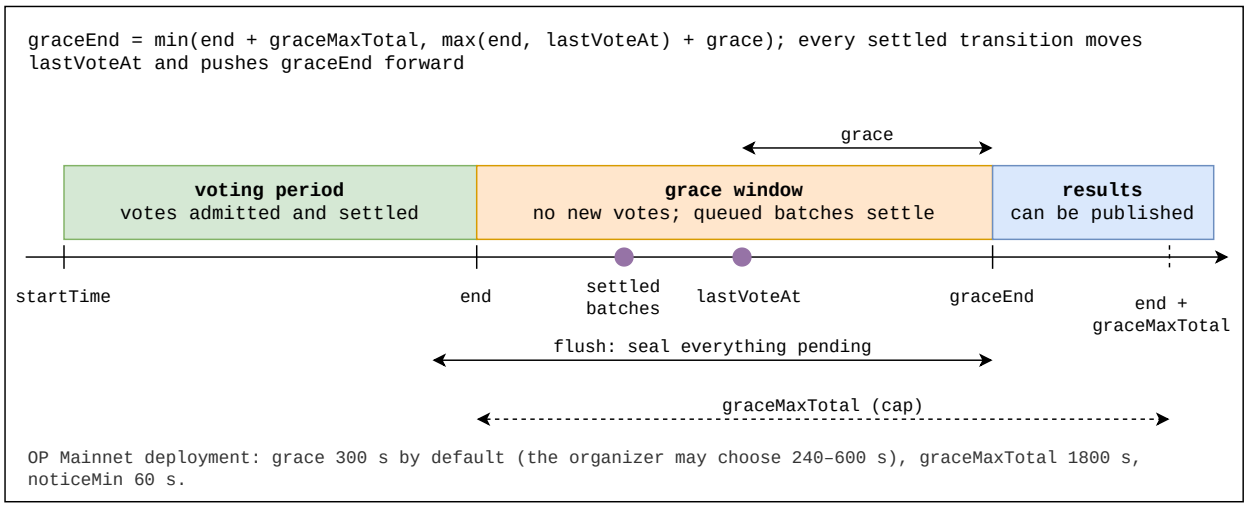


Fig. 15: The grace window. Admission closes at the end time; transitions keep settling until `graceEnd`, which every settlement pushes forward by `grace`, up to `graceMaxTotal` after the end. Results can only be published after `graceEnd`.

The grace window lets a sequencer include votes cast before the end but settled after it. The registry cannot tell a vote cast after the end from a delayed one, so a dishonest sequencer could include late votes during the window; sequencers refuse them, and the exposure is bounded by the window and visible, since every transition in the window is recorded on-chain with its time.

4.6.5 Tally decryption

After the grace window closes, the ProcessRegistry smart contract stops accepting new state updates, fixing the final state root and thus the aggregated encrypted results `encryptedResults`. Every sequencer that follows the election holds `encryptedResults` in its own state tree, and the results can only be requested or published from this point on, which guarantees that the decrypted tally is that of the final state. How the tally is decrypted depends on the key mode.

Threshold decryption (DKG and council modes). Any sequencer with a signing key submits the final accumulator `encryptedResults`, together with its inclusion proof $\texttt{inclusionProof}^{\texttt{encryptedResults}}$ under the latest state root, by calling `requestResultsDecryption`. The registry checks that every coordinate of `encryptedResults` is canonical, that SHA256(`encryptedResults`) is the value of leaf `0x4` under the latest state root, moves the process to the ended status, and submits the ciphertext of every active field to the key-management contract through its adapter. Fields whose ciphertext is the identity decrypt to zero under any key and are recorded as zero without being submitted. A threshold of θ out of ν key wardens then cooperate to decrypt each submitted ciphertext (C_1, C_2): rather than reconstructing the election secret key, each participating warden i publishes a partial decryption $D_i = \mathsf{sh}_i \cdot C_1$, where sh_i is its key share, together with a Groth16 proof that D_i is consistent with its committed share. Once θ valid partial decryptions are available, anyone combines them by Lagrange interpolation, $M = C_2 - \sum_i \lambda_i D_i$, recovers the plaintext m with $M = m \cdot G^{\text{BJ}}$ by a bounded discrete-logarithm search, and submits it; the key-management contract checks that m is the correct combination (with a ZK proof in the DKG network, and by direct computation in the council) before storing it. In the locked mode, partial decryptions are only accepted after the organizer reveals $\mathsf{sk}_{\mathsf{org}}$, and the combination also removes its contribution. In the council mode, they are only accepted once the ceremony's decryption gate has opened. Because only the aggregated results are decrypted and never the individual ballots, voter privacy is preserved throughout the election, and the secret key is never reconstructed by any single party. Moving the process to the ended status when the decryption is requested matters: the plaintexts become public on the key-management contract before the registry records them, and an organizer that could still cancel the process could otherwise veto a tally after reading it.

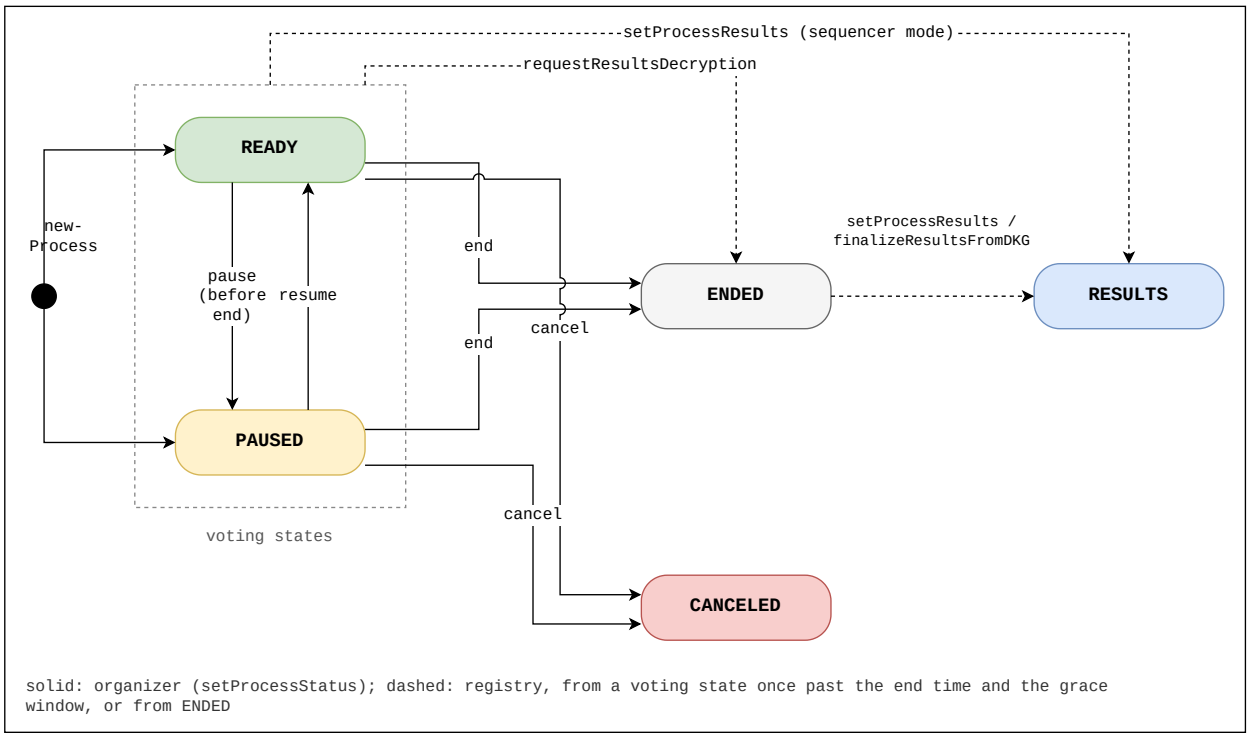


Fig. 16: Statuses of a process in the registry. Solid transitions are made by the organizer; dashed transitions are made by the registry when results are requested or published.

Sequencer decryption (sequencer mode). The sequencer that holds sk decrypts each field j of `encryptedResults` as $M_j = C_{2,j} - \mathsf{sk} \cdot C_{1,j}$, recovers results_j with $M_j = \text{results}_j \cdot G^{\text{BJ}}$ by a baby-step giant-step search bounded by $\texttt{maxVoters} \cdot \texttt{maxValue}$, and produces a Chaum–Pedersen proof π_j^{dec} that $\log_{G^{\text{BJ}}} \texttt{encryptionKey} = \log_{C_{1,j}} M_j$ (see Appendix A.6). It can do so during the grace window, once nothing is pending, and submit the result as soon as the window closes, provided that the root has not changed.

4.6.6 Election finalization

In this final phase, the election results are published on-chain, together with the evidence of their correct computation. Fig. 17 shows the messages of both finalization paths.

Sequencer mode. The key holder proceeds as follows:

1. Retrieve the encrypted results accumulator `encryptedResults` and the encryption key `encryptionKey` from the final state, together with their Merkle inclusion proofs against the final state root `stateRoot`.
2. Use the ResultsProgram from Section 4.5.3 to generate the results proof:

$$(\texttt{publicValues}, \texttt{resultsProof}) = \mathsf{VM.Prove}(\mathsf{ResultsProgram}, \mathsf{witness}), \quad \text{with}$$
$$\mathsf{witness} = \big(\texttt{encryptedResults}, \texttt{encryptionKey},$$
$$\texttt{inclusionProof}^{\texttt{encryptedResults}}, \texttt{inclusionProof}^{\texttt{encryptionKey}}, \{\text{results}_j, \pi_j^{\text{dec}}\}_{j=1}^{16}\big),$$

   whose public values contain `stateRoot` and the results.
3. Submit the public values and the proof to the ProcessRegistry smart contract with `setProcessResults`. The contract checks that the process is in the sequencer mode, that it has neither been canceled nor finalized, that it has ended and its grace window has closed, that $\texttt{ok} = 1$ and $\texttt{failMask} = 0$, and that the state root in the public values is the latest root of the process. It then verifies the proof with the ZiskVerifier under the pinned results program key, records the first `numFields` results, sets the status to `RESULTS`, and emits `ProcessResultsSet`.

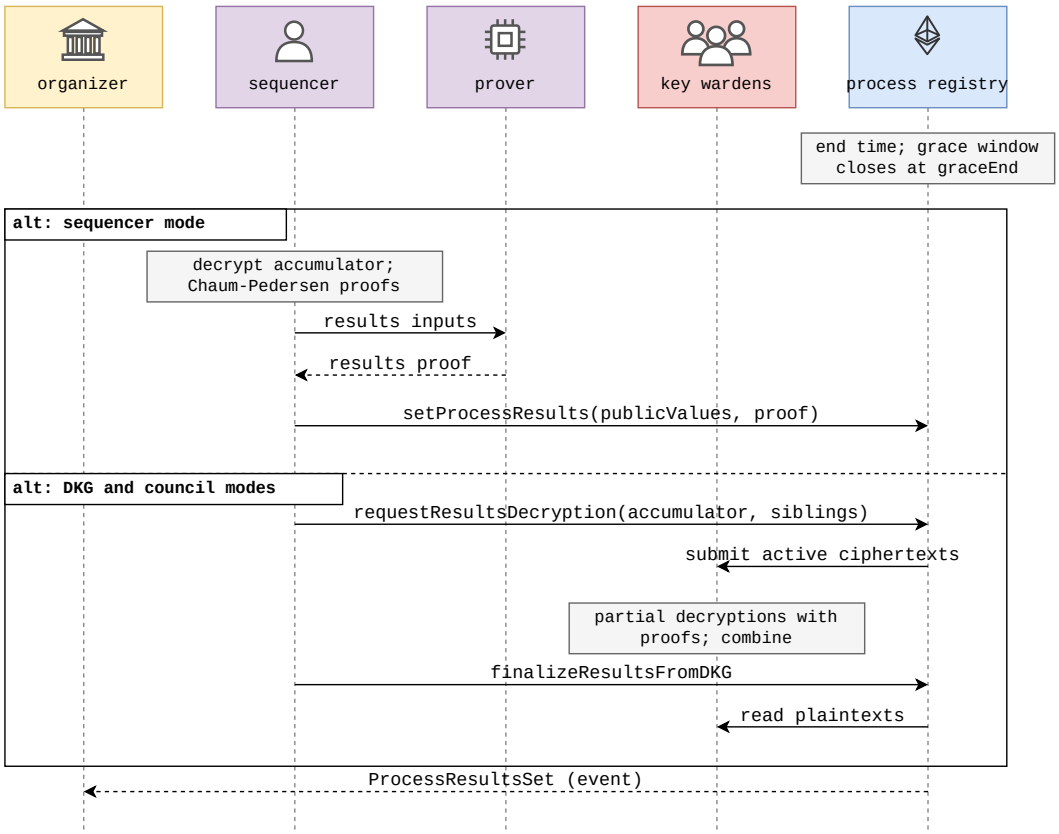


Fig. 17: Message flow of an election: decryption and finalization. The path depends on who holds the election key.

DKG and council modes. Once every submitted ciphertext has been decrypted, any sequencer calls `finalizeResultsFromDKG` for the process. The registry reads the stored plaintexts through the adapter, requires all of them to be available and, for a council process, the decryption gate to be open, fills in the zero-valued fields, records the results, sets the status to `RESULTS`, and emits `ProcessResultsSet`. Both this call and the request are permissionless, and every signing sequencer is ready to make them; to avoid duplicate transactions, each sequencer waits a random delay and sends a call only if it is still missing.
The election is considered finalized once the decrypted results are available on-chain. At this stage, both the results and the complete integrity of the process are permanently recorded and publicly auditable: anyone can recompute the on-chain commitments and check the proofs. In particular, the combination of the final state root, the verified decryption of the aggregated results, and the publicly verifiable proofs of every state transition and of the results guarantees the integrity of the entire election.

5 Ballot protocol

The ballot protocol provides a unified and parametric way to represent a wide range of voting systems within DAVINCI. Instead of designing a separate circuit for each voting rule, the protocol defines ballots as fixed-length arrays of integers, subject to a small set of configurable parameters. These parameters constrain the values that can appear in each field of the ballot and the aggregate properties of the ballot as a whole. By adjusting these parameters, the same circuit can implement approval voting, ranking, quadratic voting, multiple choice, and many other schemes. In the protocol we refer to the different configurations as the *ballot mode* (`ballotMode`). This abstraction has two key advantages. First, it allows a broad variety of voting systems to be supported with a minimal circuit design, avoiding conditional logic that would otherwise

increase constraint counts. Second, it provides a common interface for tallying, since all ballots are aggregated into a single results array regardless of the voting mode.

Definition of parameters. Each ballot is defined as an array of integer values subject to a set of configurable parameters, illustrated in Fig. 18. These parameters are defined as follows:

- `numFields`: the number of fields (i.e., options) in the ballot, between 1 and 16. The ballot circuit and the batch program always handle 16 fields; the fields beyond `numFields` are inactive.
- `groupSize`: the size of the groups into which an application may split the fields to present several questions in one ballot. It must not exceed `numFields`, and it does not change the constraints below.
- `minValue`: the minimum value that each field in the ballot can take.
- `maxValue`: the maximum value that each field in the ballot can take.
- `uniqueValues`: a Boolean flag indicating whether all field values must be different (as in ranking systems).
- `costExponent`: the exponent applied when computing the cost of casting votes in a field. This parameter enables quadratic or higher-order voting rules.
- `minValueSum`: the minimum allowed total sum of a ballot, computed as $\sum_{i=1}^{\texttt{numFields}} v_i^{\texttt{costExponent}}$, where v_i are the field values.
- `maxValueSum`: the maximum allowed total sum of a ballot, computed as $\sum_{i=1}^{\texttt{numFields}} v_i^{\texttt{costExponent}}$, where v_i are the field values (e.g., to enforce a budget of credits). The value zero means that the bound is the voter's weight.

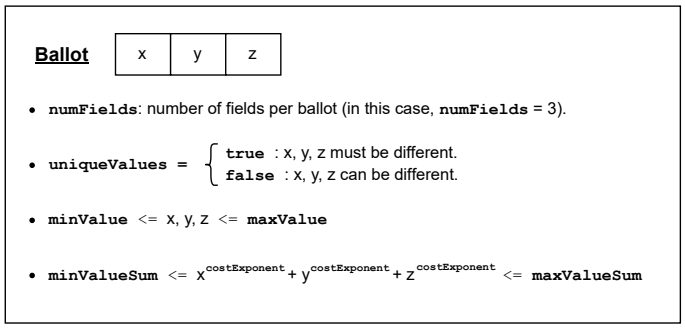


Fig. 18: Schematic representation of the parameters that define a ballot in DAVINCI.

The ballot mode is stored in the state tree, and given to the ballot circuit, packed into a single integer below 2^{247}, as shown in Fig. 19. The packing fixes the range of every parameter: `maxValue` and `minValue` are below 2^{48}, `minValueSum` and `maxValueSum` below 2^{63}, and `costExponent`, `numFields`, and `groupSize` below 2^8. When a process is created, the registry rejects a ballot mode that does not fit this packing or that violates $1 \leq \texttt{numFields} \leq 16$, $\texttt{groupSize} \leq \texttt{numFields}$, $\texttt{minValue} \leq \texttt{maxValue}$, or $\texttt{minValueSum} \leq \texttt{maxValueSum}$, and it rejects a process whose worst-case tally could not be decrypted, i.e., one with $\texttt{maxValue} \cdot \texttt{maxVoters} > 10^{12}$.

Definition of constraints. Given a ballot with $n_f = \texttt{numFields}$ active fields, each filled with a value v_i for $i = 1, \ldots, n_f$, and cast by a voter with weight `weight`, we enforce the following constraints (captured in Fig. 18):

- $\texttt{groupSize} \leq \texttt{numFields} = n_f$,
- If $\texttt{uniqueValues} = \texttt{true}$, then $v_i \neq v_j$ for all $i \neq j$.
- $\texttt{minValue} \leq v_i \leq \texttt{maxValue}$ for all $i \in \{1, \ldots, n_f\}$.
- $\texttt{minValueSum} \leq \sum_{i=1}^{n_f} v_i^{\texttt{costExponent}} \leq \mathsf{max}$, where $\mathsf{max} = \texttt{maxValueSum}$ if $\texttt{maxValueSum} > 0$, and $\mathsf{max} = \texttt{weight}$ otherwise.

The inactive fields $n_f < i \leq 16$ are not constrained by the ballot circuit, but the batch program requires their ciphertexts to be the identity, so they always contribute zero to the tally.

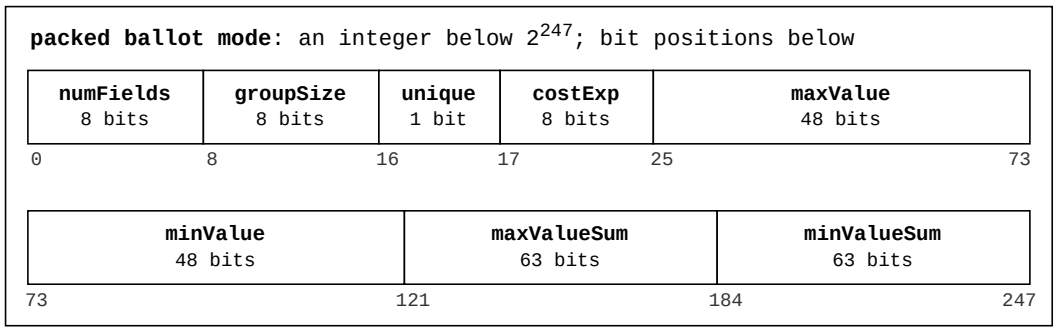


Fig. 19: Packing of the ballot mode into one integer. Numbers below the fields are the position of their least significant bit.

General mechanism. A ballot is valid if and only if all the above constraints are satisfied. Invalid ballots are rejected at the circuit level and do not contribute to the tally. Valid ballots are aggregated into a results array, where each entry is the sum of all votes cast for the corresponding field across all voters. Voter weights, defined in the census (see Section 4.3), enter the ballot rules as a budget: when `maxValueSum` is zero, a voter may distribute at most their weight across the fields, so the weight bounds each voter's contribution to the tally. By default, all voters have the same weight. The ballot circuit enforces the bound with the weight committed in `inputsHash`, and the batch program checks that this weight is the one certified by the census (see Sections 4.5.1 and 4.5.2 for further details).

Ballot modes. Ballot modes correspond to different voting systems, each adjusted with the variables above ballot configurations. For usability, the organizer does not need to set every parameter manually: instead, they simply select one of the predefined ballot modes (e.g., quadratic voting, ranking, approval), each corresponding to a fixed parameter configuration. Some representative modes are shown in Table 4, which illustrates the expressiveness of the ballot protocol. For example, in *approval voting*, each field is binary and voters can approve or reject multiple options simultaneously. In *ranking*, fields must form a permutation of the integers from 1 to `numFields`, enforcing uniqueness of values. In *quadratic voting*, voters allocate credits across options, and the cost is quadratic in the number of credits used, captured by setting `costExponent` = 2. Other systems such as single-choice or multiple-choice elections are special cases of the same framework. Table 5 illustrates examples of voters' ballots, with the last column indicating whether each ballot is valid or invalid under the rules of the selected mode, and the tally of the valid ones.

Table 4: Voting systems expressed as ballot modes. Each row is a fixed configuration of the ballot parameters; `groupSize` is zero in all of them.

Mode	Description	Example	numFields	minValue	maxValue	uniqueValues	costExponent	minValueSum	maxValueSum
Approval	Several fields, each 0 or 1	A referendum with yes/no questions	5	0	1	false	1	0	5
Rating	Each field rated independently	Rate 5 items from 0 to 10 stars	5	0	10	false	1	0	50
Ranking	Number the options from 1 to n_f, each number once	Rank 5 destinations for a school trip	5	1	5	true	1	6	15
Quadratic	Distribute credits among the options, at quadratic cost	Distribute 12 credits among 5 options	5	0	12	false	2	0	12
Single choice	Select one option	Select one winner among 5 candidates	5	0	1	false	1	1	1
Multiple choice	Select up to a given number of options	Select up to 3 finalists among 5 candidates	5	0	1	false	1	0	3

Table 5: Example ballots for the modes of Table 4. For each ballot, the cost sum $\sum_i v_i^{\texttt{costExponent}}$ and whether it is valid (1) or invalid (0) under the mode; the last row of each mode is the tally of its valid ballots.

Mode		v_1	v_2	v_3	v_4	v_5	Cost sum	Valid
Approval	ballot 1	0	1	0	1	1	3	1
	ballot 2	1	1	1	1	1	5	1
	ballot 3	0	1	0	0	0	1	1
	tally	**1**	**3**	**1**	**2**	**2**		**3**
Rating	ballot 1	8	6	4	7	10	35	1
	ballot 2	5	4	6	12	4	31	0
	ballot 3	0	1	3	5	2	11	1
	tally	**8**	**7**	**7**	**12**	**12**		**2**
Ranking	ballot 1	1	3	2	4	5	15	1
	ballot 2	1	5	3	4	5	18	0
	ballot 3	0	1	3	5	2	11	0
	tally	**1**	**3**	**2**	**4**	**5**		**1**
Quadratic	ballot 1	1	1	2	0	0	6	1
	ballot 2	3	0	0	0	2	13	0
	ballot 3	3	1	0	1	0	11	1
	tally	**4**	**2**	**2**	**1**	**0**		**2**
Single choice	ballot 1	1	0	0	0	0	1	1
	ballot 2	0	1	0	0	0	1	1
	ballot 3	0	1	1	0	0	2	0
	tally	**1**	**1**	**0**	**0**	**0**		**2**
Multiple choice	ballot 1	1	0	1	1	0	3	1
	ballot 2	0	1	1	0	1	3	1
	ballot 3	1	2	3	0	0	6	0
	tally	**1**	**1**	**2**	**1**	**1**		**2**

6 Analysis

6.1 Security discussion

Based on the above principles, the protocol provides a number of concrete security properties that ensure the integrity, confidentiality, and verifiability of voting events. In what follows we discuss how DAVINCI achieves these properties, with particular emphasis on integrity, receipt-freeness, and ballot privacy, before turning to data availability, liveness, end-to-end verifiability, and robustness against quantum threats. Table 6 summarizes the parties that each property relies on.

Integrity and soundness. The integrity of the election rests on the soundness of the proofs rather than on trust in any operator: the knowledge soundness of Groth16 for ballot proofs, and the soundness of the ZisK STARKs and of the PlonK wrapper for the batch and results programs. No party (including a malicious sequencer) can advance the state from S_i to S_{i+1} without supplying a valid batch proof, since the registry rejects any update not accompanied by such a proof. Because a zkVM proves the execution of a program, the statement that is proven is exactly what the program checks, and the registry pins the program through its verification key: an update proven by any other program, under any other setup, or with a failed verdict ($\texttt{ok} \neq 1$) is rejected. The registry also binds every proof to the election: the old root must be the latest root, the census root must be one the election accepts, the count of occupied slots must match, and the blob openings must match the published blobs. Consequently, every counted ballot originates from an eligible census member and every accumulator update follows the protocol rules. Double voting is prevented *without* nullifiers: each voter is bound to a unique, census-authenticated slot in the state tree, so a second submission by the same voter overwrites their previous ballot at that slot rather than adding a new contribution to the tally. Vote identifiers, by contrast, are derived per ballot and therefore differ across submissions, which is

precisely what enables individual verifiability without linking a voter to their ciphertext. The guest programs are built reproducibly, so anyone can recompile the published source, recompute the program keys, and compare them with the keys pinned by the registry.

Receipt-freeness. Receipt-freeness prevents voters from proving to others how they voted, thereby mitigating coercion and vote-buying. In DAVINCI this is achieved through re-encryption, ballot overwrites, and randomized state updates.

- *Ballot re-encryption*: since our encryption scheme supports re-randomization, that is, a ciphertext can be refreshed with new randomness without changing the underlying message, sequencers re-encrypt the received ballots before committing them to the state Merkle tree. Under the decisional Diffie–Hellman (DDH) assumption on BabyJubjub, a re-randomized ciphertext is indistinguishable from a fresh encryption, so linking a submitted ciphertext to its stored counterpart is as hard as solving DDH. The re-encryption scalars are derived inside the batch program from a single secret seed per batch and the old state root, and every scalar is consumed exactly once, so no scalar can repeat within or across batches.
- *Handling receipts*: because ballots are re-randomized, voters cannot generate a receipt by revealing the randomness k used in encryption, since the ciphertext on-chain no longer corresponds to their k. This blocks vote-selling and coercion.
- *Overwrites*: voters may cast a new ballot at any time, replacing their earlier submission. This ensures that even if coercion occurs, a voter can subsequently change their vote as many times as desired, preserving their ability to express their true choice. When an overwrite occurs, the batch subtracts the previous ballot and adds the new one to the results accumulator, and re-encrypts the updated ballot before storing it in the state tree.
- *Concealing overwrites (silent revoting)*: to prevent observers from distinguishing overwrites from routine re-randomizations, every batch also re-encrypts a uniformly random sample of the occupied slots it does not write, with a size that the batch program enforces, $\min(2048, \max(16, 2w, n))$ slots when available. The blobs list every updated slot in a single sorted list, so an overwrite is indistinguishable from a refresh. Sequencers keep a history of the batches that carried each vote, so that a vote sealed again after a lost race or a failed settlement refreshes every slot of its earlier batches and cannot be singled out by comparing them.

Receipt-freeness holds against observers of the public data, under two conditions. First, the sequencer that builds a batch, and its prover, learn which slots of the batch are overwrites and which are refreshes, and must not reveal the seed or the refresh selection; a voter who fears coercion should overwrite through a sequencer they trust. Second, the first write to a slot is distinguishable, since refreshes only touch occupied slots, so participation is not hidden (see the privacy paragraph). A further, minor limitation is that re-encryption scalars are 256-bit SHA-256 digests reduced modulo the 254-bit order s of the BN254 scalar field, which leaves them at statistical distance about $2^{-4.7}$ from uniform in $[0, s)$; after the implicit reduction modulo the subgroup order $t \approx s/8$, the scalars actually applied are at statistical distance about $2^{-7.6}$ from uniform. Removing this gap requires reducing a wider digest, and hence a new program key.

Privacy. Ballots remain secret even though encrypted ballots are publicly stored and processed. Ballot secrecy is preserved through ElGamal encryption, which allows votes to be aggregated without decryption. When the encryption key is generated through a DKG (or a council ceremony) and its secret counterpart is never reconstructed, no single party can decrypt individual ballots: the protocol only invokes threshold decryption on the final aggregate, never on each ballot separately. Under the assumption that at most $\theta - 1$ of the ν key wardens are compromised, the shared secret key cannot be reconstructed and no individual ballot ciphertext can be decrypted. A coalition of θ key wardens, however, could decrypt every ballot published in the blobs, so the threshold must be chosen to reflect the trust placed in the committee; in the locked DKG mode, the coalition would also need the organizer's secret. In the sequencer mode, the sequencer that holds the key can decrypt every ballot, which is why this mode is reserved for elections that accept that trust. Note that, while the content of a ballot stays hidden, the protocol does reveal whether a given voter has cast a vote, that is, turnout is public, but the choice is not. With a Merkle census, the slot of a voter is a public function of their address, so the participation of each census member can be read from the state.

Data availability. Ballots and state updates are published in Ethereum blobs, and the registry checks, at settlement, that the published blobs open to the values the batch program derived from the verified state. A sequencer therefore cannot settle a transition whose data it withholds. In L1-blob mode, the guarantee is the same: the blobs live on Ethereum, and the registry on the rollup verifies their inclusion in the Ethereum chain from block headers anchored at the rollup's view of Ethereum, so it relies, in addition, on the rollup deriving its L1 origin correctly, an assumption the rollup itself already makes. Since blobs are pruned after about 18 days, long-term availability requires archival: every sequencer stores and serves the blobs of the transitions it has seen, and observers can rebuild any election from them, but a node that starts after the blobs have been pruned depends on such an archive. Since EIP-7594, only nodes that custody full blobs can serve them, so sequencers should rely on such nodes or on archivers.

Liveness and censorship resistance. Sequencing is permissionless, so a voter whose ballot is ignored by one sequencer can submit it to another, and anyone can run a sequencer for any election. Settlement is a race in which the first valid transition wins; a sequencer that loses only wastes its own work. The grace window lets ballots received before the end of the voting period settle after it, and its extension is capped by Eq. (7), so the results cannot be delayed indefinitely. Once results are requested from the key wardens, the process leaves the organizer's control and can no longer be canceled. The liveness of threshold decryption depends on the key wardens: if more than $\nu - \theta$ members of the committee that holds an election's key become unavailable before the tally is decrypted, the results are lost, since keys are not reshared to later committees. In the sequencer mode, the results depend on the availability of the single key holder.

Quantum resistance. Quantum computers threaten discrete-logarithm-based cryptography such as ECDSA and ElGamal, and pairing-based proof systems such as Groth16 and PlonK. The STARKs that prove the batch and results programs rely only on hash functions and are plausibly post-quantum, but their PlonK wrapper, the ballot proofs, the voters' signatures, and the encryption of ballots are not. For this reason, DAVINCI is designed in a modular way that would allow migration to post-quantum primitives in the longer term. Candidate replacements include lattice-based signature schemes such as CRYSTALS-Dilithium (ML-DSA) and Falcon, and the hash-based SPHINCS+ (SLH-DSA); lattice-based homomorphic encryption such as BGV or BFV; and the direct on-chain verification of STARKs.

End-to-end verifiability. Voters can check that their own ballot was included correctly, while anyone can audit the entire process to verify the final tally. Every voter can check, with an inclusion proof of their vote identifier against a state root recorded on-chain, that their ballot was recorded as cast (individual verifiability). Additionally, any third party can replay every state transition from the published blobs, check that it reproduces the roots recorded on-chain, and verify the proofs, which confirms the results (universal verifiability) and that each counted ballot comes from a registered voter (eligibility verifiability). The protocol does not provide, by itself, a mechanism for a voter to verify that their device encrypted the intended choice (cast-as-intended verifiability); this property can be added by the client application, for instance with a second device that audits a ballot before it is cast.

6.2 Implementation

DAVINCI is implemented as a modular, fully open-source system: the ballot circuit [48], the zkVM programs [52], the smart contracts [47], the sequencer node and its client library [51], and the key-management systems [50,49] are all publicly available at `https://github.com/vocdoni`. Its components can be recombined or integrated with external services through adaptable interfaces, enabling redundancy and integration with third-party applications such as our voting-as-a-service APIs.

Technological stack. The ballot circuit is implemented with Circom [4] over BN254 and proven with Groth16 [27] for its minimal proof size and cheap verification; its ElGamal operations run over BabyJubjub, whose base field matches the BN254 scalar field. The batch and results programs are `no_std` Rust programs compiled for the RISC-V target of ZisK 1.3.2 and proven by a GPU prover service, which wraps every proof in PlonK over BN254 for on-chain verification. Inside the programs, SHA-256, Keccak, BN254 pairings and scalar multiplications, SECP256K1 key recovery, BabyJubjub point additions, and 256-bit modular arithmetic run on ZisK precompiles; Poseidon is implemented in software over the BN254 field. The smart contracts are written in Solidity, and the sequencer node and its client library in Rust.

Table 6: Trust assumptions of the main properties.

Property	Holds if
Integrity of the tally	The proof systems are sound, the pinned programs and circuits are correct, and the settlement chain executes the registry correctly. No sequencer, prover, or key warden is trusted.
Ballot secrecy	Fewer than θ key wardens collude (DKG and council modes), or the key-holding sequencer is honest (sequencer mode); DDH holds on BabyJubjub.
Receipt-freeness	As ballot secrecy, and the sequencer that seals an overwrite does not reveal its seed and refresh selection.
Data availability	Ethereum makes the blobs available at settlement; archivers retain them after pruning. L1-blob mode also relies on the rollup's L1 origin.
Liveness	At least one honest sequencer serves the election; at least θ key wardens are available for decryption (or the key-holding sequencer, in sequencer mode).

Circuit and program sizes. The ballot circuit has 64,816 constraints for 16 fields; its proving key is 35.5 MiB and its witness generator 6.5 MiB, which a browser downloads before proving. A zkVM program has no fixed size: its cost is the number of executed steps, which grows with the batch. A batch program execution with 512 votes and 512 refreshes takes about $2.3 \cdot 10^8$ steps at $n_f = 2$ and $4.0 \cdot 10^8$ steps at $n_f = 16$. The program accepts batches of up to 1024 votes, 2048 refreshes, and 32 blobs; sequencers currently seal at most 512 votes per batch.

Trusted setup. Groth16 relies on a circuit-specific common reference string (CRS, see Appendix A.8). The parameters of the ballot circuit must be generated through an MPC ceremony in two phases: a universal powers-of-tau phase, for which a public ceremony of 2^{18} constraints can be reused, followed by a per-circuit phase that binds the parameters to the constraint system and introduces the secret δ. As long as at least one participant in each phase discards their secret contribution, the resulting CRS is secure. The zkVM STARKs need no setup, and their PlonK wrapper uses the universal setup distributed with the ZisK release. The circuits of the DKG network and of the council have setups of their own.

Release pins. A release of the protocol is identified by the values in Table 7, which the registry stores as immutable parameters and sequencers check at startup. A sequencer refuses a registry, a verifier contract, or a prover whose values differ.

Table 7: Release pins of the protocol version described in this paper (davinci-zkvm v0.3.0, ZisK 1.3.2).

Value	Hash
Batch program key	`0xe39e460085ecddecf365649a61183a71c0fe778aa9c06fd2aa1a6c43cddb988b`
Results program key	`0x394694558f39d0b2da0bbb09322333a0f567ee8d16bea2f354543a086587d167`
`rootCVadcopFinal`	`0x6b2490fba336fb00f0c8719a961de309a8db6f43a9bfb7575ca5f014d609b265`
`ballotVKHash`	`0xbf1e6590bb1ba883d601c4d7d1c6fa2722a78590716874019db6d68fc776bb0e`
ZiskVerifier code hash	`0xadab570d3e3a4c172d0c75e5aef5d06098b94d70a9c8ff76af239a90fdcadcc1`

Implementation status. The current implementation includes the ballot circuit, the batch and results programs, the smart contracts, the sequencer node, and both key-management systems, and it runs in production on OP Mainnet in L1-blob mode, with all four key modes enabled. The registry is deployed at `0x64d41aD49009204E8dAcc153Fc98a5Ee14cB6916`, with a default grace window of 300 s (organizers may choose between 240 s and 600 s), a maximum extension of 1800 s, and a minimum notice of 60 s. The council circuits currently use a development setup produced by a single party, which is acceptable for a beta but must be replaced by a multi-party ceremony. No complete independent security audit of the system has been published yet.

6.3 Performance evaluation

We evaluate DAVINCI along four axes: voter-side proving, sequencer proving, on-chain verification, and data availability, together with the end-to-end throughput of the system. Unless stated otherwise, sequencer measurements use one NVIDIA RTX 5090 GPU (32 GB) on a host with an AMD Ryzen 9 9950X3D CPU.

Voter proving. A ballot proof is generated with snarkjs; natively, one 16-core CPU produces about six proofs per second, and a single proof takes about 0.16 s. In a browser, proving is slower and is dominated by the download of the proving artifacts, but it requires no specialized hardware and remains within the latency a voter tolerates at casting time.

Sequencer proving. Table 8 and Fig. 20 report the time to prove one batch, including witness generation, the STARK proof, its recursion, and the PlonK wrapper. The *first* batch of an election has nothing to refresh; a *steady* batch refreshes as many slots as it writes, which is the regime that determines throughput. The fixed cost per batch is about 15.5 s, and each additional refresh costs about 13 ms at $n_f = 2$ and 40 ms at $n_f = 16$. Throughput peaks around 512 votes per batch at $n_f = 2$ and 256 at $n_f = 16$; beyond that, the batch size is bounded by the host memory of the prover (about 54 GB for a steady batch of 1024 votes), not by the GPU. The proof is 768 bytes, plus 512 bytes of public values, at every batch size. These figures were measured on ZisK 1.3.0, the predecessor of the release in Table 7, with the same batch program logic.

Table 8: Proving time of the batch program on one RTX 5090 (seconds), and blobs needed by a steady batch.

	$n_f = 2$			$n_f = 16$		
Votes	First	Steady	Blobs	First	Steady	Blobs
64	22.0	22.5	1	27.4	30.3	2
128	28.5	30.1	1	40.7	45.7	3
256	42.6	46.2	1	66.3	76.7	5
512	76.6	84.3	2	139.4	164.3	9
1024	154.5	171.2	3	311.0	363.9	17

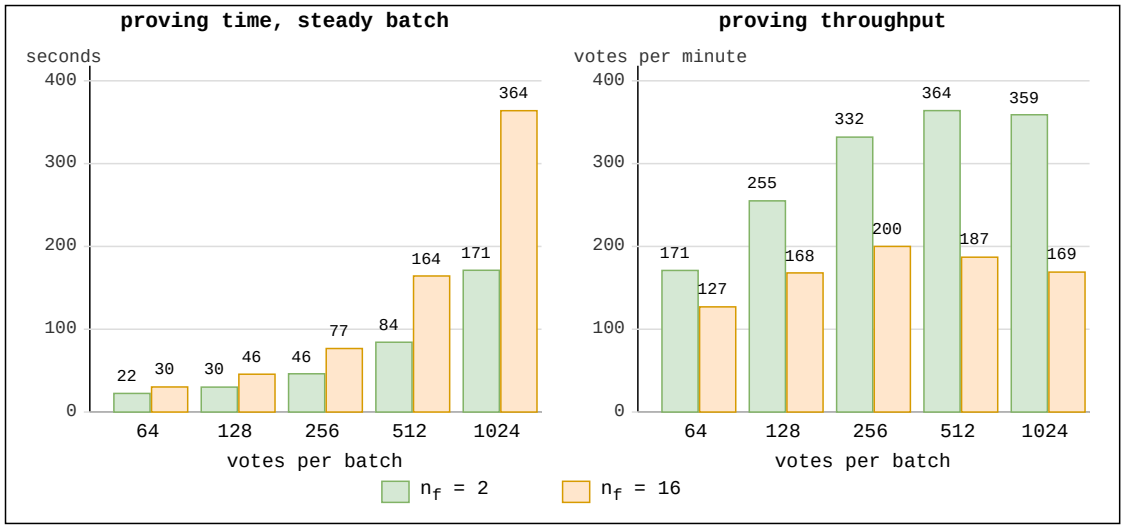


Fig. 20: Proving time of a steady batch (votes plus as many refreshes) and the resulting proving throughput, on one RTX 5090.

End-to-end throughput. With two provers, each serving one sequencer that runs two elections of 2048 voters at $n_f = 2$ in batches of 512, the system proved and settled 8192 votes in 678.7 s, a sustained throughput of 12.07 votes per second (724 per minute), on a local chain with one-second blocks. Each prover was busy for the whole run, so throughput scales with the number of provers: one RTX 5090 settles about six votes per second at $n_f = 2$.

On-chain verification. Thanks to the succinctness of PlonK, verifying a batch proof costs an essentially constant amount of gas, independent of the number of ballots in the batch; the cost of a settlement grows only with the number of blobs, by one point evaluation each. In blob mode, measured with the same contracts on Gnosis Chain, which supports blobs, a settlement with one blob costs 0.40–0.44M gas and one with two blobs 0.46–0.50M gas. On OP Mainnet in L1-blob mode, verifying the L1 proof with three Ethereum headers costs 430K–460K gas, and a whole settlement 800K–860K gas.

Data availability. DAVINCI publishes the data needed to reconstruct each state transition through EIP-4844 blobs. Compared with calldata, blobs substantially reduce data-availability costs by separating large protocol data from permanent Ethereum state. A blob holds 4096 cells; a steady batch needs $1 + \lceil 3n/4 \rceil + 4n_f n$ cells for n new votes and n refreshes, so a batch of 128 votes at $n_f = 2$ fits in a single 128 KiB blob, while 512 votes at $n_f = 16$ need nine. In the measurements on OP Mainnet in L1-blob mode, in October 2026, posting the blobs of a batch cost 1.9–3.7 μETH on Ethereum and settling it 0.04–0.05 μETH on OP Mainnet, and batches settled 85–115 s after being sealed, a delay dominated by the time the rollup takes to adopt a new Ethereum block as its L1 origin.

Together, these results indicate that DAVINCI scales to large elections at predictable settlement costs and practical proving times. Since proving dominates and parallelizes across provers and elections, throughput grows with the proving capacity of the sequencers, while the settlement chain only verifies one constant-size proof per batch.

7 Conclusions

We presented DAVINCI, a decentralized, E2E-verifiable voting protocol implemented as a specialized ZK rollup. By modeling elections as ZK-verified state transitions settled on Ethereum or on an Ethereum rollup, DAVINCI combines public auditability with scalable off-chain execution while minimizing reliance on any single party. Ballot secrecy is achieved through threshold ElGamal encryption under a key generated by a permissionless DKG network or an invited council, while ciphertext re-randomization and silent revoting strengthen protection against coercion and vote-selling.

The central change of this version of the protocol is the use of a zkVM: the work of a sequencer, from the verification of every ballot proof, signature, and census proof to the update of the state, the re-encryption of the ballots, and the layout of the published data, is a single program whose execution is proven and verified on-chain with one constant-size proof. Expressing the transition function as a program made it practical to prove properties that the earlier circuit-based design left outside the proof, such as the exact content of the data blobs and the silent refresh of stored ballots, and it makes the statement that is enforced auditable as ordinary code. The data availability layer is decoupled from the settlement layer, so elections can settle on a low-cost rollup while their data remains on Ethereum.

To our knowledge, DAVINCI is the first system to combine a universal ballot abstraction, an open sequencer set, and fully proof-enforced election execution within a single rollup architecture. This enables a reusable voting infrastructure in which elections with different ballot formats and organizers can share the same settlement layer, verifier contracts, and proving pipeline.

Several directions remain open for future work. An important long-term direction is post-quantum migration, in particular the direct verification of STARKs and post-quantum signature and encryption schemes. Others are the chained proving mode, which folds a whole election into one on-chain verification, for elections that do not need per-batch settlement; the resharing of election keys across DKG committees, so that a key outlives the committee that generated it; decentralized archival of blobs beyond their retention period; and a cast-as-intended mechanism for voters. We also plan to integrate with real-world credential providers and conduct large-scale deployment experiments in realistic electoral settings.

Acknowledgments

The authors would like to thank the following reviewers and contributors for their valuable feedback and support: all team members from the Vocdoni association, Jordi Baylina (Iden3 and Polygon), Adrià Massanet (Privacy Scaling Explorations, Ethereum Foundation), Arnaucube (0xPARC), Javier Herranz (Polytechnic University of Catalonia), Jordi Puiggali (Secrets Vault), and Carla Ràfols (Pompeu Fabra University).

References

1. Adida, B.: Helios: Web-based open-audit voting. In: 17th USENIX Security Symposium (USENIX Security 08). pp. 335–348. USENIX Association, San Jose, CA (2008), https://www.usenix.org/legacy/event/sec08/tech/full_papers/adida/adida.pdf
2. Baylina, J., Bellés, M.: Sparse Merkle trees (2019), https://docs.iden3.io/publications/pdfs/Merkle-Tree.pdf
3. Bellare, M., Garay, J.A., Rabin, T.: Fast batch verification for modular exponentiation and digital signatures. In: Advances in Cryptology – EUROCRYPT '98. Lecture Notes in Computer Science, vol. 1403, pp. 236–250. Springer (1998)
4. Bellés-Muñoz, M., Isabel, M., Muñoz-Tapia, J.L., Rubio, A., Baylina, J.: Circom: A circuit description language for building zero-knowledge applications. IEEE Transactions on Dependable and Secure Computing (2023). https://doi.org/10.1109/TDSC.2022.3232813
5. Bellés-Muñoz, M., Whitehat, B., Baylina, J., Daza, V., Muñoz-Tapia, J.L.: Twisted edwards elliptic curves for zero-knowledge circuits. Mathematics **9**(23) (2021). https://doi.org/10.3390/math9233022, https://www.mdpi.com/2227-7390/9/23/3022
6. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Fast Reed-Solomon interactive oracle proofs of proximity. In: 45th International Colloquium on Automata, Languages, and Programming (ICALP 2018). LIPIcs, vol. 107, pp. 14:1–14:17 (2018)
7. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046 (2018), https://eprint.iacr.org/2018/046
8. Benaloh, J., Naehrig, M., Pereira, O., Wallach, D.S.: ElectionGuard: a cryptographic toolkit to enable verifiable elections. In: 33rd USENIX Security Symposium (USENIX Security 24). pp. 5485–5502. USENIX Association, Philadelphia, PA (Aug 2024), https://www.usenix.org/conference/usenixsecurity24/presentation/benaloh
9. Benaloh, J., Tuinstra, D.: Receipt-free secret-ballot elections (extended abstract). In: Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing (STOC). pp. 544–553 (1994)
10. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: The KECCAK SHA-3 submission (2011), https://keccak.team/files/Keccak-submission-3.pdf
11. Blake, I.F., Seroussi, G., Smart, N.P.: Elliptic curves in cryptography. Cambridge University Press, USA (1999)
12. Brown, D.R.L.: SEC 2: Recommended elliptic curve domain parameters. In: Standards for efficient cryptography 2 (SEC 2) (2010), https://www.secg.org/sec2-v2.pdf
13. Bruestle, J., Gafni, P., RISC Zero Team: RISC Zero zkVM: Scalable, transparent arguments of RISC-V integrity. RISC Zero technical report (2023), https://dev.risczero.com/proof-system-in-detail.pdf
14. Buterin, V.: Minimal anti-collusion infrastructure. Ethereum Research Forum (ethresear.ch) (2019), https://ethresear.ch/t/minimal-anti-collusion-infrastructure/5413
15. Buterin, V., Feist, D., Loerakker, D., Kadianakis, G., Garnett, M., Taiwo, M., Dietrichs, A.: EIP-4844: Shard blob transactions, Ethereum Improvement Proposals, no. 4844, [online serial] (February 2022), https://eips.ethereum.org/EIPS/eip-4844
16. Canetti, R.: Universally composable security: a new paradigm for cryptographic protocols. In: Proceedings 42nd IEEE Symposium on Foundations of Computer Science. pp. 136–145 (2001). https://doi.org/10.1109/SFCS.2001.959888
17. Chaum, D., Pedersen, T.P.: Wallet databases with observers. In: Advances in Cryptology – CRYPTO '92. Lecture Notes in Computer Science, vol. 740, pp. 89–105. Springer (1992)
18. Chaum, D.L.: Untraceable electronic mail, return addresses, and digital pseudonyms. Communications of the ACM **24**(2), 84–88 (1981)
19. Cortier, V., Gaudry, P., Glondu, S.: Belenios: A simple private and verifiable electronic voting system. In: Foundations of Security, Protocols, and Equational Reasoning. LNCS, vol. 11565, pp. 214–238. Springer, Cham (2019)
20. Doan, T.V.T., Pereira, O., Peters, T.: Threshold receipt-free single-pass eVoting. In: Electronic Voting – E-Vote-ID 2024. LNCS, vol. 15014. Springer (2024)

21. Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: 28th Annual Symposium on Foundations of Computer Science (FOCS 1987). pp. 427–438. IEEE (1987)
22. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Conference on the theory and application of cryptographic techniques. pp. 186–194. Springer (1986)
23. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PlonK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019)
24. Gennaro, R., Gentry, C., Parno, B., Raykova, M.: Quadratic span programs and succinct nizks without pcps. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 626–645. Springer (2013)
25. Glaeser, N., Seres, I.A., Zhu, M., Bonneau, J.: Cicada: A framework for private non-interactive on-chain auctions and voting. Cryptology ePrint Archive, Paper 2023/1473 (2023)
26. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 519–535 (2021)
27. Groth, J.: On the size of pairing-based non-interactive arguments. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 305–326. Springer (2016)
28. Gurkan, K., Wei Jie, K., Whitehat, B.: Semaphore: Zero-knowledge signaling on Ethereum. ZKProof Community Proposal, Workshop 3 (2020), https://semaphore.pse.dev/whitepaper-v1.pdf
29. Hirt, M., Sako, K.: Efficient receipt-free voting based on homomorphic encryption. In: Advances in Cryptology – EUROCRYPT 2000. LNCS, vol. 1807, pp. 539–556. Springer (2000)
30. Johnson, D., Menezes, A., Vanstone, S.: The elliptic curve digital signature algorithm (ECDSA). Int. J. Inf. Secur. **1**(1), 36–63 (August 2001). https://doi.org/10.1007/s102070100002, https://doi.org/10.1007/s102070100002
31. Kampa, A., Escrich, P., Bellés-Muñoz, M., Baig, R.: NI-DKG: Non-interactive distributed key generation using blockchain and zero-knowledge proofs. Cryptology ePrint Archive, Paper 2026/552 (2026), https://eprint.iacr.org/2026/552
32. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: International conference on the theory and application of cryptology and information security. pp. 177–194. Springer (2010)
33. Kirillov, D., Korkhov, V., Petrunin, V., Makarov, M., Khamitov, I.M., Dostov, V.: Implementation of an E-voting scheme using Hyperledger Fabric permissioned blockchain. In: Computational Science and Its Applications – ICCSA 2019. LNCS, vol. 11620, pp. 509–521. Springer, Cham (2019)
34. McCorry, P., Shahandashti, S.F., Hao, F.: A smart contract for boardroom voting with maximum voter privacy. In: Financial Cryptography and Data Security. LNCS, vol. 10322, pp. 357–375. Springer, Cham (2017)
35. Merkle, R.C.: A digital signature based on a conventional encryption function. In: Conference on the theory and application of cryptographic techniques. pp. 369–378. Springer (1987)
36. National Institute of Standards and Technology: Secure hash standard (SHS). Federal Information Processing Standards Publication 180-4, NIST (2015). https://doi.org/10.6028/NIST.FIPS.180-4
37. Noack, J., Bormet, L., et al.: Shutter network: Private transactions from threshold cryptography. Cryptology ePrint Archive, Paper 2024/1981 (2024)
38. Optimism Collective: The OP Stack specification. Online specification (2026), https://specs.optimism.io
39. Polygon Labs: ZisK: A high-performance zkVM for RISC-V programs. GitHub repository, version 1.3.2-alpha (2026), https://github.com/0xPolygonHermez/zisk
40. Privacy and Scaling Explorations: LeanIMT: Lean incremental Merkle tree. zk-kit library (2024), https://github.com/privacy-scaling-explorations/zk-kit
41. Rarimo: Freedom tool: Privacy-preserving passport-based voting. Whitepaper, Rarimo (2024), https://docs.rarimo.com/freedom-tool/
42. Reuters: India's 2024 general election sees record participation with 642 million votes cast. https://www.reuters.com/world/india/india-poll-panel-says-642-mln-voters-cast-ballots-general-election-2024-06-03/ (2024), accessed 2026-01-22
43. Ryan, D., Feist, D., D'Amato, F., Wang, H.W., Stokes, A.: EIP-7594: PeerDAS – peer data availability sampling, Ethereum Improvement Proposals, no. 7594 (2024), https://eips.ethereum.org/EIPS/eip-7594
44. Succinct Labs: SP1: A performant, open-source zkVM. GitHub repository (2024), https://github.com/succinctlabs/sp1
45. Sutikno, S., Surya, A., Effendi, R.: An implementation of elgamal elliptic curves cryptosystems. In: IEEE. APCCAS 1998. 1998 IEEE Asia-Pacific Conference on Circuits and Systems. Microelectronics and Integrating Systems. Proceedings (Cat. No.98EX242). pp. 483–486 (1998). https://doi.org/10.1109/APCCAS.1998.743829
46. Vocdoni: arbo: Sparse Merkle tree compatible with the circomlib verifier. GitHub repository (2021), https://github.com/vocdoni/arbo
47. Vocdoni: DAVINCI contracts: EVM smart contracts. GitHub repository (2025), https://github.com/vocdoni/davinci-contracts

48. Vocdoni: DAVINCI circom: Ballot circuits. GitHub repository (2026), https://github.com/vocdoni/davinci-circom
49. Vocdoni: DAVINCI DKG council: Invite-only threshold key ceremonies. GitHub repository, release v0.3.0 (2026), https://github.com/vocdoni/davinci-dkg-council
50. Vocdoni: DAVINCI DKG: Permissionless threshold key generation and decryption. GitHub repository, release v0.11.0 (2026), https://github.com/vocdoni/davinci-dkg
51. Vocdoni: DAVINCI sequencer: Node and client library. GitHub repository, release v0.6.0 (2026), https://github.com/vocdoni/davinci-sequencer
52. Vocdoni: DAVINCI zkVM: Batch, results and aggregator programs for the ZisK zkVM. GitHub repository, release v0.3.0 (2026), https://github.com/vocdoni/davinci-zkvm
53. Waterman, A., Asanović, K.: The RISC-V instruction set manual, volume I: Unprivileged ISA. RISC-V Foundation, document version 20191213 (2019), https://riscv.org/specifications/
54. WhiteHat, B., Bellés, M., Baylina, J.: ERC-2494: Baby Jubjub elliptic curve [DRAFT], Ethereum Improvement Proposals, no. 2494, [online serial] (January 2020), https://eips.ethereum.org/EIPS/eip-2494
55. Wood, G., et al.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper **151**(2014), 1–32 (2014)

A Cryptographic primitives

In this section, we present the cryptographic primitives used in the protocol described in Section 4. We first introduce elliptic curves as the underlying algebraic setting, and then describe the following building blocks, each tied to a specific role in the system: hash functions and Merkle trees for commitments to structured data; polynomial commitments for data availability; digital signatures for voter authentication; encryption schemes for ballot confidentiality; DKG for decentralizing trust in the election keys; and zero-knowledge proofs (ZK-SNARKs and zkVMs) for verifiability of computations. Standard primitives are cited rather than re-derived; we spell out only the operations and instantiations specific to DAVINCI.

A.1 Elliptic curves

DAVINCI relies on multiple elliptic curves to ensure interoperability with Ethereum, compatibility with available cryptographic primitives, and efficient operations inside proofs. On the one hand, SECP256K1 [12] is used because Ethereum public keys are elements of this curve. As DAVINCI assumes each voter holds a standard Ethereum address, cryptographic operations such as digital signatures and identity verification rely on SECP256K1 to match the Ethereum ecosystem. On the other hand, BN254 [55] is chosen because it is the curve supported by Ethereum's precompiled contracts for pairing-based proof verification, which makes it the optimal choice for verifying SNARK proofs on-chain with minimal gas cost; both the ballot proofs and the PlonK wrapper of the zkVM proofs live on it. Then, BabyJubjub [5] is used as an inner curve for the encryption of ballots, since its base field is the scalar field of BN254 and its arithmetic is therefore native to the ballot circuit. Finally, BLS12-381 is the curve of the KZG commitments that Ethereum uses for data blobs [15]; DAVINCI only uses its scalar field, in which blob cells and evaluation points live. Below, we give the details of these curves.

Parameters.

- $p =$ `0xfffffffffffffffffffffffffffffffffffffffffffffffffffffffefffffc2f` (256-bit prime).
- $q =$ `0xfffffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141` (256-bit prime).
- $r =$ `0x30644e72e131a029b85045b68181585d97816a916871ca8d3c208c16d87cfd47` (254-bit prime).
- $s =$ `0x30644e72e131a029b85045b68181585d2833e84879b9709143e1f593f0000001` (254-bit prime).
- $t =$ `0x60c89ce5c263405370a08b6d0302b0bab3eedb83920ee0a677297dc392126f1` (251-bit prime).
- $u =$ `0x1a0111ea397fe69a4b1ba7b6434bacd764774b84f38512bf6730d2a0f6b0f624 1eabfffeb153ffffb9feffffffffaaab` (381-bit prime).
- $v =$ `0x73eda753299d7d483339d80809a1d80553bda402fffe5bfeffffffff00000001` (255-bit prime).

Elliptic curve groups.

- SECP256K1 curve: $E^{\mathsf{SEC}}/\mathbb{F}_p$ defined by equation $E^{\mathsf{SEC}} : y^2 = x^3 + 7$, with group $\mathbb{G}^{\mathsf{SEC}}$ of prime order q.
- BN254 curve: $E^{\mathsf{BN}}/\mathbb{F}_r$ defined by equation $E^{\mathsf{BN}} : y^2 = x^3 + 3$, with subgroups $\mathbb{G}_1^{\mathsf{BN}}, \mathbb{G}_2^{\mathsf{BN}}$ of prime order s, and an efficiently computable pairing $e : \mathbb{G}_1^{\mathsf{BN}} \times \mathbb{G}_2^{\mathsf{BN}} \to \mathbb{G}_T^{\mathsf{BN}}$, where $\mathbb{G}_T^{\mathsf{BN}} \subset \mathbb{F}_{r^{12}}$ and has order s.
- BabyJubjub curve: $E^{\mathsf{BJ}}/\mathbb{F}_s$ defined by the twisted Edwards equation $E^{\mathsf{BJ}} : 168700x^2 + y^2 = 1 + 168696x^2y^2$, of order $8t$, with subgroup $\mathbb{G}^{\mathsf{BJ}}$ of prime order t. Its identity is the point $(0, 1)$, and the inverse of (x, y) is $(-x, y)$.
- BLS12-381 curve: $E^{\mathsf{BLS}}/\mathbb{F}_u$ defined by equation $E^{\mathsf{BLS}} : y^2 = x^3 + 4$, with subgroups $\mathbb{G}_1^{\mathsf{BLS}}, \mathbb{G}_2^{\mathsf{BLS}}$ of prime order v, and an efficiently computable pairing $e : \mathbb{G}_1^{\mathsf{BLS}} \times \mathbb{G}_2^{\mathsf{BLS}} \to \mathbb{G}_T^{\mathsf{BLS}}$, where $\mathbb{G}_T^{\mathsf{BLS}} \subset \mathbb{F}_{u^{12}}$ and has order v. In the body of the paper, we write $q_{\mathsf{BLS}} = v$.

Finite fields.

- $\mathbb{F}_p$: base field of the SECP256K1 curve.
- $\mathbb{F}_q$: scalar field of $\mathbb{G}^{\mathsf{SEC}}$.
- $\mathbb{F}_r$: base field of the BN254 curve.
- $\mathbb{F}_s$: scalar field of $\mathbb{G}_1^{\mathsf{BN}}, \mathbb{G}_2^{\mathsf{BN}}, \mathbb{G}_T^{\mathsf{BN}}$ and base field of the BabyJubjub curve. Ballot coordinates, Poseidon inputs and outputs, and census leaves are elements of $\mathbb{F}_s$.
- $\mathbb{F}_t$: scalar field of $\mathbb{G}^{\mathsf{BJ}}$, to which election secret keys belong.
- $\mathbb{F}_u$: base field of the BLS12-381 curve.
- $\mathbb{F}_v$: scalar field of $\mathbb{G}_1^{\mathsf{BLS}}, \mathbb{G}_2^{\mathsf{BLS}}, \mathbb{G}_T^{\mathsf{BLS}}$, to which blob cells and evaluation points belong.

Generators.

We denote by G^{SEC} the generator of $\mathbb{G}^{\mathsf{SEC}}$ as defined in [12], G^{BN} the generator of $\mathbb{G}_1^{\mathsf{BN}}$ as defined in [55], G^{BJ} the generator of $\mathbb{G}^{\mathsf{BJ}}$ as defined in [54] (the point usually called *Base8*), and G^{BLS} the standard generator of $\mathbb{G}_1^{\mathsf{BLS}}$ [15].

- $G^{\mathsf{SEC}} = (\texttt{0x79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798}, \texttt{0x483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8})$.
- $G^{\mathsf{BN}} = (\texttt{0x01}, \texttt{0x02})$.
- $G^{\mathsf{BJ}} = (\texttt{0x0bb77a6ad63e739b4eacb2e09d6277c12ab8d8010534e0b62893f3f6bb957051}, \texttt{0x25797203f7a0b24925572e1cd16bf9edfce0051fb9e133774b3c257a872d7d8b})$.
- $G^{\mathsf{BLS}} = (\texttt{0x17f1d3a73197d7942695638c4fa9ac0fc3688c4f9774b905a14e3a3f171bac586c55e83ff97a1aeffb3af00adb22c6bb}, \texttt{0x08b3f481e3aaa0f1a09e30ed741d8ae4fcf5e095d5d00af600db18cb2c04b3edd03cc744a2888ae40caa232946c5e7e1})$.

A.2 Hash functions

DAVINCI uses three hash functions: Keccak256 [10] for Ethereum compatibility, SHA-256 [36] for the state tree and every hash computed by the zkVM programs and verified by smart contracts, and Poseidon [26] for every hash computed by the ballot circuit.

Keccak256. Keccak256 is the standard hash function used by Ethereum and is employed in DAVINCI to compress messages for signing, to derive Ethereum addresses from public keys over $\mathbb{G}^{\mathsf{SEC}}$, and to hash Ethereum block headers when the registry verifies blob transactions in L1-blob mode. It maps arbitrary-length bitstrings to 256-bit digests, $\mathsf{Keccak} : \{0,1\}^* \to \{0,1\}^{256}$. Inside the zkVM, it is used only by the authentication and CSP checks of the batch program (see Section 4.5.2), where verifying Ethereum-compatible signatures requires reproducing the message hash computed by Ethereum clients.

SHA-256. SHA-256 maps arbitrary-length bitstrings to 256-bit digests, $\mathsf{SHA256} : \{0,1\}^* \to \{0,1\}^{256}$. It is a precompile of the EVM and an accelerated operation of the ZisK zkVM, which makes it the natural choice for every value that is computed in a zkVM program and checked by a contract: the nodes and leaves of the state tree, the digests of ballots, accumulators, and verification keys stored in it, the derivation of ballot slots, the chain of re-encryption scalars, the evaluation points of the blobs and the digest of their openings, the transcript of the batched ballot proof verification, and the digest of the public values of a zkVM proof. When a digest must be interpreted as a field element, it is read as a big-endian integer and reduced modulo the field order, unless stated otherwise.

Poseidon. Poseidon is a SNARK-optimized hash function designed for efficient evaluation inside arithmetic circuits. It is used in DAVINCI for computing vote identifiers, the commitment to the ballot circuit's inputs, the per-field encryption randomness of a ballot, the nodes of the census tree, and the challenges of decryption proofs. We write $\mathsf{Poseidon} : \mathfrak{F}_s \to \mathbb{F}_s$, where $\mathfrak{F}_s$ denotes tuples of $\mathbb{F}_s$-elements of any length and $\mathbb{F}_s$ is the field described in Section A.1. A single Poseidon permutation accepts at most 16 inputs, so longer tuples are hashed in two levels: the tuple is split into consecutive chunks of at most 16 elements, each chunk is hashed, and the chunk digests are hashed together. For a ballot of 16 fields, the commitment `inputsHash` covers 71 elements (the process identifier, the packed ballot mode, the two coordinates of the key, the address, the vote identifier, the 64 coordinates of the ciphertexts, and the weight), so it is computed as

$$\begin{aligned}\texttt{inputsHash} = \mathsf{Poseidon}\big(&\mathsf{Poseidon}(x_{1..16}), \mathsf{Poseidon}(x_{17..32}), \mathsf{Poseidon}(x_{33..48}),\\ &\mathsf{Poseidon}(x_{49..64}), \mathsf{Poseidon}(x_{65..71})\big).\end{aligned}$$

A.3 Merkle trees

DAVINCI uses two Merkle-tree variants as hash-based accumulators: a *sparse Merkle tree* (SMT) for the election state, and a *lean incremental Merkle tree* (IMT) for the census. Only the root of each tree is published on-chain, so that large datasets can be committed and verified with minimal on-chain storage.

Sparse Merkle tree (state tree). Following [2], the state tree is a binary tree of maximum depth 64 in which each leaf encodes a key–value pair and empty subtrees are represented by a constant, supporting both inclusion and non-inclusion proofs. The key determines a unique root-to-leaf path independent of insertion order, which makes membership and emptiness checks cheap to verify inside a proof. As in the circomlib construction, a leaf is placed at the shallowest level at which its path is unique, and a transition is proven by the processor of [2], which checks the old root and computes the new one from the same list of siblings, for insertions, updates, and deletions. DAVINCI instantiates the tree with SHA-256, with leaf hash $\mathsf{SHA256}(\kappa_{\mathrm{LE8}} \,||\, v_{\mathrm{LE32}} \,||\, \texttt{0x01})$ and node hash $\mathsf{SHA256}(L \,||\, R)$, as implemented by the arbo library [46]. Its key space is partitioned into disjoint regions for configuration parameters, encrypted ballots, and vote identifiers (see Section 4.4).

Lean incremental Merkle tree (census tree). The census is maintained as a lean IMT [40], a binary tree optimized for sequential insertion: voters are appended at consecutive leaf positions rather than at key-derived paths, so the tree grows only as needed and appending a voter recomputes hashes along a single path. A node without a right sibling is promoted to the next level without hashing, so a proof contains only the non-empty siblings of the path. Each leaf stores a voter's identifier and weight; concretely, the Ethereum address and weight packed into a single field element (see Section 4.3). Nodes are hashed with two-input Poseidon over $\mathbb{F}_s$, and only the root `censusRoot` is published on-chain. The batch program accepts proofs of at most 61 siblings.

Tree operations. To support state transitions and in-circuit verification, the trees expose the following operations. For the state tree $\mathsf{MT}^{\mathtt{state}}$, indexed by keys κ:

- $\mathsf{MT}^{\mathtt{state}}.\mathsf{Root}()$: returns the current Merkle root of the tree.
- $\mathsf{MT}^{\mathtt{state}}.\mathsf{Insert}(\kappa, \mathsf{v})$: stores the value v under the key κ, which must be empty. A transition proof of an insertion shows that κ was empty under the old root.
- $\mathsf{MT}^{\mathtt{state}}.\mathsf{Update}(\kappa, \mathsf{v})$: replaces the value stored under the existing key κ with v. A transition proof of an update shows the replaced value under the old root.
- $\mathsf{MT}^{\mathtt{state}}.\mathsf{Verify}(\mathsf{v}, \kappa, \mathsf{Proof}, \mathsf{root})$: returns `true` if and only if Proof shows that v is stored under κ in the tree with root root.

For the census tree $\mathsf{MT}^{\mathtt{census}}$, whose leaves are indexed by their position:

- $\mathsf{MT}^{\mathtt{census}}.\mathsf{Verify}(\mathsf{leaf}, \mathtt{idx}, \mathsf{Proof}, \mathsf{root})$: returns `true` if and only if Proof shows that leaf is stored at position `idx` in the tree with root root. The position is carried by the proof; as discussed in Section 2.3, it is not authenticated.

These operations let sequencers evolve the state off-chain while keeping every update independently verifiable on-chain and within ZK proofs. Additional details on the tree structures and their leaf encodings are given in Sections 4.3 and 4.4.

A.4 Commitment scheme

DAVINCI commits to the data published in each Ethereum blob using the KZG polynomial commitment scheme [32] over BLS12-381, the same scheme that underlies EIP-4844; it therefore reuses Ethereum's structured reference string rather than requiring a dedicated ceremony. A blob of $\ell = 4096$ cells $d_0, \ldots, d_{\ell-1} \in \mathbb{F}_v$ defines the unique polynomial P of degree less than ℓ with $P(\omega_i) = d_i$, where ω_i are the ℓ-th roots of unity of $\mathbb{F}_v$ in bit-reversed order. Its value at a point z outside the domain is given by the barycentric formula

$$P(z) = \frac{z^\ell - 1}{\ell} \sum_{i=0}^{\ell-1} \frac{d_i\, \omega_i}{z - \omega_i},$$

which the batch program evaluates with $O(\ell)$ multiplications and one batched inversion, without any pairing. The sequencer commits to P and proves a single evaluation at a point derived from the commitment itself, which lets the registry check, with one call to the point-evaluation precompile, that the blob published on-chain is the one whose contents the batch program derived (see Section 4.5.2). We use two operations:

- $\mathsf{C.Commit}(P)$: outputs a succinct commitment $C \in \mathbb{G}_1^{\text{BLS}}$ to the polynomial P, encoded in 48 bytes. Its *versioned hash*, $\texttt{0x01} \,\|\, \mathsf{SHA256}(C)_{[1..32]}$, is what a blob-carrying transaction exposes.
- $\mathsf{C.Verify}(C, z, y, \pi)$: returns `true` if and only if π is a valid proof that $P(z) = y$ for the polynomial committed in C.

Because the evaluation point is a hash of the commitment, the process identifier, and the old state root, a sequencer cannot choose the commitment after the point, and a commitment prepared for another election or another state does not open to the value the batch program computed. By the Schwartz–Zippel lemma, two distinct blobs agree at a random point with probability at most $\ell/v \approx 2^{-243}$.

A.5 Digital signature scheme

DAVINCI uses the elliptic curve digital signature algorithm (ECDSA) [30] over the SECP256K1 curve, so that voters authenticate with standard Ethereum wallets without managing additional keys. We use ECDSA in its standard form and refer to [30] for the full algorithms; here we only fix the notation used throughout the protocol:

- $\mathsf{S.Sign}_{\mathsf{sk}}(\mathsf{message})$: produces a signature σ on message message under the secret key sk; in Ethereum's 65-byte format, σ carries the two ECDSA scalars and a recovery identifier.
- $\mathsf{S.Verify}_{\mathsf{pk}}(\mathsf{message}, \sigma)$: returns `true` if σ is a valid signature on message under the public key pk.
- $\mathsf{S.ExtractPublicKey}(\mathsf{message}, \sigma)$: recovers the signer's public key pk from a message and its signature; this is the operation Ethereum exposes as `ecrecover`.

An Ethereum address is derived from a public key as the low-order 20 bytes of its Keccak digest, $\texttt{address} = \mathsf{low\text{-}20}\big(\mathsf{Keccak}(\mathsf{pk})\big)$. Messages are signed in the Ethereum personal-message format: the signed digest of a message message of ℓ bytes is $\mathsf{Keccak}(\texttt{"\textbackslash x19Ethereum Signed Message:\textbackslash n"} \,\|\, \ell \,\|\, \mathsf{message})$, with ℓ written in decimal. A voter signs the 32-byte message formed by 24 zero bytes followed by the 8-byte big-endian vote identifier, and a CSP signs the 92-byte message of Eq. (2). Inside the zkVM, signatures are verified by recovering the public key with a SECP256K1 precompile and comparing its address.

A.6 Encryption scheme

To preserve ballot secrecy while enabling tallying, DAVINCI employs a threshold variant of the ElGamal cryptosystem instantiated over the BabyJubjub curve [45]. This scheme offers two properties crucial for the protocol: *additive homomorphism*, which allows the aggregation of encrypted votes without decryption, and *re-encryption*, which enables ciphertext randomization without altering the underlying plaintext. Together, these properties allow sequencers to tally votes while preventing voters from producing receipts that could be used for coercion or vote selling. The corresponding public key is generated collectively by the key wardens via the distributed key generation protocol of Section A.7, ensuring that no single entity can decrypt ballots unilaterally, or, in the sequencer key mode, by a single sequencer.

Encryption and decryption. Given a message m and randomness k, the ElGamal encryption algorithm maps m into a group element M of $\mathbb{G}^{\text{BJ}}$ and outputs a ciphertext as follows. In DAVINCI, k is an element of $\mathbb{F}_s$ used as a scalar, which the group arithmetic implicitly reduces modulo t.

- $\mathsf{Enc}_{\mathsf{pk}}(m; k)$:
  1. Map m to $\mathbb{G}^{\text{BJ}}$ via $M = m \cdot G^{\text{BJ}}$.
  2. Compute $C_1 = k \cdot G^{\text{BJ}} \in \mathbb{G}^{\text{BJ}}$.
  3. Compute $C_2 = k \cdot \mathsf{pk} + M \in \mathbb{G}^{\text{BJ}}$.
  4. Output $\mathsf{ciphertext} = (C_1, C_2) \in (\mathbb{G}^{\text{BJ}})^2$.
- $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ciphertext})$:
  1. Parse $\mathsf{ciphertext} = (C_1, C_2)$.
  2. Compute $K = \mathsf{sk} \cdot C_1 \in \mathbb{G}^{\text{BJ}}$.
  3. Output $M = C_2 - K$.

Since the plaintext message space is small, recovering m from M is efficient: although the mapping $m \mapsto M$ is not generally invertible, it can be reversed by brute-force search or optimized techniques such as baby-step giant-step [11], in time proportional to the square root of the bound on m. The registry bounds every tally by $10^{12} < 2^{40}$, so each field is recovered with about 2^{20} group operations.

Homomorphic addition and re-encryption. ElGamal encryption is additively homomorphic. Given two ciphertexts (C_1, C_2) and (C_1', C_2'), their component-wise sum $(C_1 + C_1',\ C_2 + C_2')$ is a valid ciphertext that decrypts to the sum of the two underlying messages, which allows sequencers to aggregate encrypted ballots directly; adding the inverse of a ciphertext subtracts its message, which the batch program uses to remove overwritten ballots from the accumulator. Re-encryption exploits the same property by adding an encryption of zero to a ciphertext, yielding a fresh ciphertext of the same message under new randomness and making it computationally infeasible to link the re-encrypted ballot with the original submission.

- $\mathsf{EncAdd}_{\mathsf{pk}}(\mathsf{ciphertext} \in (\mathbb{G}^{\text{BJ}})^2, \mathsf{ciphertext}' \in (\mathbb{G}^{\text{BJ}})^2)$:
  1. Parse $\mathsf{ciphertext} = (C_1, C_2) \in (\mathbb{G}^{\text{BJ}})^2$.
  2. Parse $\mathsf{ciphertext}' = (C_1', C_2') \in (\mathbb{G}^{\text{BJ}})^2$.
  3. Output $(C_1 + C_1', C_2 + C_2') \in (\mathbb{G}^{\text{BJ}})^2$.
- $\mathsf{ReEnc}_{\mathsf{pk}}(\mathsf{ciphertext} \in (\mathbb{G}^{\text{BJ}})^2; k)$:
  1. Parse $\mathsf{ciphertext} = (C_1, C_2) \in (\mathbb{G}^{\text{BJ}})^2$.
  2. Compute $\mathsf{ciphertext}' = \mathsf{Enc}_{\mathsf{pk}}(0; k)$.
  3. Output $\mathsf{EncAdd}_{\mathsf{pk}}(\mathsf{ciphertext}, \mathsf{ciphertext}')$.

In the protocol, re-encryption is applied not only to newly submitted ballots but also to ballots already stored in the state tree. By refreshing the randomness of both new and existing ciphertexts, sequencers ensure that it is indistinguishable whether a ballot has been overwritten or merely re-randomized. This mechanism is essential for receipt-freeness: voters cannot produce a verifiable receipt of their choice, and adversaries cannot detect or prove whether a particular vote has been replaced (see Section 6). In the batch program, the randomness of each re-encryption is a scalar of the chain of Section 4.5.2, a SHA-256 output reduced modulo s.

Proof of correct decryption. To publish a decryption without revealing the secret key, the key holder proves that the decrypted point is consistent with the public key using a Chaum–Pedersen proof of equality of discrete logarithms [17], made non-interactive with the Fiat–Shamir transform [22]. For a ciphertext (C_1, C_2) and a claimed message m, let $D = C_2 - m \cdot G^{\text{BJ}}$; the statement is that $\log_{G^{\text{BJ}}} \mathsf{pk} = \log_{C_1} D$, which holds exactly when m is the decryption of the ciphertext under sk.

- $\mathsf{CP.Prove}_{\mathsf{sk}}(\mathsf{ciphertext}, m)$:
  1. Sample $w \xleftarrow{\$} \mathbb{F}_t$.
  2. Compute $A_1 = w \cdot G^{\text{BJ}}$, $A_2 = w \cdot C_1$.
  3. Compute $e = \mathsf{Poseidon}(\hat{\mathsf{pk}}, \hat{\mathsf{pk}}, \hat{C}_1, \hat{D}, \hat{A}_1, \hat{A}_2)$.
  4. Output $\pi = (A_1, A_2, z = w + e \cdot \mathsf{sk} \bmod t)$.
- $\mathsf{CP.Verify}_{\mathsf{pk}}(\mathsf{ciphertext}, m, \pi)$:
  1. Recompute D and e.
  2. Check $z \cdot G^{\text{BJ}} = A_1 + e \cdot \mathsf{pk}$.
  3. Check $z \cdot C_1 = A_2 + e \cdot D$.

The challenge is a single 12-input Poseidon permutation over the coordinates of the points, each hashed in the *reduced* twisted Edwards form used by gnark, $\hat{P} = (-f \cdot x \bmod s,\ y)$ for $P = (x, y)$, with $f = 6360561867910373094066688120553762416144456282423235903351243436111059670888$; the public key appears twice, as in the reference implementation. The results program additionally requires all points to be on the curve and all coordinates and scalars to be canonical, so that each statement has a single encoding.

A.7 Distributed key generation scheme

To ensure that no single entity controls the decryption key, DAVINCI employs a DKG protocol to jointly derive the threshold ElGamal encryption key pair used for ballot encryption. Participants who contribute to the ceremony are called *key wardens*. The outcome is a collective public key (`encryptionKey`) that is published on-chain and used by voters to encrypt their ballots, while the corresponding private key is secret-shared among the key wardens so that only a threshold θ out of ν of them can later collaborate to decrypt the tally. Unlike classical DKG protocols in which shares are exchanged in the clear, our construction distributes encrypted shares together with ZK-SNARK proofs of correctness, so that the key-management smart contracts can verify each contribution without learning any of the underlying secrets. We use the non-interactive distributed key generation protocol of [31], in which each dealer publishes a Feldman verifiable secret sharing [21] of its contribution, with every share encrypted to its recipient and a Groth16 proof that the dealing is consistent, and we refer the reader to that work for the full construction and its security analysis. The permissionless DKG network deals sixteen independent keys per run, so that one run serves sixteen elections, and the invite-only council deals a single key per ceremony; both decrypt by collecting θ partial decryptions $D_i = \mathsf{sh}_i \cdot C_1$, each with a proof of consistency with the committed share sh_i, and combining them by Lagrange interpolation in the exponent.

A.8 Zero-knowledge proof systems

Zero-knowledge proofs are the backbone of DAVINCI's integrity guarantees: voters prove that their encrypted ballots are well-formed and rule-compliant without revealing their choices, and sequencers prove that ballot verification, vote aggregation, and state transitions were carried out correctly.

ZK-SNARKs. For a circuit $\mathcal{C}$ with public inputs PI and witness $\mathsf{witness}$, we write:

- $\mathsf{P.Prove}(\mathcal{C}, \mathsf{witness}, \mathsf{PI})$: outputs a proof π that the prover knows a witness satisfying $\mathcal{C}$ for the given public inputs, using the circuit's proving key.
- $\mathsf{P.Verify}(\pi, \mathsf{PI})$: returns `true` if and only if π is valid for PI, using the circuit's verification key.

The ballot circuit uses the Groth16 proof system [27] over BN254, chosen for its constant-size proofs and cheap verification. A Groth16 proof $(A, B, C) \in \mathbb{G}_1^{\mathsf{BN}} \times \mathbb{G}_2^{\mathsf{BN}} \times \mathbb{G}_1^{\mathsf{BN}}$ for public inputs x is valid if $e(A, B) = e(\alpha, \beta) \cdot e(L_x, \gamma) \cdot e(C, \delta)$, where $\alpha, \beta, \gamma, \delta$ belong to the verification key and L_x is a linear combination of its elements with coefficients x. The batch program verifies n such proofs at once by checking a single random linear combination of their equations [3], with 128-bit coefficients derived by hashing all the proofs and public inputs; an invalid proof passes with probability at most 2^{-128}.

Zero-knowledge virtual machines. For a guest program $\mathcal{P}$, compiled to RISC-V, and a private input $\mathsf{witness}$, we write:

- $\mathsf{VM.Prove}(\mathcal{P}, \mathsf{witness})$: executes $\mathcal{P}$ on $\mathsf{witness}$ and outputs the public values `publicValues` it commits and a proof π of the execution.
- $\mathsf{VM.Verify}(\texttt{programVK}, \texttt{publicValues}, \pi)$: returns `true` if and only if π proves that the program identified by `programVK`, executed on some input, committed `publicValues`.

DAVINCI uses the ZisK zkVM [39]. The execution is proven with FRI-based STARKs over the Goldilocks field, aggregated recursively into a single VADCOP-final STARK, and wrapped in a PlonK proof over BN254 [23], whose single public input is $\mathsf{SHA256}(\texttt{programVK} \,||\, \texttt{publicValues} \,||\, \texttt{rootCVadcopFinal}) \bmod s$. The program key `programVK` is the root of a commitment to the compiled program, and `rootCVadcopFinal` identifies the setup of the VADCOP-final circuit. The STARK part is transparent and relies only on collision-resistant hashing, while the PlonK wrapper relies on a universal structured reference string and on the hardness of discrete logarithms in the BN254 groups. Further details on the programs are provided in Section 4.5 and a summary of the instantiations can be found in Fig. 4.